

codesign-mcdp: A Python Library for Monotone Co-Design Problems

Version 0.2.0 – Reference Manual

July 2026

Abstract

`codesign-mcdp` is a Python library for formulating and solving *Monotone Co-Design Problems* (MCDPs) in the framework of Censi (2015). A design problem is a relation between two posets, a functionality poset F and a resource poset R ; given a target functionality, the problem asks for the antichain of minimal resources needed to deliver it. Design problems compose under three operators (series, parallel, feedback), and the resulting class is closed under composition. The library implements the antichain calculus, six primitive design-problem types, the three composition operators, a Kleene fixed-point solver, and two high-level builders (an MCDPL-style declarative builder and a modular `System` builder). Further layers add set-based and stochastic uncertainty, compositional online learning, and temporal, vector-state, and online co-design, alongside a suite of worked examples. This document is the reference manual for version 0.2.0.

Contents

1	Introduction	4
2	Mathematical Background	4
2.1	Posets and Lattices	4
2.2	Antichains as a Lattice	5
2.3	Design Problems	5
2.4	Composition	5
2.5	The Kleene Fixed-Point Algorithm	6
3	Installation and Project Layout	6
4	Core Data Types	7
4.1	Posets	7
4.2	Antichains	8
5	Primitive Design Problem Types	8
5.1	AlgebraicDP	8
5.2	FunctionDP	9
5.3	CatalogDP	9
5.4	ConstraintDP	10
5.5	ODE_DP	10
5.6	UncertainDP	11

6	Composition Operators	11
6.1	Series	12
6.2	Parallel	12
6.3	Loop	12
7	The Solver	13
7.1	Top-level entry point	13
7.2	The Kleene iteration	14
7.3	Scalar minimisation over an antichain	14
7.4	Observability: trace, verbose, callback, status	15
8	Uncertainty	16
8.1	Declaration	16
8.2	Set-based uncertainty (deterministic)	17
8.3	Stochastic uncertainty	17
8.4	Querying	18
9	Online Learning	19
9.1	Optimistic evaluators	19
9.2	The online solver	20
9.3	Warm-start, picker strategies, and the GP evaluator	21
10	Visualisation	22
10.1	Block diagrams of Systems	23
11	The MCDP Builder	24
12	The System Builder	25
12.1	Operator-overloaded syntax (recommended)	25
12.2	Legacy lambda syntax	26
12.3	Class-based modules	26
12.4	Semantics	27
12.5	Validation	28
13	API Reference	28
13.1	Order structures	28
13.1.1	<code>posets</code>	28
13.1.2	<code>antichains</code>	31
13.2	Design problems and solving	32
13.2.1	<code>dp</code>	32
13.2.2	<code>primitives</code>	35
13.2.3	<code>composition</code>	36
13.2.4	<code>solver</code>	37
13.3	Builders and rendering	40
13.3.1	<code>mcdpl</code>	40
13.3.2	<code>system</code>	41
13.3.3	<code>module</code>	42
13.3.4	<code>sugar</code>	43
13.3.5	<code>diagram</code>	44
13.3.6	<code>viz</code>	45
13.4	Uncertainty and online learning	47
13.4.1	<code>uncertainty</code>	47
13.4.2	<code>online</code>	51

13.5	Temporal layers	54
13.5.1	<code>temporal</code>	55
13.5.2	<code>dynamic</code>	57
13.5.3	<code>sequential</code>	61
13.6	State and closed loop	67
13.6.1	<code>state</code>	67
13.6.2	<code>vector_dp</code>	70
13.6.3	<code>online_codesign</code>	73
14	Worked Examples	75
14.1	Example 1: the drone (Fig. 48 of the paper)	76
14.2	Example 2: integer optimisation (Sec. VI-D)	77
14.3	Example 3: AUV seabed surveying (Sec. VIII)	78
14.4	Example 4: <code>UncertainDP</code> and <code>ODE_DP</code>	79
14.5	Example 5: visualising the Kleene ascent	80
14.6	Example 6: the drone in MCDPL syntax	81
14.7	Example 7: the drone, modular	82
14.8	Example 8: a modular vehicle with Pareto tradeoffs	84
14.9	Example 9: a robotic arm with non-trivial topology	85
14.10	Example 10: watching the solver work	86
14.11	Example 11: set-based deterministic uncertainty	87
14.12	Example 12: stochastic uncertainty with a Gaussian copula	88
14.13	Example 13: the microgrid flagship	89
14.14	Example 14: online elimination over a robot catalogue	90
14.15	Example 15: monoclonal antibody fed-batch co-design	91
14.16	Example 16: online DOE for the mAb fed-batch process	92
14.17	Example 17: full-vehicle co-design across ICE, hybrid, and EV	93
15	Temporal, Vector-State, and Online Co-Design	95
15.1	Case 1: architecture switching (<code>temporal</code>)	95
15.2	Case 2, scalar value: dynamic programming (<code>dynamic</code>)	95
15.3	Case 2, antichain value: sequential co-design (<code>sequential</code>)	96
15.4	The general carried state: vectors (<code>state</code> , <code>vector_dp</code>)	96
15.5	Precompute-then-DP: the Formula 1 structure	97
15.6	Online feedback co-design (<code>online_codesign</code>)	97
15.7	Example 18: metabolic architecture switching	97
15.8	Example 19: rover module activation	98
15.9	Example 20: antichain-valued sequential co-design	99
15.10	Example 21: reconfigurable robot with a state vector	100
15.11	Example 22: online feedback co-design	101
15.12	Example 23: hierarchical co-design for a Formula 1 season	102
15.13	Example 24: catalog-driven car co-design from one table	104
15.14	Example 25: online co-design paper benchmarks	105
16	Realistic Problem Domains for Temporal Co-Design	107
16.1	Self-reconfiguring and modular robots	107
16.2	Programmable self-assembly	108
16.3	Multi-objective protein design	108
16.4	Control co-design (CCD)	108
16.5	Summary of the mapping	108
17	Modelling Guidelines and Pitfalls	109

18 Limitations and Future Work	109
19 API Reference Summary	110
References	111

1 Introduction

A *co-design problem* is the joint design of components whose specifications mutually depend on each other. A canonical example: a battery must store enough energy to power an actuator, but the actuator must lift the battery itself, so the battery’s mass appears in the energy it has to supply. There is no way to fix one without fixing the other.

Censi (2015) showed that under modest monotonicity assumptions, such problems have a clean algebraic theory: the relations between resources and functionalities form a category closed under series, parallel, and feedback composition, and solutions are computed by a Kleene fixed-point iteration in the lattice of antichains. The original publication included a software tool, MCDPL, distributed through `co-design.science`, which exposes the framework via a domain-specific language.

`codesign-mcdp` is a from-scratch Python implementation of the algorithmic core, designed around composable Python objects rather than a separate DSL. It targets users who want to formulate and solve MCDPs from inside their existing Python codebase, without context switching to a DSL or an external solver. The algorithmic core (posets, antichains, primitive design problems, composition, solver, and both builders) has no required runtime dependencies beyond the standard library; the uncertainty, online-learning, and visualisation layers pull in `numpy`, `scipy`, and `matplotlib` as optional extras. The whole package is roughly 8000 lines of Python and is licensed under MIT.

Scope of this document. This manual covers the public API and the mathematical structure it implements. Section 2 recalls the formal background. Sections 4 through 7 describe the data types, primitive design problems, composition operators, and the solver in detail. Section 8 covers set-based and stochastic uncertainty, and Section 9 the online-learning layer; Section 10 documents the visualisation helpers. Sections 11 and 12 document the two high-level builders. Section 13 is a complete, per-module reference for every public symbol in the library. Section 14 walks through every example shipped with the library. Section 15 introduces the temporal, vector-state, and online co-design layers, and Section 16 surveys realistic problem domains for them. Section 17 collects modelling guidelines and common pitfalls, and Section 18 discusses limitations and future directions.

Conventions. Code listings are typeset in monospaced font. Type signatures use Python’s standard notation ($f: F \rightarrow R$, for example). Mathematical quantities use F , R , h , $\top$, $\perp$, Min , etc.

2 Mathematical Background

This section recalls the formal definitions from Censi (2015) needed to specify the API. Proofs are omitted; the reader is referred to the paper for the full development.

2.1 Posets and Lattices

Definition 2.1 (Poset). A poset (*partially ordered set*) is a pair $(P, \leq)$ where $\leq$ is a reflexive, antisymmetric, transitive binary relation on the set P .

Definition 2.2 (Bottom and top). An element $\perp \in P$ is a bottom (least element) if $\perp \leq x$ for every $x \in P$. An element $\top \in P$ is a top (greatest element) if $x \leq \top$ for every $x \in P$. A poset has at most one bottom and at most one top.

Example 2.3 (Reals and Naturals). The non-negative reals augmented with infinity, $\overline{\mathbb{R}_{\geq 0}} = \mathbb{R}_{\geq 0} \cup \{+\infty\}$, ordered by the usual $\leq$, form a poset with $\perp = 0$ and $\top = +\infty$. Likewise $\overline{\mathbb{N}} = \mathbb{N} \cup \{+\infty\}$.

Example 2.4 (Product order). Given posets $(P_1, \leq_1), \dots, (P_n, \leq_n)$, the product $P_1 \times \dots \times P_n$ with the componentwise order

$$(x_1, \dots, x_n) \leq (y_1, \dots, y_n) \iff \forall i. x_i \leq_i y_i$$

is a poset. The bottom is $(\perp_1, \dots, \perp_n)$ and the top is $(\top_1, \dots, \top_n)$.

Definition 2.5 (Antichain). A subset $A \subseteq P$ is an antichain if no two distinct elements of A are comparable: $\forall x, y \in A, x \leq y \implies x = y$. We write $\mathbf{A}[P]$ for the set of all antichains of P .

Definition 2.6 (Min operator). For any subset $S \subseteq P$, the Min operator returns the set of minimal elements:

$$\text{Min}(S) = \{x \in S : \nexists y \in S. y < x\}.$$

$\text{Min}(S)$ is always an antichain. For a finite S , $\text{Min}(S)$ is the Pareto front of S .

2.2 Antichains as a Lattice

The set of antichains of a poset carries a natural order: $A \leq A'$ in $\mathbf{A}[P]$ iff every point in A' is dominated by some point in A . Intuitively, $A \leq A'$ means A is "lower" or "easier", having weaker Pareto-minimal demands.

Proposition 2.7. $(\mathbf{A}[P], \leq)$ is a complete lattice when P is finite or chain-complete. Joins are computed by union followed by Min. The bottom of $\mathbf{A}[P]$ is $\{\perp_P\}$ (or $\emptyset$ if P has no bottom); the top is $\emptyset$.

This lattice is the state space of the Kleene iteration (Section 6.3).

2.3 Design Problems

Definition 2.8 (Design Problem). A design problem (DP) is a tuple $\langle F, R, h \rangle$ where F and R are posets and $h : F \rightarrow \mathbf{A}[R]$ is a function from functionality requests to antichains of resources. $h(f)$ is interpreted as the Pareto front of minimal resources required to deliver functionality f .

Definition 2.9 (Monotone Design Problem). A DP $\langle F, R, h \rangle$ is monotone (an MCDP) if h is monotone with respect to the antichain order: $f \leq f'$ in F implies $h(f) \leq h(f')$ in $\mathbf{A}[R]$.

Intuitively, monotonicity says that demanding more functionality cannot reduce the minimal resources needed. This corresponds to the common-sense engineering principle that you don't get more for less.

2.4 Composition

The class of MCDPs is closed under three operators.

Definition 2.10 (Series). Given DPs $d_1 = \langle F, M, h_1 \rangle$ and $d_2 = \langle M, R, h_2 \rangle$ whose interface poset M matches, their series composition is

$$(d_1 \circ d_2)(f) = \text{Min} \left(\bigcup_{r_1 \in h_1(f)} h_2(r_1) \right).$$

Definition 2.11 (Parallel). Given DPs $d_1 = \langle F_1, R_1, h_1 \rangle$ and $d_2 = \langle F_2, R_2, h_2 \rangle$, their parallel composition is $\langle F_1 \times F_2, R_1 \times R_2, h_1 \otimes h_2 \rangle$ where

$$(h_1 \otimes h_2)(f_1, f_2) = \text{Min}(h_1(f_1) \times h_2(f_2)).$$

Definition 2.12 (Feedback). Given a DP $d = \langle F \times X, R \times X, h \rangle$ with a common factor X , the feedback composition on X is the DP $d^{\dagger X} = \langle F, R, h^{\dagger X} \rangle$ where $h^{\dagger X}(f)$ is the antichain obtained by closing the recursion on X .

The recursion is solved by Kleene iteration in $\mathbf{A}[R \times X]$, seeded at $\{\perp\}$ and ascending until a fixed point is reached.

2.5 The Kleene Fixed-Point Algorithm

Theorem 2.13 (Censi 2015, Prop. 4). For an MCDP $d = \langle F \times X, R \times X, h \rangle$ closed on X , define the operator $\Phi_f : \mathbf{A}[R \times X] \rightarrow \mathbf{A}[R \times X]$ by

$$\Phi_f(A) = \text{Min} \left(\bigcup_{r \in A} h(f, r_X) \cap \uparrow r \right),$$

where r_X is the X -component of r and $\uparrow r$ is the upward closure. Then Φ_f is monotone, and the Kleene ascent

$$A_0 = \{\perp\}, \quad A_{k+1} = \Phi_f(A_k)$$

converges to the least fixed point, which equals $h^{\dagger X}(f)$ projected to $\mathbf{A}[R]$.

In implementation, the iteration terminates when one of three conditions holds: $A_{k+1} = A_k$ (fixed point), $A_{k+1} = \emptyset$ (infeasibility), or every point in A_{k+1} has $r_X = \top_X$ (loop axis saturates).

3 Installation and Project Layout

`codesign-mcdp` requires Python 3.9 or newer. The algorithmic core has no required runtime dependencies; the optional layers listed below pull in `numpy`, `scipy`, `matplotlib`, or `graphviz` as needed.

Installation. Clone the repository and install in editable mode:

```
1 git clone https://github.com/cbriat/codesign-mcdp.git
2 cd codesign-mcdp
3 pip install -e .
```

Optional extras.

- `pip install -e ".[viz]"` adds `matplotlib` for the visualisation helpers (Section 10) and the plotting examples.
- `pip install -e ".[online]"` adds `numpy` and `scipy` for the uncertainty (Section 8) and online-learning (Section 9) layers.
- `pip install -e ".[diagram]"` adds `graphviz` for the block-diagram renderer (Section 10.1).
- `pip install -e ".[nb]"` adds the Jupyter tooling needed to (re)generate the notebooks.

Repository layout.

```
codesign-mcdp/
  codesign/          the Python package (21 modules)
  examples/          25 runnable example scripts
  notebooks/         the same examples as executed .ipynb notebooks
  tests/             smoke tests
  docs/              this manual plus rendered trace images
  build_notebooks.py script to regenerate the notebooks
  pyproject.toml     packaging metadata, PEP 621
  .github/workflows/ CI configuration
```

4 Core Data Types

This section documents the data types provided by `codesign.posets` and `codesign.antichains`.

4.1 Posets

The `Poset` abstract base class has the following interface:

```
1 class Poset(ABC):
2     name: str
3
4     def leq(self, x, y) → bool: ...      # is  $x \leq y$ ?
5     def eq(self, x, y) → bool: ...
6     def lt(self, x, y) → bool: ...
7     def bottom(self): ...                # least element, if any
8     def top(self): ...                  # greatest element, if any
9     def is_bottom(self, x) → bool: ...
10    def is_top(self, x) → bool: ...
11    def join(self, x, y): ...             # least upper bound
12    def format(self, x) → str: ...
```

Four concrete poset classes are provided.

`Reals(unit="")`. The non-negative reals augmented with $+\infty$. Bottom is 0.0; top is `math.inf`. The optional `unit` string is used for pretty-printing only.

`Naturals(unit="")`. The non-negative integers augmented with $+\infty$. Bottom is 0; top is `math.inf`.

`Ports(components: dict[str, Poset])`. The componentwise-ordered product of named factor posets. Elements are Python dicts mapping component names to values. The bottom is the dict of all bottoms; the top is the dict of all tops. `Ports` can be nested freely: a component poset may itself be a `Ports`. This is the type used for every design problem's F and R .

The name reflects the library's vocabulary: each named component is a *port* of the design problem, addressable by name in the constraint DSL. For backward compatibility the older name `NamedProduct` remains exported as an alias, so existing code that imports `NamedProduct` continues to work; new code should prefer `Ports`.

`Discrete(elements, leq=None)`. A finite poset with explicit element set and a caller-supplied `leq` predicate. Useful for catalog-like enumerations where the order is not the default discrete order.

4.2 Antichains

The `Antichain` class wraps a set of poset elements together with the poset they live in. The constructor normalises the input by removing dominated points and deduplicating.

```
1 class Antichain:
2     poset: Poset
3     points: list
4
5     @classmethod
6     def of_bottom(cls, poset) → "Antichain": ...
7     @classmethod
8     def empty(cls, poset) → "Antichain": ...
9     @classmethod
10    def singleton(cls, poset, point) → "Antichain": ...
11    @classmethod
12    def from_set(cls, poset, points) → "Antichain": ...
13
14    @classmethod
15    def union_min(cls, poset, antichains) → "Antichain": ...
16
17    def filter_above(self, floor) → "Antichain": ...
18    def leq(self, other) → bool: ...
19    def eq(self, other) → bool: ...
20    def has_any_top(self) → bool: ...
21    def is_empty(self) → bool: ...
```

The two key operations on antichains are `union_min`, a class method that takes the union of a collection of antichains and re-applies `Min`, and `filter_above`, which keeps only points dominating a given floor. Together they implement the body of the Kleene iteration in Theorem 2.13; the full method surface is documented in the reference card (Section 13.1.2).

5 Primitive Design Problem Types

Six concrete subclasses of `DesignProblem` are provided, each appropriate for a different style of relation between functionality and resources.

5.1 AlgebraicDP

For relations where each resource is a closed-form monotone function of the functionality.

```
1 AlgebraicDP(
2     F: Ports,
3     R: Ports,
4     equations: dict[str, Callable[[dict], float]],
5     name: str = "algebraic",
6 )
```

The `equations` dictionary maps each resource name to a callable taking a functionality dict and returning a scalar. `h(f)` returns a singleton antichain.

Example 5.1. *A battery sized by specific energy:*

```
1 battery = AlgebraicDP(
2     F=Ports({"capacity": Reals(unit="J")}),
3     R=Ports({"mass": Reals(unit="kg")}),
4     equations={"mass": lambda f: f["capacity"] / 1.8e6},
5 )
```


5.2 FunctionDP

For relations where the user supplies h directly as a function from functionality dicts to antichains. Use this when the relation is multi-valued (a true Pareto front) or has branching logic.

```
1 FunctionDP(  
2     F: Ports,  
3     R: Ports,  
4     h_fn: Callable[[dict], Antichain],  
5     name: str = "function",  
6 )
```

Example 5.2 (A two-option amplifier). *The same audio amplifier output spec admits two implementations: a class-A topology (clean, hot) or a class-D topology (efficient, harsh). $h(f)$ emits both points, so the engineer sees the real tradeoff:*

```
1 F = Ports({"watts": Reals(unit="W")})  
2 R = Ports({"power_in": Reals(unit="W"),  
3           "thd": Reals()})  
4  
5 def h_amp(f):  
6     watts = f["watts"]  
7     return Antichain.from_set(R, [  
8         {"power_in": watts / 0.25, "thd": 0.001}, # class-A  
9         {"power_in": watts / 0.90, "thd": 0.020}, # class-D  
10    ])  
11  
12 amp = FunctionDP(F, R, h_amp)
```

5.3 CatalogDP

For selecting from a finite catalog of implementations (motors, batteries, sensors).

```
1 CatalogDP(  
2     F: Ports,  
3     R: Ports,  
4     catalog: list[CatalogEntry],  
5     name: str = "catalog",  
6 )  
7  
8 CatalogEntry(  
9     provides: dict[str, Any],  
10    costs: dict[str, Any],  
11    name: str = "",  
12 )
```

For a request f , $h(f)$ is the antichain of `entry.costs` for every entry whose `provides` dominates f in F . Entries that are dominated by another feasible entry are pruned by `Min` in R .

Example 5.3 (A small motor catalog). *Three motors with incomparable mass–cost tradeoffs at the same torque rating. h returns whichever entries cover the requested torque, then `Min` drops dominated ones automatically:*

```
1 motor = CatalogDP(  
2     F=Ports({"torque": Reals(unit="N*m")}),  
3     R=Ports({"mass": Reals(unit="kg"),  
4             "cost": Reals(unit="USD")}),  
5     catalog=[  
6         CatalogEntry(name="Tiny", provides={"torque": 2.0},  
7             costs={"mass": 0.2, "cost": 30.0}),  
8     ]  
9 )
```

```

8         CatalogEntry(name="Light", provides={"torque": 8.0},
9                       costs={"mass": 0.5, "cost": 200.0}),
10        CatalogEntry(name="Heavy", provides={"torque": 8.0},
11                      costs={"mass": 0.8, "cost": 120.0}),
12    ],
13 )
14 # motor.h({"torque": 7.0}) yields the antichain
15 # Antichain[(mass=0.5, cost=200), (mass=0.8, cost=120)]

```

The *Tiny* entry is filtered out (does not cover 7 N·m). *Light* and *Heavy* are mutually incomparable, so both survive the *Min*.

5.4 ConstraintDP

A feasibility predicate plus a scalar cost, lifted to *Min*. Useful when the resource-functionality relation is most natural to express as *is this choice good enough, and how much does it cost?*.

```

1 ConstraintDP(
2     F: Ports,
3     R: Ports,
4     sampler: Callable[[dict], Iterable[dict]],
5     feasible: Callable[[dict, dict], bool],
6     cost: Callable[[dict], dict],
7     name: str = "constraint",
8 )

```

The *sampler* yields candidate implementations for the given functionality; *feasible*(*impl*, *f*) filters them; *cost*(*impl*) maps each feasible implementation into the resource poset, and the result is reduced by *Min*.

5.5 ODE_DP

Derives a monotone resource relation from a differential equation $\dot{x} = \text{rhs}(x, t, f)$. Two modes are supported.

Final value integrates the ODE from $t = 0$ to $t = t_{\text{end}}$ by explicit Euler and reports the extracted resources of the final state.

Steady state solves $\text{rhs}(x, \cdot, f) = 0$ for x by Newton iteration and reports the extracted resources of the steady-state value.

```

1 ODE_DP(
2     F: Ports,
3     R: Ports,
4     rhs: Callable[[state, float, dict], state],
5     extract: Callable[[state], dict],
6     mode: str = "final_value", # or "steady_state"
7     t_end: float = 10.0,
8     n_steps: int = 200,
9     x0_fn: Callable[[dict], state] = None, # default: lambda f: 0.0
10    name: str = "ode",
11 )

```

Example 5.4 (A heater at thermal steady state). A wall heater sized from Newton's law of cooling: to hold a temperature rise ΔT above ambient, the input power must balance the heat loss $k \Delta T$. Modelling the delivered power as a scalar state x that relaxes towards its steady value, $\dot{x} = k \Delta T - x$, the root is $x = k \Delta T$. The heater's resource is that steady input power:

```

1 F = Ports({"delta_T": Reals(unit="K")})
2 R = Ports({"power": Reals(unit="W")})
3 k = 0.5 # heat-loss coefficient, W/K
4
5 heater = ODE_DP(
6     F, R,
7     rhs=lambda x, t, f: k * f["delta_T"] - x,
8     extract=lambda x: {"power": float(x)},
9     mode="steady_state",
10    x0_fn=lambda f: 0.0,
11 )
12 # solve(heater, {"delta_T": 30.0}) → Antichain[(power=15 W)]

```

The steady-state root $x = k \Delta T$ is recovered exactly by the Newton iteration. Note that the state carried by `ODE_DP` is a plain scalar (or a list of scalars), not a dict: `rhs`, `extract`, and `x0_fn` all operate on that state directly.

5.6 UncertainDP

Brackets an unknown h with a known lower and upper bound. The optimistic mode (`lower`) yields a lower bound on the minimal resources; the pessimistic mode (`upper`) yields an upper bound. Section VII of Censi (2015) develops the theory.

```

1 UncertainDP(
2     F: Ports,
3     R: Ports,
4     lower: DesignProblem,
5     upper: DesignProblem,
6     mode: str = "upper",
7     name: str = "uncertain",
8 )

```

`with_mode("lower" | "upper")` returns the bracket selected for solving.

Example 5.5 (A battery with uncertain specific energy). *Specific energy of the cell chemistry is known only to lie in $[1.6, 2.0]$ MJ/kg. The optimistic and pessimistic brackets give lower and upper bounds on the required mass:*

```

1 F = Ports({"capacity": Reals(unit="J")})
2 R = Ports({"mass": Reals(unit="kg")})
3
4 lo = AlgebraicDP(F, R, {"mass": lambda f: f["capacity"] / 2.0e6})
5 hi = AlgebraicDP(F, R, {"mass": lambda f: f["capacity"] / 1.6e6})
6
7 battery = UncertainDP(F, R, lower=lo, upper=hi)
8
9 # Solve both brackets and report the interval.
10 m_opt = solve(battery.with_mode("lower"), {"capacity": 3.6e6})
11 m_pess = solve(battery.with_mode("upper"), {"capacity": 3.6e6})
12 # m_opt → Antichain[(mass=1.80 kg)]
13 # m_pess → Antichain[(mass=2.25 kg)]

```

6 Composition Operators

The three operators of Section 2 are exposed as classes (`Series`, `Parallel`, `Loop`) with lowercase function aliases (`series`, `par`, `loop`) matching the paper's notation. Each takes existing design problems and returns a new `DesignProblem`, so composites nest freely and are solved by the same `solve` entry point. Because the class of MCDPs is closed under all three, a composite is itself a monotone design problem.

6.1 Series

```
1 series(dp1: DesignProblem, dp2: DesignProblem) → Series
2 Series(dp1, dp2)
```

Requires $dp1.R == dp2.F$ structurally (same component names and posets). Implements the series definition: for each point of $h_1(f)$ run h_2 and take the union Min.

Example 6.1 (Battery feeding an actuator). *A battery converts energy demand into mass; the actuator converts that mass demand into a power draw. Series composition feeds the battery's output directly into the actuator's input:*

```
1 battery = AlgebraicDP(
2   F=Ports({"capacity": Reals(unit="J")}),
3   R=Ports({"mass": Reals(unit="kg")}),
4   equations={"mass": lambda f: f["capacity"] / 1.8e6},
5 )
6 actuator = AlgebraicDP(
7   F=Ports({"mass": Reals(unit="kg")}),
8   R=Ports({"power": Reals(unit="W")}),
9   equations={"power": lambda f: 10.0 * f["mass"] ** 2},
10 )
11 chain = series(battery, actuator)
12 # solve(chain, {"capacity": 3.6e6}) maps energy → power directly.
```

6.2 Parallel

```
1 par(dp1: DesignProblem, dp2: DesignProblem) → Parallel
2 Parallel(dp1, dp2)
```

Requires non-overlapping component names in F_1, F_2 and in R_1, R_2 . The output F and R are the concatenated Portss. The antichain is the Cartesian product followed by Min.

Example 6.2 (Two independent subsystems). *A drone has a propulsion subsystem (energy in → mass out) and a sensing subsystem (sample rate in → power out). They share no ports and run in parallel; the composite DP takes both inputs and emits both outputs:*

```
1 propulsion = AlgebraicDP(
2   F=Ports({"energy": Reals(unit="J")}),
3   R=Ports({"prop_mass": Reals(unit="kg")}),
4   equations={"prop_mass": lambda f: f["energy"] / 1.8e6},
5 )
6 sensor = AlgebraicDP(
7   F=Ports({"sample_rate": Reals(unit="Hz")}),
8   R=Ports({"sensor_pwr": Reals(unit="W")}),
9   equations={"sensor_pwr": lambda f: 0.05 * f["sample_rate"]},
10 )
11 combined = par(propulsion, sensor)
12 # combined.F has both energy and sample_rate;
13 # combined.R has both prop_mass and sensor_pwr.
```

6.3 Loop

```
1 loop(inner: DesignProblem, axis: str) → Loop
2 Loop(inner, axis="...")
```

Closes the feedback loop on a single named axis that must appear in both $inner.F$ and $inner.R$. The resulting DP has $F = inner.F$ minus the axis and $R = inner.R$ minus the axis. Calling $h(f)$ triggers the Kleene fixed-point iteration described in Theorem 2.13.

Example 6.3 (Battery mass feedback). *A drone’s battery must carry both the payload and its own mass. Closing the loop on `battery_mass` converts the self-referential inequality into a fixed-point problem that the Kleene solver handles automatically:*

```

1 inner = AlgebraicDP(
2     F=Ports({"payload": Reals(unit="kg"),
3              "battery_mass": Reals(unit="kg")}),
4     R=Ports({"battery_mass": Reals(unit="kg")}),
5     equations={
6         "battery_mass":
7             lambda f: 0.5 * (f["payload"] + f["battery_mass"]),
8     },
9 )
10 drone = loop(inner, axis="battery_mass")
11 # drone.F = {"payload"}, drone.R = {}
12 # solve(drone, {"payload": 1.0}) iterates to the fixed point.

```

The closed-form solution is $m^* = p/(1 - 0.5) = 2p$, the fixed point to which the Kleene iteration converges (creeping to machine precision over several dozen contraction steps).

Remark 6.4 (Multiple feedback loops). *Loop closes one axis at a time. To close several loops, chain them: `loop(loop(inner, axis="x"), axis="y")`. The `System` builder (Section 12) closes all feedback loops simultaneously over a single bundled axis, which is usually more convenient.*

7 The Solver

All design problems, primitive or composite, are solved through a single entry point, `solve`. It evaluates the problem at a given functionality, running the Kleene fixed-point iteration of Theorem 2.13 whenever a feedback loop is present, and returns a `SolveResult` carrying the resulting antichain together with convergence and feasibility metadata. This section documents that entry point, the underlying iteration, scalar minimisation over the returned front, and the solver’s observability features.

7.1 Top-level entry point

```

1 solve(
2     dp: DesignProblem,
3     functionality: dict | None = None,
4     max_iter: int = 200,
5     *,
6     trace: bool = False,
7     verbose: int = 0,
8     on_iteration: Callable[[TraceEntry], None] | None = None,
9     start_from: SolveResult | Antichain | None = None,
10    uncertainty: list[str] | None = None,
11    n_samples: int = 1000,
12    rng_seed: int | None = None,
13 ) → SolveResult

```

The keyword-only arguments `trace`, `verbose`, `on_iteration`, and `start_from` are described in Section 7.4; `uncertainty`, `n_samples`, and `rng_seed` in Section 8. The result is a dataclass with the following fields:

```

1 @dataclass
2 class SolveResult:
3     antichain: Antichain          # the Pareto front (in dp.R)
4     iterations: int               # Kleene steps taken (0 if no loop)
5     status: str                  # "converged", "max_iter", or "diverged"
6     feasible: bool               # True if antichain has finite, nonempty points

```

```

7     trace: list[TraceEntry]    # the per-step trace if trace=True, else None
8
9     # converged is a read-only alias for status == "converged"

```

Example 7.1 (Inspecting a solve result). *The full lifecycle of a solve: build the DP, call `solve`, introspect the result, and pick a single design with `minimize_cost`:*

```

1  result = solve(drone, {"endurance":    300.0,
2                        "extra_payload": 0.5,
3                        "extra_power":   5.0},
4                  max_iter=200, trace=True)
5
6  print(result.status)          # "converged"
7  print(result.iterations)      # e.g. 17
8  print(result.feasible)        # True
9  print(result.antichain)        # Antichain[(total_mass=0.55 kg)]
10
11 # Scalarise: cheapest design under a custom cost function
12 best = minimize_cost(result, cost_fn=lambda r: r["total_mass"])
13 print(best)                   # {"total_mass": 0.5492}

```

7.2 The Kleene iteration

```

1  kleene_loop(
2      loop_dp: Loop,
3      f_outer: dict | None,
4      max_iter: int = 200,
5      *,
6      trace: bool = False,
7      verbose: int = 0,
8      on_iteration: Callable[[TraceEntry], None] | None = None,
9      start_from: Antichain | None = None,
10     info_out: dict | None = None,
11 ) → Antichain

```

The body of the iteration is direct from Theorem 2.13. The implementation adds:

- a *divergence cap* of 10^{30} that converts an iterate exceeding the cap to $\top$, so floating-point overflow becomes infeasibility rather than `inf/nan`;
- `try/except` around the inner h to catch `OverflowError`, `ValueError`, and `ZeroDivisionError`, again mapping to $\top$;
- three termination conditions: fixed point reached, antichain empty (infeasible), or every point's loop axis equal to $\top$ (provably infeasible).

7.3 Scalar minimisation over an antichain

```

1  minimize_cost(
2      result: SolveResult,
3      cost_fn: Callable[[dict], float],
4  ) → dict | None

```

Returns the single resource bundle in `result.antichain` that minimises `cost_fn`, or `None` if the result is infeasible. This is the scalarisation step that turns a Pareto front into a single engineering choice.

7.4 Observability: trace, verbose, callback, status

The solver can be made to report its progress in three independent ways, and its termination reason is exposed through a structured `status` field that is orthogonal to `feasible`.

The status field. `SolveResult.status` is one of three strings:

"converged" The iteration reached a fixed point (or provably hit $\top$) within `max_iter` steps. The answer in `result.antichain` is the algorithm's verdict; `feasible` tells you whether that verdict is finite (a real design) or $\top$ (no design exists).

"max_iter" The iteration was cut off at `max_iter`. The current antichain may or may not be near a fixed point. Usually a sign to increase `max_iter`, or to look for an unintentional non-monotonicity in the model.

"diverged" At least one numeric component crossed the divergence cap of 10^{30} before the iteration could settle, and the solver stopped. Distinguishes pure numerical blow-up from clean $\top$ -infeasibility.

The previous Boolean `converged` field is preserved as a backward-compatibility alias for `status == "converged"`.

Structured trace. Passing `trace=True` to `solve` populates `result.trace` with a list of `TraceEntry` dataclasses, one per Kleene step (plus the seed at iteration 0):

```
1 @dataclass
2 class TraceEntry:
3     iteration: int          # 0 = seed, then 1, 2, ...
4     antichain: Antichain    # snapshot at this step
5     n_points: int          # convenience: len(antichain)
6     delta: float | None    # max absolute change (numeric)
7                             # or 0/1 (discrete); None at iter 0
8     elapsed_ms: float      # wall time for this step alone
```

With `trace=False` (the default) the field is `None`, so the cost of unused tracing is zero.

Verbose printing. The integer `verbose` controls live output:

`verbose=0` (default) silent.

`verbose=1` one summary line at the end, e.g. `[solve] converged: 17 iters, |A|=1, total=2.0ms, feasible=True`.

`verbose=2` one line per iteration, showing the delta, the antichain size, and the step's wall time. Useful for watching oscillating fixed points.

Iteration callback. Passing `on_iteration=callable` registers a callback that receives each `TraceEntry` as soon as it is produced. Useful for live plots, custom logging, or interrupting an iteration that looks pathological:

```
1 def log_every_5(entry):
2     if entry.iteration % 5 == 0:
3         print(f"iter {entry.iteration}: delta={entry.delta:.3e}")
4
5 solve(dp, f, on_iteration=log_every_5)
```

Example 7.2 (Warm-starting a parameter sweep). When sweeping a parameter (load, mission length, demand level), the fixed point at one parameter is a great seed for the next. Pass the previous *SolveResult* (or its *antichain*) via *start_from=* and the Kleene iteration picks up from there instead of restarting at $\perp$:

```

1 prev = None
2 results = []
3 for L in np.linspace(5.0, 30.0, 50):
4     r = solve(dp, {"daily_load_kwh": float(L), ...},
5               max_iter=400, start_from=prev)
6     results.append(r)
7     prev = r           # reuse the converged inner antichain

```

For the microgrid example (notebook 13) this reduces the cumulative iteration count by roughly 10 % across the sweep, more for finer parameter grids.

Example 7.3 (Plotting convergence after a trace). With *trace=True*, the result carries the entire iteration history; a single *viz.plot_convergence* call renders the delta-vs-iteration semilog:

```

1 from codesign import viz
2
3 r = solve(drone, f, trace=True, max_iter=200)
4 ax = viz.plot_convergence(r)
5 ax.set_title(f"converged in {r.iterations} steps")

```

8 Uncertainty

Two kinds of parameter uncertainty are supported, declared on *Module* instances and consumed by the same *solve* entry point. Both are independent of the model's structure: a module can have set-based uncertainty, stochastic uncertainty, both, or neither.

8.1 Declaration

A *Module* carries optional attributes:

```

1 class Battery(Module):
2     F = {"capacity": Reals(unit="J")}
3     R = {"mass": Reals(unit="kg")}
4
5     def __init__(self, specific_energy=2.0e6, efficiency=0.9):
6         self.specific_energy = specific_energy
7         self.efficiency = efficiency
8         super().__init__()
9
10    def h(self, f):
11        # nominal product 2.0e6 * 0.9 = 1.8 MJ/kg delivered,
12        # matching the canonical drone (examples 1/6/7)
13        return {"mass": f["capacity"]
14                / (self.specific_energy * self.efficiency)}
15
16 b = Battery()
17 b.uncertain_set = Box(...)           # deterministic, set-based
18 b.uncertain_dist = Stochastic(...)   # statistical

```

The uncertainty solver discovers every such module by walking the *_codesign_modules* attribute that *System.build* attaches to the returned DP. Before each solve, the module's named parameters are reassigned; afterwards, their nominal values are restored. The user's original *Battery()* instance is unchanged.

8.2 Set-based uncertainty (deterministic)

Box. Axis-aligned interval product. Each parameter is declared as (lo, hi) or (lo, hi, direction), where direction is one of the four tokens "more_is_better", "more_is_worse", "less_is_better", "less_is_worse". With all directions declared, the worst case is a single corner read off in constant time:

```
1 b.uncertain_set = Box(  
2     specific_energy=(1.7e6, 2.3e6, "more_is_better"),  
3     efficiency=(0.83, 0.97, "more_is_better"),  
4 )  
5 # worst case = (specific_energy=1.7e6, efficiency=0.83)
```

When some directions are undeclared, all 2^n endpoint combinations are evaluated and the worst is kept; cheap when n is modest.

Ellipsoid. An n -D ellipsoid $(p - c)^\top \Sigma^{-1} (p - c) \leq 1$ in parameter space. The covariance Σ encodes both scale and correlation between parameters, so an ellipsoid is a more honest model than a box when parameters are believed to vary together:

```
1 b.uncertain_set = Ellipsoid(  
2     center={"specific_energy": 2.0e6, "efficiency": 0.9},  
3     cov=[  
4         [1.0e10, -2.0e3],  
5         [-2.0e3, 2.5e-3],  
6     ],  
7     params=["specific_energy", "efficiency"],  
8     directions={  
9         "specific_energy": "more_is_better",  
10        "efficiency": "more_is_better",  
11    },  
12 )
```

With all directions declared, the worst case is the boundary point $c + L u^*$, where L is the Cholesky factor of Σ and u^* is the unit vector in the direction of badness. Otherwise, the boundary is sampled (controlled by boundary_samples) and the worst sample is returned.

2D conveniences. `Disk(center, radius, ...)` and `Circle(center, radius, ...)` reduce to the isotropic ellipsoid $\Sigma = r^2 I$. For monotone systems with declared directions, the two are equivalent: the worst case lies on the boundary either way.

8.3 Stochastic uncertainty

Stochastic. A joint distribution built from named scipy-stats frozen marginals plus a copula:

```
1 from scipy import stats  
2  
3 b.uncertain_dist = Stochastic(  
4     marginals={  
5         "specific_energy": stats.uniform(loc=1.7e6, scale=0.6e6),  
6         "efficiency": stats.uniform(loc=0.83, scale=0.14),  
7     },  
8     copula=GaussianCopula(correlation=[[1.0, 0.4],  
9                                       [0.4, 1.0]]),  
10 )
```

Sampling is by the standard copula-marginal trick: the copula generates correlated uniforms in $[0, 1]^d$; the marginal ppfs map them to the named parameter axes.

Copulas. `Independence()` is the default (coordinatewise uniform). `GaussianCopula(correlation)` uses the Gaussian copula with the given correlation matrix; samples are drawn by Cholesky factorisation followed by Φ on each coordinate. Subclassing `Copula` requires only one method, `sample_uniform(n, d, rng)`, so other copulas (e.g. Clayton, Gumbel) drop in without further plumbing.

8.4 Querying

The same `solve` entry point accepts an `uncertainty` list and returns a richer object:

```

1 result = solve(
2     dp, f,
3     uncertainty=["worst_case", "mean", "p95", "cvar95", "samples"],
4     n_samples=1000,
5     rng_seed=42,
6 )
7
8 result.worst_case      # SolveResult at the worst point of the set
9 result.mean           # dict[r_port → mean across MC samples]
10 result.p95            # dict[r_port → 95th percentile]
11 result.cvar95         # dict[r_port → CVaR at 95% level]
12 result.samples        # list[Antichain], one per MC sample
13 result.feasibility_rate # fraction of feasible MC samples

```

The allowed labels are:

"worst_case" The deterministic worst over every module's `uncertain_set`. One canonical solve at the worst-case parameter point.

"mean", "p95", "cvar95" Marginal statistics over Monte Carlo samples, drawn from every module's `uncertain_dist`. The CVaR (conditional value at risk) at the 95% level is the mean of the worst 5% of samples.

"samples" The raw antichain per MC sample, for the user to aggregate however they like.

For a single solve call you typically observe the ordering

$$\text{nominal} < \text{mean} < \text{p95} < \text{CVaR95} < \text{worst_case},$$

which separates the four standard ways of reporting an answer "under uncertainty." Notebooks 11 and 12 visualise this for the example-7 drone.

Example 8.1 (Box worst-case of a drone battery). *Specific energy and efficiency of the battery cell live in declared ranges, each with a direction of badness. The worst-case mass is the corner where both parameters take their worst endpoint:*

```

1 bat = Battery() # the Module subclass from example~7
2 bat.uncertain_set = Box(
3     specific_energy=(1.7e6, 2.3e6, "more_is_better"),
4     efficiency=(0.83, 0.97, "more_is_better"),
5 )
6 # ... wire 'bat' into the drone system ...
7
8 result = solve(drone, f, uncertainty=["worst_case"])
9 worst_mass = list(result.worst_case.antichain.points)[0]["total_mass"]
10 # worst_mass = 0.5668 kg, versus 0.5492 kg nominal

```

Example 8.2 (Monte Carlo with a Gaussian copula). *The same battery, now with stochastic specific energy and efficiency, correlated at $\rho = 0.4$. One `solve` call returns all four summary statistics:*

```

1 from scipy import stats
2
3 bat.uncertain_dist = Stochastic(
4     marginals={
5         "specific_energy": stats.uniform(loc=1.7e6, scale=0.6e6),
6         "efficiency":      stats.uniform(loc=0.83, scale=0.14),
7     },
8     copula=GaussianCopula(correlation=[[1.0, 0.4],
9                                     [0.4, 1.0]]),
10 )
11
12 res = solve(drone, f,
13             uncertainty=["mean", "p95", "cvar95", "samples"],
14             n_samples=1000, rng_seed=42)
15 # res.mean["total_mass"]      → 0.5506 kg
16 # res.p95["total_mass"]      → 0.5632 kg
17 # res.cvar95["total_mass"]    → 0.5647 kg
18 # len(res.samples)           → 1000

```

9 Online Learning

When a co-design problem has many discrete candidates (catalog entries, robot types, component families) and each candidate’s inner solve is non-trivial, the naive “evaluate every entry” strategy is wasteful. The `codesign.online` module implements the compositional online learner of Alharbi, Dahleh & Zardini (arXiv:2604.22624): maintain history-dependent bounds on each candidate’s inner-solve output, evaluate the most promising one under the `lcb` (lower-confidence-bound) rule by default, then prune any candidate whose lower bound is already dominated by the incumbent.

9.1 Optimistic evaluators

The bound machinery is encapsulated in a small class hierarchy. `OptimisticEvaluator` is the abstract base; subclasses implement `bound(candidate)` returning a pair of dicts (`lower`, `upper`) mapping each numeric R component to its current lower and upper bound at the queried feature point. Tighter bounds prune more candidates; looser bounds are always safe.

Three concrete evaluators are provided:

Monotonicity. `MonotonicityEvaluator(features, r_components)` assumes the inner-solve output is component-wise monotone in the named features. Given an observation at feature point f_0 with antichain-min R_0 , any candidate whose features dominate f_0 has resources $\geq R_0$ (lower bound); any candidate dominated by f_0 has resources $\leq R_0$ (upper bound). Aggressive when applicable. Only correct if the monotonicity assumption genuinely holds: choosing a derived feature (e.g. `cost_per_capacity = unit_cost / (speed * payload)`) is often what makes this safe in practice.

Lipschitz. `LipschitzEvaluator(features, r_components, L)` assumes $|h(c_1) - h(c_2)| \leq L \cdot \|features(c_1) - features(c_2)\|$ (Euclidean distance, per R component). Each observation tightens the bound by a cone of slope L . The constant can be a scalar (same for every R component) or a dict per component. Safe default: with a sensible L the bound never prunes a Pareto-optimal candidate.

Linear-parametric. `LinearParametricEvaluator(features, r_components, min_obs=3, prior_box=None, noise_bound=0.0, solver="highs")` implements the *certified* optimistic bound of Alharbi et

al. (Section V-C3, eqs. 26–28 and Lemma V.5). It assumes each resource coordinate is an *exact* affine function of the features, $\text{req}_k(c) = \phi(c)^\top \theta_k^*$ with $\phi(c) = [1, \text{features}(c)]$ (the paper’s noiseless linear model). Rather than fit a single least-squares line, it maintains the whole *confidence polytope* $\Theta(H)$ — the subset of the prior box Θ_0 consistent with every observation taken as a linear equality — and returns, per resource coordinate, the coordinatewise minimum of the predicted resource over that polytope, $[\text{req}_{\text{opt}}(i, H)]_k = \min_{\theta \in \Theta(H)} \phi(i)^\top \theta$. Each such minimum is one linear program, solved with `scipy.optimize.linprog` (the `solver` kwarg selects the method). Because the true parameter θ_k^* is feasible for the LP, its optimum is always $\leq \phi(i)^\top \theta_k^* = \text{req}_k(i)$: the returned value is a *guaranteed* lower bound (the optimism guarantee of Lemma V.5), so — unlike the former OLS $\pm$ confidence-band heuristic — it can *never* wrongly eliminate a Pareto-optimal candidate. No upper bound is certified, so the returned upper bound stays at $+\infty$. `prior_box` sets Θ_0 (a per-parameter box) to keep the LP bounded while the fit is under-determined: `None` leaves every parameter unbounded, which is always safe but yields the trivial bound until enough observations pin θ down; `noise_bound > 0` relaxes each observation equality into a two-sided band of that half-width, a documented extension for bounded observation noise. The old `confidence` kwarg is deprecated and ignored (passing it emits a `DeprecationWarning`). Net effect: this evaluator is both safe *and* effective when the affine assumption genuinely holds, and degrades *safely* — to the no-information bound, never to a wrong elimination — when it does not.

Gaussian process. `GaussianProcessEvaluator(features, r_components, length_scale=0.3, sigma_f=1.0, noise=1e-3, confidence=2.0, min_obs=3)` fits a zero-mean GP with an RBF kernel and bounds new queries by a confidence band on the predictive standard deviation. Implemented in pure numpy. The right tool when the response surface has feature interactions or local nonlinearity: the GP still produces informative (if uncertified) bounds there, whereas the certified linear-parametric evaluator correctly degrades to the no-information bound once the affine assumption fails. When the map *is* affine, prefer the linear-parametric evaluator — its bound is then both certified-safe and tight.

Subclassing. Custom evaluators only need to override `bound(candidate)`; the base class handles observation recording and the default fallback bound of $(0, +\infty)$.

9.2 The online solver

```

1 result = solve_online(
2     candidate_fn,          # candidate_fn(candidate) → DP
3     functionality,        # outer F vector
4     *,
5     candidates,           # list of feature dicts
6     evaluator,            # OptimisticEvaluator instance
7     budget=None,          # max inner solves; None = unbounded
8     max_iter=200,         # forwarded to each inner solve
9     verbose=0,
10    warm_start=None,       # seed indices or n for farthest-point
11    picker="lcb",          # "lcb", "ucb", "random", or callable
12 )

```

Algorithm (a simplified Algorithm 2 from Alharbi et al.):

1. Bound every remaining candidate via the evaluator.
2. Eliminate every candidate whose lower bound is already dominated by the current incumbent antichain.

3. Pick the most promising survivor with the configured picker (lcb by default) on its lower-bound sum.
4. Run the inner solve at that candidate; observe the result in the evaluator; merge into the incumbent via `union_min`.
5. Repeat until the candidates are exhausted or the budget is hit.

The returned `OnlineResult` exposes:

`antichain` Min over the evaluated, surviving candidates.

`n_evaluated`, `n_eliminated`, `n_candidates` Bookkeeping for the elimination cascade.

`evaluated_ids`, `eliminated_ids`, `incumbent_ids` Lists of indices into the original candidate list.

`history` Per-iteration log: `{pick, antichain, remaining, evaluated, eliminated}`.

Example 9.1 (Fleet sizing across 200 robot types). *A logistics service has a 200-entry catalog of candidate robot types, each described by four features. The Lipschitz evaluator caps how fast the inner-solve output can change with the features, so a single observation tightens the bound on every other candidate. With the right L the answer is exact:*

```

1  def candidate_fn(robot):
2      capacity = robot["speed"] * robot["payload"]
3      return AlgebraicDP(F, R, {
4          "total_cost": lambda f, cap=capacity, uc=robot["unit_cost"]:
5              (f["target_throughput"] / cap) * uc,
6          "total_energy": lambda f, ek=robot["energy_per_km"]:
7              f["target_range"] * ek * 24.0,
8      })
9
10 ev = LipschitzEvaluator(
11     features=["speed", "payload", "unit_cost", "energy_per_km"],
12     r_components=["total_cost", "total_energy"],
13     L={"total_cost": 300.0, "total_energy": 30.0},
14 )
15
16 result = solve_online(
17     candidate_fn, mission,
18     candidates=robot_catalog,      # list of 200 feature dicts
19     evaluator=ev,
20 )
21 # result.n_evaluated    = 192 / 200
22 # result.n_eliminated   = 8
23 # result.antichain      = 5 Pareto-optimal robot types

```

Swapping in `MonotonicityEvaluator(features=["cost_per_capacity", "energy_per_km"], ...)` cuts the evaluation count to about 13 by exploiting the structural monotonicity of the derived feature.

9.3 Warm-start, picker strategies, and the GP evaluator

The basic online solver has three small extensions that significantly expand the tuning space without complicating the core algorithm.

Warm-start. A new `warm_start` argument to `solve_online` pre-populates the evaluator with seed observations before the picker takes over. It accepts either a list of candidate indices (manually picked corner runs) or an integer n that triggers a greedy farthest-point heuristic over the feature space. The motivation is concrete: the `MonotonicityEvaluator`'s lower bounds only tighten for candidates whose features dominate every observation in the partial order. Without observations at the low-feature corner where the Pareto front typically lives, the picker explores indiscriminately and the evaluator remains uninformative throughout the budget. In example 16, evaluating the example with four corner warm-start runs lifts Monotonicity's Pareto recovery from zero to one of four classes; a hybrid with one of the other evaluators is the natural next step for cases where this is not enough.

Pluggable picker strategies. A new `picker` argument selects the candidate-scoring rule. Built-in options: `"lcb"` (the previous default, minimises the sum of lower-bound components, pure exploitation of the optimistic estimate); `"ucb"` (lower bound minus a $\kappa \cdot$ (upper – lower) exploration bonus, tunable via the tuple form (`"ucb", {"kappa": 1.0}`)); and `"random"` (uniform baseline for comparing the value of structural priors). Custom callables are accepted and receive (`lower`, `upper`, `r_components`, `**kwargs`). The strategies that matter in practice are LCB for confident priors and UCB with $\kappa \approx 0.3$ to 0.5 when the prior is uncertain enough that pure exploitation gets stuck in a local region.

Gaussian-process evaluator. A new `GaussianProcessEvaluator` class uses a zero-mean GP with an RBF kernel, implemented in pure numpy. Hyperparameters are `length_scale` (default 0.3, suitable for features in $[0, 1]$), `sigma_f` (default 1.0, rescaled per-output by the empirical observation standard deviation), `noise` (default 10^{-3} jitter), `confidence` (default 2.0σ for roughly 95% coverage), and `min_obs` (default 3). The GP is more expressive than the linear-parametric evaluator and captures local nonlinearity that a global linear fit misses. Since the linear-parametric evaluator became the certified confidence-polytope bound the two occupy clearly separated niches. The certified `LinearParametricEvaluator` is the tool of choice when the resource map is genuinely affine in the features: on the linear fleet catalogue of example 14 it recovers all five Pareto types (where the former OLS-band version wrongly dropped one). But the example 16 bioprocess effect model is markedly nonlinear (temperature U-shapes, a power-law failure rate), and there the certified bound correctly refuses to over-claim — it degrades to the no-information bound and recovers zero of the four Pareto classes rather than eliminate a candidate it cannot rule out. The GP, willing to extrapolate a smooth surrogate, stays informative on that same grid (recovering one of the four classes). Rule of thumb: certified linear-parametric when you trust the affine assumption (safe *and* effective), GP when you do not.

10 Visualisation

The `codesign.viz` module exposes four helpers built on `matplotlib` and a small `GraphViz` emitter:

`plot_antichain(result, axes)` Scatter the antichain on the chosen 2D or 3D axes. Accepts a `SolveResult`, an `UncertaintyResult` (uses its worst case), or a bare `Antichain`. The 2D version optionally shades each point's dominated region for visual clarity.

`plot_convergence(result)` Semilog plot of the Kleene delta against iteration index. Accepts a `SolveResult` with a trace or a trace list directly. Useful for diagnosing oscillating fixed points and tuning `max_iter`.

`plot_uncertainty(unc_result, port, nominal=None)` Histogram of the Monte Carlo samples for the named R port, with the nominal, mean, p95, and CVaR95 marked as vertical lines.

`to_dot(dp, name="codesign")` GraphViz dot string for the system structure: modules as boxes, outer ports as labelled rectangles, constraint connections as edges. Render with `dot -Tpng` or paste into an online viewer.

All four accept an optional `ax=...` argument so they compose into larger figures. The module is imported as `from codesign import viz`.

10.1 Block diagrams of Systems

The `to_dot` function above emits a plain string with module-level wiring. For richer visualisation, the library ships a dedicated diagram renderer in `codesign.diagram` that produces Simulink-style block diagrams:

One box per subsystem with the F (functionality) ports listed on the left and the R (resource) ports on the right. Each port is individually addressable, so edges attach to specific ports rather than to the module as a whole.

Outer F and outer R as separate nodes on the diagram's left and right margins. The system inputs and outputs are visible, not buried inside the constraint list.

Port-level edges whenever the constraint was written in the operator-overloaded form (`module1.r_port >= module2.r_port * ...`). Constraints defined by callables that cannot be introspected are rendered with a dashed edge from a small “λ” marker, so they remain visible.

Cycle detection via Tarjan's strongly-connected-components algorithm on the module-level constraint graph. Edges within any cycle of size ≥ 2 are coloured amber, making the Kleene-iteration loop visible at a glance.

The renderer returns a `graphviz.Digraph` object, which can be displayed inline in a Jupyter notebook or written to SVG / PDF / PNG via `dot.render(filename, format="svg")`:

```
1 from codesign import draw_system
2
3 dot = system.draw_diagram()          # convenience method on System
4 # or equivalently:
5 dot = draw_system(system, rankdir="LR", highlight_cycles=True)
6
7 dot.render("bioprocess", format="svg", cleanup=True)
```

Optional dependency: install via `pip install codesign-mcdp[diagram]` (which pulls the `graphviz` Python package), and ensure the `dot` binary is on PATH (`apt-get install graphviz` on Debian / Ubuntu, `brew install graphviz` on macOS).

Example 10.1 (Bioprocess block diagram). *The worked example 15 (`make_bioprocess`) builds a four- module System with a single outer F (target titer) and three outer R aggregations defined by callables. The diagram renderer surfaces this structure compactly:*

```
1 from codesign import draw_system
2 dp = make_bioprocess(cell_line=CHO_K1, glucose_setpoint_mm=8.0,
3                       annual_demand_kg=100.0)
4 draw_system(dp, name="bioprocess").render("bioprocess",
5                                           format="svg",
6                                           cleanup=True)
```


The resulting diagram shows the four subsystems (*cell*, *feed*, *bior*, *media*), the outer functionality *target_titer* feeding *cell.target_titer*, the metabolic-load chain *cell.peak_vcd*, *feed.metabolic_factor* $\rightarrow$ *bior.peak_vcd* and *media.peak_vcd*, and a dashed λ marker feeding the three outer *R* aggregations.

Example 10.2 (Cyclic drone modular system). *Example 7's modular drone has a battery-actuator feedback loop: the battery's mass adds to the actuator's lift demand, and the actuator's electrical power sets the battery's required capacity. `draw_system` detects the cycle automatically and colours the two edges between `battery` and `actuator` in amber, while the supporting edges (from outer *F* endurance and `extra_power` into the cycle, and from each module's mass to the outer *R* `total_mass`) remain in muted grey.*

Example 10.3 (Pareto front of a vehicle co-design). *The vehicle from worked example 8 has two Pareto-incomparable designs at the Small-parcel mission. A single call paints them on a mass-cost plane and shades the dominated quadrants so the front is visible at a glance:*

```
1 from codesign import viz
2 import matplotlib.pyplot as plt
3
4 result = solve(vehicle, {"payload": 2.0, "mission_energy": 2.0e5})
5 viz.plot_antichain(result, axes=["total_mass", "total_cost"])
6 plt.show()
```

For three resources, pass three axis names to get a 3D scatter.

Example 10.4 (Uncertainty histogram with summaries). *Visualising the Monte Carlo output of a stochastic solve: the histogram of total mass across MC samples, with the nominal, mean, p95, and CVaR95 each drawn as a vertical line:*

```
1 res = solve(drone, f,
2             uncertainty=["mean", "p95", "cvar95", "samples"],
3             n_samples=1000, rng_seed=42)
4 ax = viz.plot_uncertainty(res, port="total_mass",
5                           nominal=0.5492, bins=30)
```

The function reads the requested summaries off the result and labels them automatically.

Example 10.5 (System graph in GraphViz dot). *For documentation or for understanding someone else's model, the constraint graph of a built `System` can be dumped as dot:*

```
1 dot = viz.to_dot(microgrid, name="microgrid")
2 # Render externally:
3 # echo "<dot output>" | dot -Tpng > microgrid.png
```

*Modules become boxes, outer *F* ports become source rectangles, outer *R* ports become sink rectangles, and constraint connections become directed edges labelled with the linking expression.*

11 The MCDP Builder

The MCDP class in `codesign.mcdpl` provides an MCDPL-style declarative syntax for a single self-contained design problem.

```
1 with MCDP("name") as m:
2     m.provides("functionality_name", unit="...") # outer F port
3     m.requires("resource_name", unit="...") # outer R port
4
5     m.constraint("resource_name", lambda f: <expr>) # closed-form
6     # or
7     m.rule(lambda f: Antichain.from_set(...)) # multi-valued
8
```



```

9     m.loop_on("axis_name")      # optional, may be repeated
10
11 dp = m.build()

```

The builder emits an AlgebraicDP or FunctionDP internally, then wraps it in Loops as requested by `loop_on`. Multiple `constraint` calls on the same target are joined (max). Mostly useful when the model has a small number of variables and reads naturally as a list of $\geq$ inequalities, mirroring the paper’s `mcdp { ... }` blocks.

Example 11.1 (Drone as a single MCDP block). *The Fig. 48 drone rewritten as a flat MCDP, with one closed loop on `battery_mass`:*

```

1  with MCDP("drone") as m:
2      m.provides("endurance",      unit="s")
3      m.provides("extra_payload", unit="kg")
4      m.provides("extra_power",   unit="W")
5      m.provides("battery_mass",  unit="kg")      # loop axis: in F and R
6      m.requires("battery_mass",  unit="kg")
7      m.requires("report_mass",   unit="kg")      # mirror for visibility
8
9      def battery_mass_eq(f):
10         return (f["extra_power"]
11                 + 10.0 * (9.81 * (f["battery_mass"]
12                                 + f["extra_payload"]))) ** 2
13                 ) * f["endurance"] / 1.8e6
14
15     m.constraint("battery_mass", battery_mass_eq)
16     m.constraint("report_mass",  battery_mass_eq)
17     m.loop_on("battery_mass")
18
19 drone = m.build()

```

Calling `solve(drone, {"endurance": 300.0, "extra_payload": 0.5, "extra_power": 5.0})` runs the same Kleene iteration as worked example 1 and returns the same `report_mass = 0.049 kg` fixed point. The loop axis `battery_mass` must be declared in both `provides` and `requires`, since that is the name `loop_on` closes; the extra `report_mass` resource mirrors it into the outer `R` so the converged value stays visible in the result, following the guideline in Section 17.

12 The System Builder

For larger designs the `System` class in `codesign.system` provides modular composition with named subsystems and algebraic connection constraints. This is the recommended way to build non-trivial co-design models. Two equivalent surface syntaxes are available: the operator-overloaded form (recommended for new code) and the legacy lambda form (still supported).

12.1 Operator-overloaded syntax (recommended)

`provides`, `requires`, and `add` each return a port handle. Arithmetic operators on handles build expression trees lazily; the `>=` operator registers a constraint with the parent system.

```

1  from codesign import Module, Reals, System, solve
2
3  class Battery(Module):
4      F = {"capacity": Reals(unit="J")}
5      R = {"mass":     Reals(unit="kg")}
6      def h(self, f):
7          return {"mass": f["capacity"] / 1.8e6}
8
9  class Actuator(Module):

```

```

10 F = {"lift_force": Reals(unit="N")}
11 R = {"power":      Reals(unit="W")}
12 def h(self, f):
13     return {"power": 10.0 * f["lift_force"] ** 2}
14
15 sys = System("drone")
16 endurance = sys.provides("endurance", unit="s")
17 extra_payload = sys.provides("extra_payload", unit="kg")
18 extra_power = sys.provides("extra_power", unit="W")
19 total_mass = sys.requires("total_mass", unit="kg")
20
21 battery = sys.add("battery", Battery())
22 actuator = sys.add("actuator", Actuator())
23
24 battery.capacity ≥ (actuator.power + extra_power) * endurance
25 actuator.lift_force ≥ 9.81 * (battery.mass + extra_payload)
26 total_mass ≥ battery.mass + extra_payload
27
28 drone = sys.build()

```

The four port kinds are:

- **Outer F** (from `provides`): can appear on the RHS of constraint expressions. Cannot be a constraint target.
- **Outer R** (from `requires`): can be a constraint target. Cannot appear in expression RHS.
- **Module F** (`handle.f_port`): can be a constraint target. Cannot appear in expression RHS (its value is determined by the constraints on it, not by the current iteration state).
- **Module R** (`handle.r_port`): can appear in expression RHS. Cannot be a constraint target (its value is determined by the module's own `h`, not externally).

Misusing a port (e.g. trying to put a module F port on the RHS, or constraining an outer F) raises immediately with a message naming the port and explaining the rule. F and R port names within a single subsystem must be disjoint.

The supported operators are `+`, `-`, `*`, `/`, `**`, unary `-`, along with the helper functions `codesign.sqrt`, `exp`, and `log` for expressions that need transcendental terms. For demands that cannot be expressed algebraically, the legacy lambda form remains available and can be mixed freely with the operator form (see below).

12.2 Legacy lambda syntax

```

1 sys.constrain("battery.capacity",
2               lambda x: (x["actuator.power"] + x["extra_power"]) * x["endurance"])
3 sys.constrain("actuator.lift_force",
4               lambda x: 9.81 * (x["battery.mass"] + x["extra_payload"]))
5 sys.constrain("total_mass",
6               lambda x: x["battery.mass"] + x["extra_payload"])

```

The argument `x` is a dict carrying outer F values under their bare names (`x["endurance"]`) and subsystem R ports under their dotted names (`x["battery.mass"]`). Both syntaxes compile to the same internal representation; the operator form is purely sugar.

12.3 Class-based modules

The `Module` base class lets a design problem be defined declaratively:

```

1 class Battery(Module):
2     F = {"capacity": Reals(unit="J")}
3     R = {"mass":      Reals(unit="kg")}
4
5     def __init__(self, specific_energy=1.8e6):
6         self.specific_energy = specific_energy
7         super().__init__()
8
9     def h(self, f):
10        return {"mass": f["capacity"] / self.specific_energy}

```

A Module subclass declares its F and R ports as class-level dicts and overrides $h(\mathbf{self}, \mathbf{f})$. The constructor wires everything into the underlying `DesignProblem` machinery. Parameterised modules are written by overriding `__init__` and calling `super().__init__()` after setting any instance attributes. h may return a dict (treated as a singleton antichain), a list of dicts (multi-valued antichain), or an `Antichain` directly.

Example 12.1 (A modular drone end-to-end). *The drone from worked example 7, built with the operator-overloaded constraint syntax. Each line below a `System` declaration is a textbook inequality:*

```

1 sys = System("drone")
2 endurance = sys.provides("endurance", unit="s")
3 extra_payload = sys.provides("extra_payload", unit="kg")
4 extra_power = sys.provides("extra_power", unit="W")
5 total_mass = sys.requires("total_mass", unit="kg")
6
7 b = sys.add("battery", Battery())
8 a = sys.add("actuator", Actuator())
9
10 b.capacity ≥ (a.power + extra_power) * endurance
11 a.lift_force ≥ 9.81 * (b.mass + extra_payload)
12 total_mass ≥ b.mass + extra_payload
13
14 drone = sys.build()
15 r = solve(drone, {"endurance": 300.0,
16                  "extra_payload": 0.5,
17                  "extra_power": 5.0})
18 # r.antichain → Antichain[(total_mass=0.55 kg)]

```

`build()` bundles every subsystem's R into a single Kleene axis and closes the loop automatically. The constraint graph can be exported with `viz.to_dot(drone)` for documentation.

12.4 Semantics

The `build` method emits a Loop whose axis bundles every subsystem's resource poset:

$$__\text{modules}__ = \prod_{m \in \text{modules}} R_m.$$

The inner DP performs one Kleene step:

1. Read the current bundle estimate to populate `ctx[m.r]` for every subsystem R port.
2. For each subsystem m , evaluate the constraint demands on its F ports to obtain f_m , then call $m.h(f_m)$ to obtain an antichain.
3. Take the Cartesian product across subsystems; for each combination, evaluate the outer- R constraint demands to produce a candidate point.
4. Return the resulting antichain.

The outer Kleene iteration converges over all subsystem R ports simultaneously.

12.5 Validation

`build` performs the following checks:

- every subsystem F port has at least one `constrain` target;
- every outer R has at least one `constrain` target;
- every constraint target refers to a known module F port or a declared outer R ;
- module names do not clash with the reserved axis name `__modules__` and contain no `..`.

If any check fails, `build` raises `ValueError` with a message identifying the missing or invalid item.

13 API Reference

This section is the complete, per-module reference for the public API exported by `import codesign`. The six subsections below follow the architecture of the library: order structures; design problems and their solver; the two builders and the rendering helpers; the uncertainty and online-learning layers; the temporal planners; and the vector-state and closed-loop layers. Within each group one subsubsection documents each source module, and each public symbol is a reference card giving its purpose, signature, a minimal runnable snippet, its parameters, and the exceptions it raises. Cards are cross-linked to the tutorial chapters (Sections 2–16) and the worked examples (Section 14); the condensed export list in Section 19 remains a one-page index. All reference labels use the `ref:<module>:<symbol>` scheme.

13.1 Order structures

The two foundational modules: `posets` supplies the functionality and resource spaces, and `antichains` the Pareto-front value type returned by every design problem. See Section 2 for the underlying order theory.

13.1.1 posets

The `codesign.posets` module supplies the partially ordered sets that serve as functionality spaces F and resource spaces R . Every concrete poset derives from the abstract `Poset` base and honours its contract: a least element `bottom()` that seeds the Kleene iteration, a greatest element `top()` that marks infeasibility, and a `join()` that returns the least upper bound used to combine constraints. The mathematical background is given in Section 2.

Poset Abstract base class defining the poset contract. It is not instantiable directly; a concrete subclass must override `leq`, `bottom` and `top`, and inherits working defaults for the derived operators. The four shipped subclasses (`Reals`, `Naturals`, `Ports`, `Discrete`) cover essentially every modelling need.

Signature: `class Poset(ABC)` with class attribute `name: str`. The derived operators are exercised here through a concrete subclass, since `Poset` itself cannot be instantiated:

```
1 from codesign import Reals
2 P = Reals(unit="kg")
3 # Derived operators are provided by the Poset base class:
4 print(P.eq(1.0, 1.0))           # True
5 print(P.lt(1.0, 2.0))           # True
6 print(P.comparable(1.0, 2.0))   # True (a chain: all comparable)
7 print(P.is_bottom(0.0))         # True
```

The contract methods (the abstract three plus the inherited defaults):

Method	Signature	Behaviour
<code>leq</code>	<code>leq(a, b) -> bool</code>	abstract: is $a \leq b$?
<code>bottom</code>	<code>bottom()</code>	abstract: least element
<code>top</code>	<code>top()</code>	abstract: greatest element
<code>is_top</code>	<code>is_top(x) -> bool</code>	default via <code>leq</code>
<code>is_bottom</code>	<code>is_bottom(x) -> bool</code>	default via <code>leq</code>
<code>eq</code>	<code>eq(a, b) -> bool</code>	$a \leq b \wedge b \leq a$
<code>lt</code>	<code>lt(a, b) -> bool</code>	$a \leq b \wedge a \neq b$
<code>comparable</code>	<code>comparable(a, b) -> bool</code>	$a \leq b \vee b \leq a$
<code>join</code>	<code>join(a, b)</code>	least upper bound
<code>format</code>	<code>format(x) -> str</code>	display string (<code>repr</code>)

Parameters: the constructor takes no arguments; `name` is a display label overridden by subclasses.

Errors: instantiating `Poset` directly raises `TypeError` (abstract methods unimplemented). The default `join` raises `NotImplementedError` when `a` and `b` are incomparable (neither `leq` holds); chain and product subclasses override `join` so it never fails on them.

Reals The non-negative reals $\mathbb{R}_{\geq 0}$ augmented with $+\infty$, forming a CPO. This is the scalar chain for any real-valued port (mass, cost, energy). Bottom is `0.0`, top is `math.inf`.

Signature: `Reals(unit: str = "", name: str = "R+")`.

```

1 from codesign import Reals
2 R = Reals(unit="kg")
3 print(R.bottom(), R.top())    # 0.0 inf
4 print(R.leq(0.5, 1.2))       # True
5 print(R.join(0.5, 1.2))      # 1.2
6 print(R.format(0.56))        # 0.56 kg
7 print(R.format(R.top()))     # the top marker
8 print(R.name)                # R+[kg]
```

Parameters:

- `unit` — display-only unit string; travels through composition so `format` can print `0.56 kg`. Never used in any algebraic check.
- `name` — display name; if left at the default and a `unit` is given, becomes `R+[unit]` automatically.

Deviations from the `Poset` contract: overrides `is_top` (direct $+\infty$ test), `join` (returns `max`) and `format` (prints $\top$ for $+\infty$, else the value with its unit).

Errors: none raised directly.

Naturals The non-negative integers $\mathbb{N}$ augmented with a top ($+\infty$), forming a CPO. Mirrors `Reals` for integer-valued quantities such as part counts or bit budgets. Bottom is `0`, top is `math.inf`.

Signature: `Naturals(unit: str = "", name: str = "N+")`.

```

1 from codesign import Naturals
2 N = Naturals(unit="parts")
3 print(N.leq(3, N.top()))     # True
4 print(N.join(2, 5))          # 5
5 print(N.is_top(N.top()))     # True
6 print(N.bottom())            # 0
7 print(N.format(3))           # 3 parts
8 print(N.format(N.top()))     #  $\top$ 
```

Parameters: `unit` and `name` as for `Reals`. `format` mirrors `Reals`: it appends the `unit` (so `N.format(3)` prints `3 parts`) and renders the top as `⊤`; finite values keep their integer form (`3.0` prints as `3`). When `unit` is set and the default `name` is kept, the name is decorated as `N+[unit]`.

Deviations from the `Poset` contract: overrides `leq` to treat $+\infty$ as strictly above every finite natural (and equal only to itself), `is_top`, `join`, and `format`. Finite inputs are coerced with `int()`.

Errors: none raised directly.

Ports A named product poset: a typed bundle of named component posets, ordered component-wise. This is the workhorse type; every design problem's F and R is a `Ports`. Elements are Python dicts mapping each component name to a value in its factor poset. The order is $x \leq y \iff x[k] \leq y[k]$ for every component k ; bottom and top are the dicts of all component bottoms and tops.

Signature: `Ports(components: Mapping[str, Poset])`.

```
1 from codesign import Ports, Reals
2 F = Ports({"capacity": Reals(unit="J")})
3 x = F.make(capacity=3.6e6)
4 print(F.leq(x, F.top())) # True
5 print(F.is_top(F.top())) # True
6 print(F.any_top({"capacity": F.top()["capacity"]})) # True
7 print(F.format(x)) # (capacity=3.6e+06 J)
8 print(list(F.keys())) # ['capacity']
```

Parameters:

- `components` — non-empty `{name: poset}` mapping. Copied into a fresh dict, so later mutation of the argument cannot corrupt the instance. A component poset may itself be a `Ports` (nesting is allowed).

Beyond the contract, `Ports` adds three public methods:

Method	Signature	Behaviour
<code>keys</code>	<code>keys()</code>	iterate component names
<code>any_top</code>	<code>any_top(x) -> bool</code>	<i>any</i> component at top
<code>make</code>	<code>make(**kwargs)</code>	build a validated element dict

Note the distinction between `is_top` (*every* component at top — total infeasibility) and `any_top` (*at least one* component at top — enough to disqualify a candidate in the antichain machinery).

Errors: the constructor raises `ValueError` if `components` is empty. `make` raises `ValueError` if any declared component is missing from the keyword arguments, and `ValueError` if any unknown component name is supplied. `leq`, `join` and `format` may raise `KeyError` if an element dict lacks a declared component.

Discrete A finite poset over an explicit element list with a caller-supplied order predicate. Useful for enumerated design spaces (part numbers, operating modes). The default order is equality: nothing is below anything but itself.

Signature: `Discrete(elements, leq_fn=None, name: str = "D")`.

```
1 from codesign import Discrete
2 modes = Discrete(["idle", "cruise", "boost"], name="mode")
3 print(modes.leq("idle", "idle")) # True
4 print(modes.leq("idle", "cruise")) # False (equality order)
5 # bottom()/top() raise ValueError: no canonical extremum.
```

Parameters:

- `elements` — iterable of elements; stored as a list.
- `leq_fn` — order predicate `leq_fn(a, b) -> bool`; defaults to `a == b` (the discrete order).
- `name` — display name.

Errors: `bottom()` and `top()` each raise `ValueError`, because an arbitrary discrete order has no canonical least or greatest element; subclass and override them, or wrap in a poset that adds explicit extrema.

NamedProduct Backward-compatible alias: `NamedProduct = Ports`. Existing code importing `NamedProduct` continues to work unchanged; new code should prefer `Ports` (see 13.1.1). The two names are the same object.

```
1 from codesign import NamedProduct, Ports
2 print(NamedProduct is Ports)    # True
```

Errors: as for `Ports`.

13.1.2 antichains

The `codesign.antichains` module provides the `Antichain` type, the return value of every design problem's resource map $h : F \rightarrow A[R]$. Antichains generalise the Pareto front to an arbitrary partial order; their lattice structure (order, join by union followed by `Min`) is developed in Section 2.

Antichain A finite set of mutually incomparable elements of a poset, paired with the poset they live in. Construction is self-normalising: dominated points and duplicates are discarded, so the constructor is idempotent under `Min`. Instances are iterable and support `len()` and truthiness (empty is falsey).

Signature: `Antichain(poset: Poset, points: Iterable = ())`. The class-method constructors are the primary construction API; the bare constructor is rarely called directly.

```
1 from codesign import Antichain, Ports, Reals
2 R = Ports({"mass": Reals(unit="kg"), "cost": Reals(unit="$")})
3 pts = [{"mass": 1.0, "cost": 5.0},
4         {"mass": 2.0, "cost": 3.0},
5         {"mass": 1.5, "cost": 6.0}]    # dominated by the first
6 A = Antichain.from_set(R, pts)
7 print(len(A))    # 2 (dominated point dropped)
8 B = Antichain.singleton(R, {"mass": 0.5, "cost": 7.0})
9 U = Antichain.union_min(R, [A, B])
10 print(len(U))    # 3
```

Parameters:

- `poset` — the underlying `Poset` (typically a `Ports`) supplying `leq`, `eq` and, where present, `any_top`.
- `points` — iterable of poset elements; normalised to an antichain on insertion.

Public methods (class-method constructors marked *cm*):

Method	Signature	Behaviour
<code>of_bottom</code>	<code>of_bottom(poset) cm</code>	singleton $\{\perp\}$; seeds Kleene
<code>empty</code>	<code>empty(poset) cm</code>	the empty antichain
<code>singleton</code>	<code>singleton(poset, point) cm</code>	one-point antichain
<code>from_set</code>	<code>from_set(poset, points) cm</code>	Min of a subset
<code>union_min</code>	<code>union_min(poset, antichains) cm</code>	$\text{Min}(\bigcup_i A_i)$
<code>points</code>	<code>points (property)</code>	list copy of the points
<code>leq</code>	<code>leq(other) -> bool</code>	antichain order (see below)
<code>eq</code>	<code>eq(other) -> bool</code>	equal point sets under <code>eq</code>
<code>filter_above</code>	<code>filter_above(lower)</code>	keep x with $lower \leq x$
<code>has_any_top</code>	<code>has_any_top() -> bool</code>	any point at (a component) top
<code>is_empty</code>	<code>is_empty() -> bool</code>	no points

The antichain order is $A \leq A'$ iff $\uparrow A \supseteq \uparrow A'$: every point of `other` is dominated by some point of `self`. `union_min` and `filter_above` together implement the body of the Kleene iteration (Section 2); `has_any_top` uses the poset's `any_top` when available (as on `Ports`), otherwise falls back to `is_top`.

Errors: none raised directly. Operations delegate to the underlying poset, so a malformed point may surface a `KeyError` from `Ports.leq` rather than an antichain-level exception.

13.2 Design problems and solving

The six primitive design-problem types (`dp`), the scalar `primitives` built on them, the three composition operators, and the Kleene `solver` that computes fixed points. The tutorial treatment is in Sections 5–7.

13.2.1 dp

The `dp` module defines the central abstraction, the `DesignProblem`, together with the concrete primitives that realise it in closed form, from a catalogue, from a constraint, or from an ODE. Every design problem exposes a single method `h(f)` returning the antichain of minimal resources able to deliver functionality `f`.

DesignProblem The abstract base class for all design problems: a monotone relation from a functionality poset `F` to an antichain in a resource poset `R`. Subclasses must implement `h`; the class is never instantiated directly.

```

1 class DesignProblem(ABC):
2     F: Poset          # functionality space
3     R: Poset          # resource space
4     name: str = "dp"
5     def h(self, f) → Antichain: ... # abstract

```

Concrete instances expose `h`, `F`, `R` and a readable `__repr__`:

```

1 from codesign import AlgebraicDP, Ports, Reals
2 dp = AlgebraicDP(
3     F=Ports({"speed": Reals()}),
4     R=Ports({"power": Reals(unit="W")}),
5     equations={"power": lambda f: 2.0 * f["speed"]},
6 )
7 print(repr(dp))          # DP(algebraic: speed:R+ → A[...])
8 print(dp.h({"speed": 3.0})) # Antichain[(power=6 W)]

```

- `F` — Poset — the functionality space.

- `R` — `Poset` — the resource space.
- `h(f)` — returns the minimal-resource `Antichain` for `f`.

Errors: being an ABC with an abstract `h`, direct instantiation raises `TypeError`; subclasses raise their own (below).

AlgebraicDP A functional design problem whose every resource is a closed-form (monotone) expression in the functionality. The relation is a singleton antichain. See Section 5 for the tutorial treatment. Monotonicity is the caller’s responsibility and is not verified.

```
1 AlgebraicDP(F: Poset, R: Ports,
2             equations: Mapping[str, Callable | Any],
3             name: str = "algebraic")

1 from codesign import AlgebraicDP, Ports, Reals
2 motor = AlgebraicDP(
3     F=Ports({"torque": Reals(unit="Nm")}),
4     R=Ports({"mass": Reals(unit="kg")}),
5     equations={"mass": lambda f: 0.5 * f["torque"] + 1.0},
6 )
7 print(motor.h({"torque": 4.0})) # Antichain[(mass=3 kg)]
```

- `R` — `Ports` — named resource bundle (required).
- `equations` — one entry per resource port, a callable `f_dict -> value` or a constant.

Errors: raises `TypeError` if `R` is not a `Ports`; raises `ValueError` if any resource port declared in `R` has no entry in `equations`.

FunctionDP Wraps an arbitrary user callable `h_fn`. The return value is normalised to an antichain: an `Antichain` is passed through, a dict or a bare point becomes a singleton, and any other iterable is treated as a set of points.

```
1 FunctionDP(F: Poset, R: Poset,
2            h_fn: Callable[[Any], Any],
3            name: str = "function")

1 from codesign import FunctionDP, Ports, Reals
2 fdp = FunctionDP(
3     F=Ports({"load": Reals()}),
4     R=Ports({"cost": Reals()}),
5     h_fn=lambda f: {"cost": f["load"] ** 2},
6 )
7 print(fdp.h({"load": 3.0})) # Antichain[(cost=9)]
```

- `h_fn` — callable returning an `Antichain`, a dict, a point, or an iterable of points.

Errors: none raised directly; the caller guarantees monotonicity, and any exception raised inside `h_fn` propagates unchanged.

CatalogEntry A dataclass describing one implementation in a catalogue: the functionality it can deliver and the resources it costs.

```
1 from codesign import CatalogEntry
2 e = CatalogEntry(provides={"speed": 5.0},
3                 costs={"cost": 10.0}, name="A")
```

Field	Type	Meaning
<code>provides</code>	<code>Mapping</code>	functionality this entry fulfils (any <code>f</code> $\leq$ <code>provides</code>)
<code>costs</code>	<code>Mapping</code>	resource vector it costs
<code>name</code>	<code>str</code>	optional label (default "")

Errors: none raised directly.

CatalogDP Selects the cheapest catalogue entries able to deliver f . An entry is feasible when its provides dominates f port-wise; the result is the Min of the feasible costs. If no entry works, h returns the singleton at $\top$ (infeasible), rather than raising.

```

1 CatalogDP(F: Ports, R: Ports, catalog: Sequence,
2           name: str = "catalog")

1 from codesign import CatalogDP, CatalogEntry, Ports, Reals
2 cat = CatalogDP(
3     Ports({"speed": Reals()}), Ports({"cost": Reals()}),
4     catalog=[
5         CatalogEntry({"speed": 5.0}, {"cost": 10.0}, "A"),
6         CatalogEntry({"speed": 9.0}, {"cost": 25.0}, "B"),
7     ])
8 print(cat.h({"speed": 6.0})) # only B works: (cost=25)

```

- **catalog** — a sequence of **CatalogEntry** objects or plain dicts with **provides**/**costs**/**name** keys.

Errors: raises **TypeError** if **F** or **R** is not a **Ports**; **ValueError** (naming the DP) if the catalogue is empty, or if two entries share the same non-empty **name** (the clashing name(s) are listed). Multiple entries left unnamed (the default **name=""**) are allowed.

ConstraintDP Defines a problem by a **sampler** over an implementation space, a **feasible** predicate, and a **cost** function; h returns the Min over all feasible samples (or the singleton at $\top$ when none is feasible).

```

1 ConstraintDP(F: Poset, R: Ports,
2             sampler: Callable[[Any], Iterable],
3             feasible: Callable[[Any, Any], bool],
4             cost: Callable[[Any], Mapping],
5             name: str = "constraint")

1 from codesign import ConstraintDP, Ports, Reals
2 cdp = ConstraintDP(
3     F=Ports({"span": Reals()}), R=Ports({"area": Reals()}),
4     sampler=lambda f: [0.5, 1.0, 2.0],
5     feasible=lambda impl, f: impl >= f["span"],
6     cost=lambda impl: {"area": impl},
7 )
8 print(cdp.h({"span": 0.8})) # Antichain[(area=1)]

```

- **sampler(f)** — yields candidate implementations.
- **feasible(impl, f)** — whether **impl** delivers **f**.
- **cost(impl)** — resource vector for a feasible sample.

Errors: none raised directly (no upfront interface check); exceptions inside the callables propagate.

ODE_DP Derives a monotone relation from an ODE $dx/dt = \text{rhs}(x, t, f)$, reading resources from the trajectory with **extract**. In **final_value** mode it integrates (explicit Euler) to **t_end**; in **steady_state** mode it Newton-solves $\text{rhs}(x) = 0$. See Section 7 for the wider solve pipeline.

```

1 ODE_DP(F: Poset, R: Ports,
2        rhs: Callable[[Any, float, Any], Any],
3        extract: Callable[[Any], Mapping],
4        mode: str = "final_value",
5        t_end: float = 10.0, n_steps: int = 200,

```

```

6     x0_fn: Callable[[Any], Any] | None = None,
7     name: str = "ode")

```

```

1 from codesign import ODE_DP, Ports, Reals
2 tank = ODE_DP(
3     F=Ports({"inflow": Reals()}), R=Ports({"level": Reals()}),
4     rhs=lambda x, t, f: f["inflow"] - 0.5 * x,
5     extract=lambda x: {"level": x},
6     mode="steady_state", x0_fn=lambda f: 0.0,
7 )
8 print(tank.h({"inflow": 2.0})) # Antichain[(level=4)]

```

- `mode` — "final_value" or "steady_state".
- `x0_fn(f)` — initial state; a scalar or a positional sequence of floats (defaults to 0.0).
- `t_end, n_steps` — integration horizon and step count (final_value only).

Errors: the constructor raises `ValueError` if `mode` is not "final_value" or "steady_state". On the first `h` call the state is validated: a dict-valued (named) state raises `TypeError` (name the keys onto a list and index positionally), as does a `str/bytes` state.

UncertainDP Brackets a nominal problem between an optimistic lower DP `lower` and a pessimistic upper DP `upper`, returning whichever the `mode` selects (paper Sec. VII). See Section 8 for the uncertainty solver.

```

1 UncertainDP(F: Poset, R: Poset,
2             lower: DesignProblem, upper: DesignProblem,
3             mode: str = "upper", name: str = "uncertain")

```

```

1 from codesign import AlgebraicDP, UncertainDP, Ports, Reals
2 F = Ports({"x": Reals()}); R = Ports({"y": Reals()})
3 lo = AlgebraicDP(F, R, {"y": lambda f: f["x"]})
4 hi = AlgebraicDP(F, R, {"y": lambda f: 1.5 * f["x"]})
5 u = UncertainDP(F, R, lower=lo, upper=hi, mode="upper")
6 print(u.h({"x": 2.0})) # (y=3)
7 print(u.with_mode("lower").h({"x": 2.0})) # (y=2)

```

- `lower, upper` — the two bounding DPs.
- `mode` — "lower" (optimistic) or "upper" (pessimistic).

`with_mode(mode)` Returns a fresh `UncertainDP` sharing the same bounds but with the given `mode`; the convenient way to obtain both fronts.

Errors: the constructor (and hence `with_mode`) raises `ValueError` if `mode` is not "lower" or "upper".

13.2.2 primitives

The `primitives` module provides small factory functions for the everyday plumbing DPs used to glue diagrams together (paper Fig. 26, Fig. 35). Each returns a ready-made `AlgebraicDP` (13.2.1) over scalar `Reals` ports, so its result is always a singleton antichain. Every factory accepts an optional `poset` argument; when omitted a plain `Reals()` is used for all ports. As thin wrappers over `AlgebraicDP`, they raise nothing directly; the ports they build are always `Ports`, so the wrapped constructor never faults.

adder Sums several scalar inputs into one scalar output.

```

1 adder(in_names: list[str], out_name: str, poset=None)

```

```

1 from codesign import adder
2 a = adder(["p1", "p2"], "total")
3 print(a.h({"p1": 2.0, "p2": 3.0})) # Antichain[(total=5)]

```

multiplier Multiplies two scalar inputs, e.g. current times voltage gives power.

```

1 multiplier(in_a: str, in_b: str, out_name: str, poset=None)

```

```

1 from codesign import multiplier
2 m = multiplier("current", "voltage", "power")
3 print(m.h({"current": 2.0, "voltage": 5.0})) # (power=10)

```

scale Multiplies one input by a constant factor.

```

1 scale(in_name: str, out_name: str, factor: float, poset=None)

```

```

1 from codesign import scale
2 s = scale("capacity", "mass", factor=0.02)
3 print(s.h({"capacity": 1000.0})) # Antichain[(mass=20)]

```

constant Ignores its (trivial) functionality port "_" and emits a fixed value.

```

1 constant(out_name: str, value: float, poset=None)

```

```

1 from codesign import constant
2 c = constant("base_cost", value=42.0)
3 print(c.h({"_": 0.0})) # Antichain[(base_cost=42)]

```

identity Passes a single named scalar through unchanged (same port name on both sides).

```

1 identity(name: str, poset=None)

```

```

1 from codesign import identity
2 i = identity("speed")
3 print(i.h({"speed": 7.0})) # Antichain[(speed=7)]

```

13.2.3 composition

The `composition` module supplies the three operators that close design problems under composition (Section 6): `Series`, `Parallel` and `Loop`, each with a lowercase functional alias (`series`, `par`, `loop`) matching the paper's notation. All three preserve monotonicity, so a composite is itself a `DesignProblem` and solves through the same `solve` entry point (13.2.4).

Series Series composition: `dp1` then `dp2`, with each resource of `dp1` fed in as a functionality of `dp2`. The composite has $F = dp1.F$ and $R = dp2.R$. When both interfaces are `Ports`, the connectable check `set(dp2.F.keys()) <= set(dp1.R.keys())` is enforced upfront; extra resource ports on `dp1` are permitted and ignored.

```

1 Series(dp1: DesignProblem, dp2: DesignProblem,
2       name: str | None = None)
3 # alias: series(dp1, dp2, name=None) → Series

```

```

1 from codesign import series, scale
2 s1 = scale("torque", "current", factor=2.0)
3 s2 = scale("current", "mass", factor=0.5)
4 chain = series(s1, s2)
5 print(chain.h({"torque": 4.0})) # Antichain[(mass=4)]

```

- `dp1`, `dp2` — the two stages; `dp1`'s resources must cover `dp2`'s functionality ports.

Errors: raises `ValueError` (naming the missing ports) when both interfaces are `Ports` and some port `dp2` requires is not produced by `dp1`.

Parallel Parallel composition: `dp1` and `dp2` stacked side by side. The composite has $F = F_1 \times F_2$ and $R = R_1 \times R_2$ (Ports concatenation), and its antichain is the Cartesian product of the two.

```

1 Parallel(dp1: DesignProblem, dp2: DesignProblem,
2         name: str | None = None)
3 # alias: par(dp1, dp2, name=None) → Parallel

```

```

1 from codesign import par, scale
2 p = par(scale("x", "cost_x", 1.0),
3         scale("y", "cost_y", 2.0))
4 print(p.h({"x": 3.0, "y": 4.0}))
5 # Antichain[(cost_x=3, cost_y=8)]

```

- `dp1`, `dp2` — must have disjoint F port names and disjoint R port names.

Errors: raises `TypeError` if either F or either R is not a `Ports`; raises `ValueError` if the F port names overlap, or if the R port names overlap.

Loop Feedback composition: closes a named `axis` present on both `inner.F` and `inner.R`, feeding the produced resource back as the consumed functionality (paper Def. 16, Section 6.3). The outer F and R drop the looped axis. Evaluating `h` defers to the solver's Kleene iteration (13.2.4).

```

1 Loop(inner: DesignProblem, axis: str,
2      name: str | None = None)
3 # alias: loop(inner, axis, name=None) → Loop

```

```

1 from codesign import loop, AlgebraicDP, solve, Ports, Reals
2 inner = AlgebraicDP(
3     F=Ports({"payload": Reals(), "mass": Reals()}),
4     R=Ports({"mass": Reals(), "power": Reals(unit="W")}),
5     equations={
6         "mass": lambda f: 0.5 * f["payload"] + 0.3 * f["mass"],
7         "power": lambda f: 2.0 * f["mass"],
8     })
9 lp = loop(inner, axis="mass") # feed R.mass back to F.mass
10 print(solve(lp, {"payload": 2.0}).antichain)
11 # Antichain[(power=2.857 W)]

```

- `inner` — the DP to close; needs `Ports` on both sides.
- `axis` — the port name (on both F and R) to feed back.

Errors: raises `TypeError` if `inner.F` or `inner.R` is not a `Ports`; raises `ValueError` if `axis` is not declared on both F and R of `inner`.

13.2.4 solver

The `solver` module drives every design problem through one entry point, `solve`, which runs the Kleene fixed-point iteration whenever a loop is present and returns a `SolveResult`. See Section 7 for the tutorial treatment and Section 7.4 for the observability features.

solve Solves a (possibly composite) design problem at a given functionality. For a `Loop` it iterates to the least fixed point; for any other DP it is a single evaluation of `h`. This is the single most-used function in the library.

```

1 solve(
2     dp: DesignProblem,
3     functionality: Mapping | None = None,
4     max_iter: int = 200,
5     *,
6     trace: bool = False,
7     verbose: int = 0,
8     on_iteration: Callable[[TraceEntry], None] | None = None,
9     start_from: SolveResult | Antichain | None = None,
10    record_trace: bool = False,
11    uncertainty: list[str] | None = None,
12    n_samples: int = 1000,
13    rng_seed: int | None = None,
14 ) → SolveResult

1 from codesign import AlgebraicDP, solve, minimize_cost, Ports, Reals
2 dp = AlgebraicDP(
3     F=Ports({"speed": Reals()}),
4     R=Ports({"power": Reals(unit="W")}),
5     equations={"power": lambda f: 2.0 * f["speed"]},
6 )
7 res = solve(dp, {"speed": 10.0}, trace=True)
8 print(res.status, res.iterations, res.feasible)
9 # converged 0 True
10 print(res.antichain) # Antichain[(power=20 W)]

```

- **functionality** — outer F values; when `None` the solver uses `dp.F.bottom()`.
- **max_iter** — cap on Kleene steps; reaching it sets `status="max_iter"`.
- **trace** — collect a per-step trace on `result.trace` (13.2.4).
- **verbose** — 0 silent, 1 final summary, 2 per-iteration feed.
- **on_iteration** — callback invoked with each `TraceEntry` as it is produced.
- **start_from** — warm-start seed: a prior `SolveResult` (its inner antichain is reused) or an `Antichain`; ignored for non-loop DPs.
- **record_trace** — legacy alias for `trace`.
- **uncertainty** — if given (labels "worst_case", "mean", "p95", "cvar95", "samples"), dispatches to the uncertainty solver (Section 8) and returns an `UncertaintyResult`.
- **n_samples, rng_seed** — Monte Carlo sample count and optional seed for stochastic uncertainty.

Errors: raises `TypeError` if `start_from` is not a `SolveResult`, an `Antichain`, or `None`. Non-convergence is *not* an error: it is reported through `status` ("max_iter" or "diverged") and `feasible`, so callers must inspect those fields.

kleene_loop The raw fixed-point iterator behind `solve`: it returns the loop's antichain directly (without the `SolveResult` wrapper), writing metadata into an optional `info_out` dict. Most callers should prefer `solve`.

```

1 kleene_loop(
2     loop_dp: Loop,
3     f_outer: Mapping | None,
4     max_iter: int = 200,
5     *,
6     trace: bool = False, verbose: int = 0,
7     on_iteration: Callable | None = None,
8     info_out: dict | None = None,
9     start_from: Antichain | None = None,

```

```
10 ) → Antichain
```

```
1 from codesign import loop, kleene_loop, AlgebraicDP, Ports, Reals
2 inner = AlgebraicDP(
3     F=Ports({"payload": Reals(), "mass": Reals()}),
4     R=Ports({"mass": Reals(), "power": Reals(unit="W")}),
5     equations={
6         "mass": lambda f: 0.5 * f["payload"] + 0.3 * f["mass"],
7         "power": lambda f: 2.0 * f["mass"],
8     })
9 lp = loop(inner, axis="mass")
10 print(kleene_loop(lp, {"payload": 2.0}))
11 # Antichain[(power=2.857 W)]
```

- **info_out** — receives iterations, status, inner_antichain, and (if trace) trace.
- **start_from** — seed antichain in the inner R poset; a cold start (bottom singleton) is used when None.

Errors: raises `TypeError` if **start_from** is not None and lacks a `points` attribute (i.e. is not an `Antichain`). Runaway numeric values are capped at `DIVERGENCE_CAP` (10^{30}) and reported as `status="diverged"` rather than raised.

minimize_cost Scalarises a solved antichain: returns the single resource bundle minimising a scalar `cost_fn`, or None when the result is infeasible or empty. This is the step that turns a Pareto front into one engineering choice.

```
1 minimize_cost(result: SolveResult,
2               cost_fn: Callable[[Mapping], float]) → Mapping | None
```

```
1 from codesign import AlgebraicDP, solve, minimize_cost, Ports, Reals
2 dp = AlgebraicDP(Ports({"speed": Reals()}),
3                 Ports({"power": Reals(unit="W")}),
4                 {"power": lambda f: 2.0 * f["speed"]})
5 res = solve(dp, {"speed": 10.0})
6 print(minimize_cost(res, cost_fn=lambda r: r["power"]))
7 # {'power': 20.0}
```

- **result** — a `SolveResult`, or any object with `feasible` and `antichain` attributes.
- **cost_fn(r)** — scalar cost of a resource bundle.

Errors: none raised directly; returns None on an infeasible or empty result.

SolveResult The dataclass returned by `solve`, carrying the front plus convergence and feasibility metadata.

```
1 from codesign import SolveResult, AlgebraicDP, solve, Ports, Reals
2 dp = AlgebraicDP(Ports({"x": Reals()}), Ports({"y": Reals()}),
3                 {"y": lambda f: f["x"]})
4 ac = solve(dp, {"x": 1.0}).antichain
5 sr = SolveResult(antichain=ac, iterations=3, status="max_iter")
6 print(sr.converged) # False
```

Field	Type	Meaning
<code>antichain</code>	<code>Antichain</code>	the (approximate) least fixed point
<code>iterations</code>	<code>int</code>	Kleene steps taken (0 if no loop)
<code>status</code>	<code>str</code>	"converged", "max_iter" or "diverged"
<code>feasible</code>	<code>bool</code>	False iff every point hit $\top$
<code>trace</code>	<code>list None</code>	per-step trace if <code>trace=True</code> , else None

The read-only property `converged` is a backward-compatibility alias for `status == "converged"`. Errors: none raised directly.

TraceEntry A dataclass recording one step of a Kleene iteration; a list of these is placed on `result.trace` when `trace=True`.

```
1 from codesign import solve, AlgebraicDP, Ports, Reals
2 dp = AlgebraicDP(Ports({"speed": Reals()}),
3                 Ports({"power": Reals()}),
4                 {"power": lambda f: 2.0 * f["speed"]})
5 te = solve(dp, {"speed": 10.0}, trace=True).trace[-1]
6 print(te.iteration, te.n_points, te.delta) # 1 1 None
```

Field	Type	Meaning
<code>iteration</code>	<code>int</code>	0 = seed, 1 = after first step, ...
<code>antichain</code>	<code>Antichain</code>	snapshot at this step
<code>n_points</code>	<code>int</code>	<code>len(antichain)</code>
<code>delta</code>	<code>float</code> <code>None</code>	max port change vs. the previous step; <code>None</code> at iteration 0
<code>elapsed_ms</code>	<code>float</code>	wall time for this step, in ms

Errors: none raised directly.

13.3 Builders and rendering

The two high-level builders — the declarative `mcdpl` front end and the modular `system` builder, with its module base class and `sugar` constraint DSL — together with the `diagram` and `viz` rendering helpers. See Sections 11–12 and 10.

13.3.1 mcdpl

The `codesign.mcdpl` module supplies a single builder class, `MCDP`, a thin declarative front-end that mirrors the paper’s `mcdp { ... }` concrete syntax in pure Python. The tutorial in Section 11 walks through the surface syntax; the card below is the reference summary.

MCDP Fluent builder for a single self-contained monotone design problem. One declares functionalities with `provides` and resources with `requires`, attaches one equation per resource with `constraint` (or a full antichain-valued rule), optionally closes feedback axes with `loop_on`, and emits a plain `DesignProblem` with `build`. Every declaration method returns `self`, so calls `chain`. It also works as a context manager.

Signature: `MCDP(name="mcdp").`

```
1 from codesign import MCDP, solve
2
3 with MCDP("battery") as m:
4     m.provides("capacity", unit="J")
5     m.requires("mass", unit="kg")
6     m.constraint("mass", lambda f: f["capacity"] / 1.8e6)
7 battery = m.build()
8
9 res = solve(battery, {"capacity": 3.6e6})
10 print(res.antichain) # Antichain[(mass=2 kg)]
```

Constructor parameters:

- **name** — `str`, optional (default `"mcdp"`). Names the emitted `DesignProblem` and appears in its errors.

Public methods:

Method	Behaviour
<code>provides(name, *, unit="", poset=None)</code>	Declare a functionality port. Defaults to <code>Reals(unit=unit)</code> ; pass <code>poset</code> to override. Returns <code>self</code> .
<code>requires(name, *, unit="", poset=None)</code>	Declare a resource port, same defaulting as <code>provides</code> . Returns <code>self</code> .
<code>constraint(resource, fn)</code>	Closed-form clause <code>resource >= fn(f)</code> where <code>f</code> is the functionality dict. Repeated calls on one resource are joined (max), matching MCDPL's multiple- $\geq$ semantics. Returns <code>self</code> .
<code>rule(fn)</code>	Hand-write the full <code>h: f -> Antichain</code> . Overrides all <code>constraint</code> clauses; use for multi-valued or branchy relations. Returns <code>self</code> .
<code>loop_on(axis)</code>	Close a feedback loop on <code>axis</code> , which must be declared on both the <code>provides</code> and <code>requires</code> sides; the axis is projected out of the closed loop's F/R . May be repeated. Returns <code>self</code> .
<code>build()</code>	Emit the plain <code>DesignProblem</code> (an <code>AlgebraicDP</code> , or a <code>FunctionDP</code> if <code>rule</code> was used), wrapped in a <code>Loop</code> per declared axis.

Errors: `loop_on` raises `ValueError` naming the missing side and listing the current `provides/requires` declarations if `axis` is absent from either side. `build` raises `ValueError` if no functionality was declared, if no resource was declared, or (in the closed-form path) if any required resource has neither a `constraint` nor a `rule`. The other methods raise nothing directly.

13.3.2 system

The `codesign.system` module supplies the `System` builder, the recommended surface for non-trivial models. Section 12 is the full tutorial (both the operator-overloaded and legacy-lambda syntaxes); the card below summarises the API.

System Modular MCDP composition with named subsystems. One declares outer functionalities (`provides`) and outer resources (`requires`), adds subsystems by name (`add`), wires them with connection constraints (`constrain`), and emits a single `DesignProblem` with `build`. The result solves like any other DP and can itself be nested as a subsystem. Internally `build` closes one feedback loop over the bundle of every subsystem's R ports (the hidden `__modules__` axis).

Signature: `System(name="system")`.

```

1 from codesign import Module, Reals, System, solve
2
3 class Battery(Module):
4     F = {"capacity": Reals(unit="J")}
5     R = {"mass": Reals(unit="kg")}
6     def h(self, f):
7         return {"mass": f["capacity"] / 1.8e6}
8
9 sys = System("drone")
10 endurance = sys.provides("endurance", unit="s")
11 total_mass = sys.requires("total_mass", unit="kg")
12 battery = sys.add("battery", Battery())
13 battery.capacity ≥ endurance * 5.0      # module F demand
14 total_mass ≥ battery.mass                # outer R demand

```

```

15 res = solve(sys.build(), {"endurance": 300.0})
16 print(res.antichain)           # total_mass = 0.0008333 kg

```

Constructor parameters:

- **name** — `str`, optional (default `"system"`). Names the emitted DP.

Public methods:

Method	Behaviour
<code>provides(name, *, unit="", poset=None)</code>	Declare an outer functionality. Returns a <code>Port</code> (kind <code>outer_f</code>) usable in constraint expressions.
<code>requires(name, *, unit="", poset=None)</code>	Declare an outer resource. Returns a <code>Port</code> (kind <code>outer_r</code>) usable as the LHS of a <code>>=</code> constraint.
<code>add(module_name, dp)</code>	Add a subsystem under a unique name; returns a <code>ModuleHandle</code> (see 13.3.4). Aliased as <code>sub</code> .
<code>constrain(target, demand)</code>	Register <code>target >= demand</code> . <code>target</code> is a <code>"module.port"</code> string, an outer- <code>R</code> name, or a <code>Port</code> ; <code>demand</code> is a <code>ctx</code> -dict callable or an <code>Expr</code> . Repeated calls on one target are joined (max). Aliased as <code>eq</code> .
<code>build()</code>	Validate and emit the composed <code>DesignProblem</code> .
<code>draw_diagram(**kwargs)</code>	Render a <code>GraphViz</code> block diagram; delegates to <code>draw_system</code> (see 13.3.5).

The operator-overloaded `>=` form (as in the snippet) is sugar for `constrain`: `Port.__ge__` registers the same internal constraint, so both syntaxes yield identical solves.

Errors: `provides/requires` raise `ValueError` on a duplicate name. `add` raises `ValueError` if the name is reused, contains a dot, equals the reserved `__modules__`, or if the DP's F/R are not `Ports`. `constrain` raises `TypeError` if `demand` is neither callable nor an `Expr`, or if `target` is a non-constrainable port kind. `build` (via `_validate`) raises `ValueError` when there are no subsystems and no outer resources, when no outer resource is declared, when a constraint targets an unknown module or port, or when a subsystem F port or outer R has no constraint. `draw_diagram` raises `ImportError` if `graphviz` is not installed.

13.3.3 module

The `codesign.module` module supplies `Module`, a class-based way to declare a design problem by subclassing rather than by wiring up posets and an `h_fn` by hand. It is the natural unit to hand to `System.add` (see 13.3.2).

Module Base class for declarative design problems. A subclass provides two class-level `dict[str, Poset]` attributes, `F` and `R`, and an instance method `h(self, f)`; the constructor wraps these into the underlying `DesignProblem` machinery, so an instance is itself a `DesignProblem`. The subclass `h` may return a `dict` (a singleton antichain), a list of dicts (multi-valued), or an `Antichain` directly — the return value is normalised automatically. An optional class attribute `module_name` overrides the default name (the lower-cased class name).

Signature: `Module()` (subclasses may override `__init__` to take parameters, calling `super().__init__()` last).

```

1 from codesign import Module, Reals, solve
2
3 class Battery(Module):
4     F = {"capacity": Reals(unit="J")}
5     R = {"mass":      Reals(unit="kg")}
6     def h(self, f):
7         return {"mass": f["capacity"] / 1.8e6}

```

```

8
9 battery = Battery()                                # a DesignProblem
10 print(battery.h({"capacity": 3.6e6}))              # Antichain[(mass=2 kg)]
11 print(solve(battery, {"capacity": 1.8e6}).antichain) # mass=1 kg

```

Class-level declarations (set by the subclass, not passed to `__init__`):

- `F` — `dict[str, Poset]`, functionality ports.
- `R` — `dict[str, Poset]`, resource ports. Units are carried by the individual posets (e.g. `Reals(unit="kg")`).
- `module_name` — `str`, optional name override.

`h(self, f)` The relation to override. Takes a functionality dict `f` keyed by the `F` port names, returns the resource requirement as a dict, a list of dicts, or an `Antichain`. The framework wraps non-antichain returns via `Antichain.singleton` / `Antichain.from_set`.

Errors: `__init__` raises `ValueError` if neither `F` nor `R` is declared. The default `h` raises `NotImplementedError` if a subclass fails to override it. The return normaliser raises `TypeError` if `h` returns something other than a dict, a list/tuple of dicts, or an `Antichain`.

13.3.4 sugar

The `codesign.sugar` module implements the operator-overloaded constraint DSL used by the `System` builder (Section 12). A `Port` is a typed handle onto one of a system's ports; arithmetic on ports builds an `Expr` tree lazily; `>=` on a constrainable port registers a constraint; `ModuleHandle` (returned by `System.add`) yields ports by attribute access; and `sqrt/exp/log` lift the corresponding `math` functions to operate on `Expr` trees.

The following listing exercises all six symbols; individual cards refer back to it.

```

1 from codesign import Module, Reals, System, solve, sqrt, exp, log
2
3 class Battery(Module):
4     F = {"capacity": Reals(unit="J")}
5     R = {"mass":      Reals(unit="kg")}
6     def h(self, f):
7         return {"mass": f["capacity"] / 1.8e6}
8
9 sys = System("s")
10 endurance = sys.provides("endurance", unit="s") # outer_f Port
11 total      = sys.requires("total_mass", unit="kg") # outer_r Port
12 bat        = sys.add("battery", Battery()) # ModuleHandle
13 e = (bat.mass + 0.5) * sqrt(endurance) # an Expr
14 print(e.pretty()) # ((battery.mass + 0.5) * sqrt(endurance))
15 bat.capacity >= endurance * 5.0 # >= registers a constraint
16 total >= bat.mass
17 print(solve(sys.build(), {"endurance": 300.0}).antichain)

```

Port A named handle onto a port of a subsystem or an outer *F/R* name. Its `kind` is one of `"module_f"`, `"module_r"`, `"outer_f"`, `"outer_r"`, which governs where it may appear: `module_f` and `outer_r` ports may be the LHS of a `>=` constraint; `module_r` and `outer_f` ports may appear on the demand (RHS) side. Ports are not built directly — they are returned by `System.provides/requires` and by `ModuleHandle` attribute access. Arithmetic operators (inherited from `Expr`) build expression trees; `>=` registers a constraint with the parent system.

- `full_name` — property; `"module.port"` for module ports, the bare name otherwise.
- `kind` — property; the port-kind string above.
- `pretty()` — the `full_name`, used in expression rendering.

Errors: `Port.__ge__` raises `TypeError` when the LHS is a `module_r` port (set by the module's `h`) or an `outer_f` port (a system input); both name the offending port.

Expr Base class for constraint-expression AST nodes (`Add`, `Sub`, `Mul`, `Div`, `Pow`, `Neg`, `Func`, `Const`, and `Port`). Users rarely name `Expr` directly; instances arise from arithmetic on ports and numbers. The arithmetic operators (`+`, `-`, `*`, `/`, `**` and unary `-`) return new `Expr` nodes rather than evaluating numerically, so the whole demand is captured as a tree that `System` later compiles to a callable.

- `pretty()` — a readable, fully parenthesised string of the tree (e.g. `((battery.mass + 0.5) * sqrt(endurance))`); also the `__repr__`.

Errors: `Expr.__ge__` raises `TypeError` (only ports, not compound expressions, may be a constraint LHS); `__le__` raises `TypeError` (constraints are written `target >= demand`, not reversed); `__bool__` raises `TypeError` to catch an expression used in a conditional. Coercing a non-numeric, non-`Expr` operand raises `TypeError`.

ModuleHandle The handle returned by `System.add`. Attribute access (`handle.port_name`) returns a `Port` of kind `module_f` or `module_r` depending on which side of the subsystem's DP the port lives on. Within one subsystem the *F* and *R* port names must be disjoint (enforced at construction).

- `name` — property; the subsystem's registered name.
- `dp` — property; the underlying `DesignProblem`.

Its `__repr__` lists the *F* and *R* ports, e.g. `<ModuleHandle 'battery': F=['capacity'] R=['mass']>`.

Errors: the constructor raises `ValueError` if the subsystem's *F* and *R* port names overlap; attribute access raises `AttributeError` (listing the available *F/R* ports) for an unknown port name.

sqrt Lift `math.sqrt` onto an expression: `sqrt(x) -> Expr`. The argument may be a `Port`, an `Expr`, or a number (coerced to a constant); the result is a `Func` node preserving the tree, as in the listing above (`sqrt(endurance)`). Errors: raises `TypeError` if the argument is neither a number nor an `Expr`.

exp Lift `math.exp` onto an expression: `exp(x) -> Expr`. Same argument rules as `sqrt` (see 13.3.4); e.g. `exp(endurance).pretty()` is `"exp(endurance)"`. Errors: raises `TypeError` on a non-number, non-`Expr` argument.

log Lift `math.log` (natural log) onto an expression: `log(x) -> Expr`. Same argument rules as `sqrt` (see 13.3.4); e.g. `log(endurance).pretty()` is `"log(endurance)"`. Errors: raises `TypeError` on a non-number, non-`Expr` argument.

13.3.5 diagram

The `codesign.diagram` module renders a `System` as a `GraphViz` block diagram — one box per subsystem with its *F* ports on the left and *R* ports on the right, outer *F/R* as margin nodes, constraint wiring as port-to-port edges, and any Kleene-iteration cycle coloured. See Section 10.1 for the rendered figures.

draw_system Build a `graphviz.Digraph` for a system. It accepts either a live `System` or the `DesignProblem` returned by `System.build()` (which carries the modules and constraints on its `_codesign_*` attributes). Constraints written in the operator-overloaded form resolve to individual ports; lambda-based constraints are drawn as a dashed edge from a λ marker so none are silently dropped. Building the `Digraph` object needs only the `graphviz` Python package; the `dot` binary is required only to *render* it to a file.

Signature:

```

1 draw_system(system, *, name=None, rankdir="LR", show_ports=True,
2             highlight_cycles=True, graph_attrs=None)

1 from codesign import Module, Reals, System, draw_system
2
3 class Battery(Module):
4     F = {"capacity": Reals(unit="J")}
5     R = {"mass": Reals(unit="kg")}
6     def h(self, f):
7         return {"mass": f["capacity"] / 1.8e6}
8
9 sys = System("drone")
10 endurance = sys.provides("endurance", unit="s")
11 total_mass = sys.requires("total_mass", unit="kg")
12 battery = sys.add("battery", Battery())
13 battery.capacity ≥ endurance * 5.0
14 total_mass ≥ battery.mass
15
16 dot = draw_system(sys, rankdir="LR") # a graphviz.Digraph
17 # dot.render("drone", format="svg") # writes drone.svg (needs dot)

```

Parameters:

- **system** — a live System or a built System DP.
- **name** — str or None; diagram title, defaults to `system.name`.
- **rankdir** — "LR" (default, horizontal) or "TB" (vertical, for tall systems).
- **show_ports** — bool; when False, only module names are shown and edges attach to the box body.
- **highlight_cycles** — bool; colour edges inside strongly-connected components of size > 1.
- **graph_attrs** — dict of extra GraphViz graph attributes (e.g. {"ranksep": "0.6"}).

Errors: raises `ImportError` if the `graphviz` package is not installed; raises `TypeError` if `system` is neither a System nor a built System DP (i.e. lacks the `_codesign_modules` attribute). The convenience method `System.draw_diagram` forwards to this function.

13.3.6 viz

The `codesign.viz` module collects ready-made diagnostics. It is reached via the namespace import `from codesign import viz` (its functions are deliberately not in the top-level `__all__`) and called as `viz.plot_antichain(...)`, and so on. The three `plot_*` functions require `matplotlib` (imported lazily, so the rest of the package works without it) and return a `matplotlib.axes.Axes`; `to_dot` has no plotting dependency and returns a string. Section 10 shows the figures in context.

The snippet below builds a small drone system used by every card that follows; the `plot_*` calls run headless under `MPLBACKEND=Agg`.

```

1 from codesign import Module, Reals, System, solve, viz
2
3 # ... define Battery (F: capacity → R: mass) and Actuator
4 # ... (F: lift_force → R: power) Modules as in earlier cards
5 sys = System("drone")
6 endurance = sys.provides("endurance", unit="s")
7 power_out = sys.requires("power", unit="W")
8 mass_out = sys.requires("total_mass", unit="kg")
9 b = sys.add("battery", Battery())
10 a = sys.add("actuator", Actuator())
11 b.capacity ≥ a.power * endurance
12 a.lift_force ≥ 9.81 * b.mass
13 power_out ≥ a.power
14 mass_out ≥ b.mass

```

```

15 dp = sys.build()
16 res = solve(dp, {"endurance": 300.0}, trace=True)

```

viz.plot_antichain Scatter an antichain's Pareto front on two or three chosen R -port axes, optionally shading the dominated region (2D only). Accepts a `SolveResult`, an `UncertaintyResult` (uses its worst-case antichain), or an `Antichain`.

Signature:

```

1 plot_antichain(result, axes, *, ax=None, title=None,
2               shade_dominated=True, point_size=60.0, label=None)

1 ax = viz.plot_antichain(res, ["power", "total_mass"])

```

Parameters: `result` the object to plot; `axes` a sequence of two or three R -port names; `ax` an existing axes (a new figure is made if omitted); `title`; `shade_dominated`; `point_size`; `label` (a legend entry is drawn only when named).

Errors: raises `TypeError` if `result` is not one of the three accepted types; `ValueError` if `axes` has other than 2 or 3 names, or if no point is plottable (empty, infeasible, or non-numeric); `ImportError` if `matplotlib` is absent.

viz.plot_convergence Semilog plot of the per-iteration Kleene delta from a solve trace. Accepts a `SolveResult` whose `trace` was populated (call `solve(..., trace=True)`) or the trace list directly; zero deltas are clamped to `floor` for the log scale.

Signature:

```

1 plot_convergence(result_or_trace, *, ax=None, floor=1e-18,
2                 title="Kleene-iteration convergence", label=None)

1 ax = viz.plot_convergence(res) # res from solve(..., trace=True)

```

Parameters: `result_or_trace`; `ax`; `title`; `floor` (lower clamp for zero deltas); `label`.

Errors: raises `ValueError` if the trace has no numeric deltas; `ImportError` if `matplotlib` is absent.

viz.plot_uncertainty Histogram of Monte-Carlo samples on one R port, with vertical lines for each requested summary (mean, p95, CVaR95, worst case) and an optional nominal marker. The `UncertaintyResult` must have been produced with "samples" in the `uncertainty` list.

Signature:

```

1 plot_uncertainty(result, port, *, ax=None, bins=30, title=None,
2                 nominal=None, show_summaries=True, xlabel=None,
3                 legend_loc="best")

1 ures = solve(dp, {"endurance": 300.0},
2             uncertainty=["mean", "p95", "samples"],
3             n_samples=200, rng_seed=0)
4 ax = viz.plot_uncertainty(ures, "total_mass",
5                           xlabel="total mass (kg)")

```

Parameters: `result` an `UncertaintyResult` with samples; `port` the R port to histogram; `bins`; `nominal`; `show_summaries`; `xlabel` (defaults to the raw port name); `legend_loc`.

Errors: raises `ValueError` if the result carries no samples, or if the chosen port has no finite values; `ImportError` if `matplotlib` is absent.

viz.to_dot Emit a GraphViz dot string describing a DP's structure: a System-built DP renders its subsystems and constraint edges, a composition tree (**Series/Parallel/Loop**) renders as nested boxes, and a leaf DP renders as a single box. Returning a string keeps the module free of the graphviz dependency; save it and run `dot -Tsvg`.

Signature: `to_dot(dp, *, name="codesign") -> str`.

```
1 print(viz.to_dot(dp, name="drone").splitlines()[0])
2 # digraph drone {
3 print(viz.to_dot(dp, name="my drone").splitlines()[0])
4 # digraph "my drone" {    (non-identifier name is quoted)
```

Parameters: **dp** the design problem to describe; **name** the graph id — used bare when it is a valid GraphViz identifier, otherwise double-quoted with embedded quotes and backslashes escaped.

Constraint edges track the modules referenced on the right-hand side even when a module port is wrapped in a function or negation — the reference walker descends through `.inner` (`sqrt(...)`, `exp`, a leading minus) as well as `.left/.right/.arg`, so a constraint like `bat.capacity >= sqrt(act.power)` still draws the `act→bat` edge.

Errors: none raised directly; the walker swallows introspection failures and returns a best-effort graph.

13.4 Uncertainty and online learning

The set-based and stochastic **uncertainty** layer and the compositional **online-learning** layer of optimistic evaluators. The modelling discussion is in Sections 8 and 9.

13.4.1 uncertainty

Reference cards for the uncertainty layer of module `codesign.uncertainty`. Both regimes are declared on a Module through its `uncertain_set` (set-based) and `uncertain_dist` (stochastic) attributes, and both are consumed by the ordinary `solve` entry point; see Section 8 for the modelling discussion. The set-based classes (`UncertaintySet`, `Box`, `Ellipsoid`, `Disk`, `Circle`) are grouped first, the stochastic ones (`Independence`, `GaussianCopula`, `Stochastic`) after, followed by the result dataclass and the solver entry point. Every listing requires `numpy`; the stochastic cards additionally require `scipy`.

UncertaintySet Abstract base for deterministic, set-based parameter uncertainty. A concrete subclass declares which parameters vary (`param_names`) and where the set's worst point lies (`worst_case_values`); the worst-case solve then reassigns those named module attributes before re-solving. Signature: `UncertaintySet()` (no constructor of its own).

```
1 from codesign import Box
2 from codesign.uncertainty import UncertaintySet
3 s = Box(mass=(1.0, 2.0, "more_is_worse"))
4 print(isinstance(s, UncertaintySet))    # True
5 print(s.param_names())                  # ['mass']
```

Public methods (both abstract, overridden by subclasses):

- `param_names()` -> `list[str]` — names of the uncertain parameters carried by the set.
- `worst_case_values(h_callable, base_values, f_inner)` -> `dict[str, float]` — the parameter dict at the worst point of the set. `h_callable(overrides)` -> `Antichain` is evaluated only for undeclared directions (boundary search); `base_values` supplies nominal values for out-of-set parameters.

Errors: the base methods raise `NotImplementedError`; instantiate a concrete subclass instead.

Box Axis-aligned interval product over one or more parameters — the everyday set-based model. When every parameter declares a direction of badness the worst case is a single corner, read off in constant time. Signature: `Box(**params)` where each value is (lo, hi) or (lo, hi, direction).

```
1 from codesign import Box
2 b = Box(se=(1.7e6, 2.3e6, "more_is_better"),
3         eff=(0.83, 0.97, "more_is_better"))
4 print(b.param_names())           # ['se', 'eff']
5 print(b.worst_case_values(None, {}, {}))
6 # {'se': 1700000.0, 'eff': 0.83}
```

Parameters:

- ****params** — one keyword per uncertain parameter. The value is (lo, hi) (direction undeclared) or (lo, hi, direction) with direction one of "more_is_better", "more_is_worse", "less_is_better", "less_is_worse" (in parameter units).

With n undeclared parameters all 2^n endpoint corners are probed (cheap for modest n). Errors: raises `ValueError` if a value is neither a 2- nor 3-tuple, if an unknown direction token is passed, or if $\text{lo} > \text{hi}$ (empty interval).

Ellipsoid An n -D ellipsoid $(p - c)^\top \Sigma^{-1} (p - c) \leq 1$ in parameter space; the covariance Σ captures scale and correlation, so it is a more honest set than a box when parameters vary together.

```
1 from codesign import Ellipsoid
2 e = Ellipsoid(
3     center={"se": 2.0e6, "eff": 0.9},
4     cov=[[1.0e10, 0.0], [0.0, 2.5e-3]],
5     params=["se", "eff"],
6     directions={"se": "more_is_better",
7                 "eff": "more_is_better"},
8 )
9 print(e.worst_case_values(None, {}, {}))
10 # {'se': 1929289.32..., 'eff': 0.86464...}
```

Parameters:

- **center** — dict of centre coordinates by parameter name.
- **cov** — symmetric positive-definite shape matrix Σ , rows/columns ordered as **params**.
- **params** — parameter names in **cov**'s order.
- **directions=None** — optional per-parameter badness tokens; when all are given the worst case is the analytic boundary point $c + L u^*$, else the boundary is sampled.
- **boundary_samples=8** — samples per dimension used when directions are not fully declared.

Errors: raises `ValueError` if **cov**'s shape does not match **params** or if **cov** is not positive definite; `ImportError` if `numpy` is absent.

Disk Two-dimensional convenience: a filled circle, i.e. the isotropic ellipsoid $\Sigma = r^2 I$. A factory function returning an `Ellipsoid`, not a class. Signature: `Disk(center, radius, params=None, directions=None)`.

```
1 from codesign import Disk, Ellipsoid
2 d = Disk(center={"x": 0.0, "y": 0.0}, radius=1.0,
3           directions={"x": "more_is_worse",
4                       "y": "more_is_worse"})
5 print(isinstance(d, Ellipsoid))   # True
6 print(d.worst_case_values(None, {}, {}))
7 # {'x': 0.70710..., 'y': 0.70710...}
```


Parameters: `center` (2-key dict), `radius` (in parameter units), `params` (inferred from `center` if None), `directions` (as for `Ellipsoid`). Errors: raises `ValueError` unless exactly two parameters are supplied.

Circle Two-dimensional convenience for the circle boundary only. For monotone modules with declared directions the worst case lies on the boundary, so `Circle` and `Disk` return identical worst-case answers; the implementation delegates to `Disk`. Signature: `Circle(center, radius, params=None, directions=None)`.

```
1 from codesign import Circle, Ellipsoid
2 c = Circle(center={"x": 0.0, "y": 0.0}, radius=2.0,
3           directions={"x": "more_is_worse",
4                      "y": "more_is_worse"})
5 print(isinstance(c, Ellipsoid))           # True
6 print(c.worst_case_values(None, {}, {}))
7 # {'x': 1.41421..., 'y': 1.41421...}
```

Parameters: as `Disk`. Errors: as `Disk` (raises `ValueError` unless exactly two parameters are supplied).

Independence The independence copula: each component drawn independently in $U(0,1)$. The default dependence structure of `Stochastic`. Signature: `Independence()`; method `sample_uniform(n, d, rng) -> ndarray of shape (n, d)`.

```
1 from codesign.uncertainty import Independence
2 import numpy as np
3 rng = np.random.default_rng(0)
4 u = Independence().sample_uniform(3, 2, rng)
5 print(u.shape)                       # (3, 2)
```

Parameters: none. `sample_uniform` takes the sample count `n`, the dimension `d`, and a numpy `Generator` `rng`. Errors: none raised directly.

GaussianCopula Gaussian copula with a given correlation matrix: draw $z \sim N(0, R)$ and apply Φ to each coordinate to obtain correlated uniforms. Signature: `GaussianCopula(correlation)`.

```
1 from codesign import GaussianCopula
2 import numpy as np
3 gc = GaussianCopula(correlation=[[1.0, 0.4],
4                                [0.4, 1.0]])
5 rng = np.random.default_rng(0)
6 u = gc.sample_uniform(4, 2, rng)
7 print(u.shape, repr(gc))
8 # (4, 2) GaussianCopula(d=2)
```

Parameters: `correlation` — a (d, d) symmetric positive-definite matrix with unit diagonal. Errors: the constructor raises `ValueError` if `correlation` is not square or not positive definite; `sample_uniform` raises `ValueError` if `d` differs from the copula's dimension. Sampling additionally requires `scipy`.

Stochastic Joint distribution over uncertain parameters, built from named `scipy.stats` frozen marginals plus an optional copula. This is the `uncertain_dist` declaration consumed by the Monte Carlo path of `solve`; it has a full treatment in Section 8. Signature: `Stochastic(marginals=None, copula=None, **marginals_kw)`.

```
1 from codesign import Stochastic
2 from scipy import stats
3 import numpy as np
```

```

4 st = Stochastic(marginals={
5     "m": stats.norm(loc=1.0, scale=0.1)})
6 print(st.param_names())           # ['m']
7 print(st.sample(2, np.random.default_rng(0)))
8 # [{'m': 1.0350...}, {'m': 0.9386...}]

```

Parameters:

- **marginals** — dict of parameter name to a frozen scipy distribution (e.g. `stats.norm(loc, scale)`). Marginals may also be given as keyword arguments (****marginals_kw**).
- **copula=None** — a Copula; defaults to Independence.

Methods: `param_names()` -> `list[str]`; `sample(n, rng)` -> `list[dict]` of n parameter dicts (copula uniforms mapped through each marginal's ppf). Errors: the constructor raises `ValueError` if no marginal is supplied.

UncertaintyResult Dataclass returned by an uncertainty-aware solve. Which fields are populated depends on the summaries requested via `uncertainty=[...]`; the rest stay `None`.

```

1 from codesign.uncertainty import UncertaintyResult
2 r = UncertaintyResult(mean={"mass": 0.55},
3                       feasibility_rate=1.0,
4                       n_samples_used=1000)
5 print(r.mean, r.feasibility_rate)
6 # {'mass': 0.55} 1.0

```

Fields:

Field	Type	Meaning
<code>worst_case</code>	<code>SolveResult</code> or <code>None</code>	result at the set's worst point
<code>mean</code>	<code>dict</code> or <code>None</code>	per-R-port MC mean
<code>p95</code>	<code>dict</code> or <code>None</code>	per-R-port 95th percentile
<code>cvar95</code>	<code>dict</code> or <code>None</code>	per-R-port CVaR at 95%
<code>samples</code>	<code>list</code> or <code>None</code>	raw MC antichains
<code>feasibility_rate</code>	<code>float</code> or <code>None</code>	feasible-sample fraction
<code>n_samples_used</code>	<code>int</code>	MC samples drawn (0 if none)

Statistics are computed over feasible samples only. Errors: none raised directly (a plain dataclass).

solve_with_uncertainty Entry point for an uncertainty-aware solve, returning an `UncertaintyResult`. Ordinarily reached through `solve(dp, f, uncertainty=[...])`, which dispatches here when the `uncertainty` argument is present. Signature:

```

1 solve_with_uncertainty(dp, functionality, uncertainty,
2     n_samples=1000, rng_seed=None, max_iter=200,
3     verbose=0)

```

```

1 from codesign import Module, Reals, solve, Box
2 class Battery(Module):
3     F = {"capacity": Reals(unit="J")}
4     R = {"mass": Reals(unit="kg")}
5     def __init__(self, se=2.0e6, eff=0.9):
6         self.se, self.eff = se, eff
7         super().__init__()
8     def h(self, f):
9         return {"mass": f["capacity"] / (self.se * self.eff)}
10 bat = Battery()
11 bat.uncertain_set = Box(se=(1.7e6, 2.3e6, "more_is_better"),

```

```

12         eff=(0.83, 0.97, "more_is_better"))
13 res = solve(bat, {"capacity": 1.0e6}, uncertainty=["worst_case"])
14 print(round(list(res.worst_case.antichain.points)[0]["mass"], 5))
15 # 0.70872

```

Parameters:

- **dp, functionality** — the design problem and its outer F vector, as for `solve`.
- **uncertainty** — list drawn from "worst_case", "mean", "p95", "cvar95", "samples".
- **n_samples=1000** — Monte Carlo draws for the stochastic summaries; **rng_seed=None** seeds the numpy generator.
- **max_iter=200, verbose=0** — forwarded to each inner solve.

Errors: raises `ValueError` for an unknown summary label, when no module on **dp** carries an `uncertain_set/uncertain_dist`, or when stochastic summaries are requested but no `uncertain_dist` is present; `ImportError` if numpy is absent.

13.4.2 online

Reference cards for the compositional online learner of module `codesign.online` — a port of Alharbi, Dahleh & Zardini (arXiv:2604.22624). The evaluators maintain history-dependent bounds on each candidate's inner-solve output so that provably suboptimal candidates are eliminated without ever being solved; `solve_online` is the budgeted driver. See Section 9 for the modelling discussion and Section 9.3 for the warm-start and picker options. Every listing requires numpy; `LinearParametricEvaluator` additionally requires scipy.

OptimisticEvaluator Abstract base holding the observation history and the bound contract. The public `bound(candidate)` returns a pair of dicts (**lower**, **upper**) mapping each numeric R component to its current lower and upper bound at the queried feature point; the base implementation is the trivial fallback $(0, +\infty)$, which subclasses tighten. Signature: `OptimisticEvaluator(features, r_components)`.

```

1 from codesign.online import OptimisticEvaluator
2 ev = OptimisticEvaluator(features=["x"],
3                           r_components=["cost"])
4 print(ev.bound({"x": 2.0}))
5 # ({'cost': 0.0}, {'cost': inf})

```

Parameters: **features** — candidate keys used as the feature vector; **r_components** — resource-port names to bound.

Method	Behaviour
<code>reset()</code>	forget every observation
<code>observe(candidate_id, candidate, antichain)</code>	record an inner-solve result
<code>bound(candidate)</code>	return (lower , upper) bound dicts

Errors: `observe/bound` raise `KeyError` if a candidate is missing a declared feature.

MonotonicityEvaluator Bounds from monotonicity alone: assuming the output is component-wise monotone in the features, an observation at f_0 with antichain-min R_0 lower-bounds every candidate with features $\geq f_0$ by R_0 and upper-bounds every candidate with features $\leq f_0$ by R_0 . The aggressive choice when the assumption genuinely holds. Signature: `MonotonicityEvaluator(features, r_components)`.

```

1 from codesign import (MonotonicityEvaluator, Antichain,
2                       Ports, Reals)
3 R = Ports({"cost": Reals()})

```

```

4 ev = MonotonicityEvaluator(["x"], ["cost"])
5 ev.observe(0, {"x": 1.0}, Antichain.singleton(R, {"cost": 5.0}))
6 print(ev.bound({"x": 2.0}))    # ({'cost': 5.0}, {'cost': inf})
7 print(ev.bound({"x": 0.5}))    # ({'cost': 0.0}, {'cost': 5.0})

```

Parameters: as `OptimisticEvaluator`. Behaviour is described, not its implementation: the bound is exact to float comparisons, computed by a vectorised numpy rescan of the whole history when numpy is present, with an equivalent pure-Python fallback for numpy-absent installs. Errors: raises `KeyError` for a missing feature.

LipschitzEvaluator Bounds from a Lipschitz assumption: with $|h(c_1) - h(c_2)| \leq L \|features(c_1) - features(c_2)\|$, each observation tightens the bound everywhere by a cone of slope L . Signature: `LipschitzEvaluator(features, r_components, L)`.

```

1 from codesign import (LipschitzEvaluator, Antichain,
2                       Ports, Reals)
3 R = Ports({"cost": Reals()})
4 ev = LipschitzEvaluator(["x"], ["cost"], L=2.0)
5 ev.observe(0, {"x": 1.0}, Antichain.singleton(R, {"cost": 5.0}))
6 print(ev.bound({"x": 2.0}))    # ({'cost': 3.0}, {'cost': 7.0})

```

Parameters: L — a single positive float (same constant for every R component) or a dict mapping R component name to its own constant. Behaviour, not implementation: distances use a vectorised numpy path when numpy is present and an equivalent pure-Python fallback otherwise. Errors: raises `KeyError` for a missing feature, or if L is a dict lacking an R component.

LinearParametricEvaluator The *certified* optimistic bound of Alharbi *et al.* (Section V-C3, eqs. 26–28 and Lemma V.5). Each resource coordinate is assumed an exact affine function of the features, $req_k(c) = \phi(c)^\top \theta_k^*$ with $\phi(c) = [1, features(c)]$; the evaluator maintains the confidence polytope $\Theta(H)$ of parameters consistent with every observation and returns, per coordinate, $\min_{\theta \in \Theta(H)} \phi(i)^\top \theta$ — one LP solved with `scipy.optimize.linprog`. Because θ_k^* is feasible, the optimum is $\leq req_k(i)$: a *guaranteed* lower bound (the optimism guarantee), so a Pareto-optimal candidate is never wrongly eliminated. No upper bound is certified, so the returned upper bound stays at $+\infty$. Signature:

```

1 LinearParametricEvaluator(features, r_components,
2                           confidence=None, min_obs=3, *, prior_box=None,
3                           noise_bound=0.0, solver="highs")

1 from codesign import (LinearParametricEvaluator, Antichain,
2                       Ports, Reals)
3 R = Ports({"cost": Reals()})
4 ev = LinearParametricEvaluator(["x"], ["cost"],
5                               prior_box=(-100.0, 100.0))
6 for x in (1.0, 2.0, 3.0):      # cost = 2x + 1, affine
7     ev.observe(0, {"x": x},
8               Antichain.singleton(R, {"cost": 2*x + 1}))
9 lo, hi = ev.bound({"x": 4.0})  # certified lower bound
10 print(round(lo["cost"], 3), hi["cost"]) # 9.0 inf

```

Parameters:

- `prior_box=None` — the prior box Θ_0 keeping the LP bounded while the fit is under-determined. `None` leaves every parameter unbounded (always safe, but trivial until θ is pinned down); a (lo, hi) float pair applies to every parameter; a dict gives each R component its own (lo, hi) . A finite box must contain the true θ^* or the bound may become invalid.
- `noise_bound=0.0` — half-width of the observation band $|\phi(j)^\top \theta - r| \leq \text{noise_bound}$; `0.0` is the paper’s exact/noiseless model, positive values a documented bounded-noise extension.

- `min_obs=3` — below this many observations the trivial $(0, +\infty)$ bound is returned.
- `solver="highs"` — method forwarded to `linprog`.
- `confidence=None` — **deprecated and ignored**, retained only for backward compatibility with the former OLS $\pm$ confidence-band version; passing it emits a `DeprecationWarning`. The certified bound has no confidence-band parameter.

Errors: raises `ValueError` if `noise_bound` is negative; `TypeError` if a `prior_box` entry is not a (lo, hi) pair; `ImportError` if `numpy` or `scipy` is absent. An unbounded, infeasible, or failed LP degrades safely to the trivial lower bound.

GaussianProcessEvaluator Bounds from a zero-mean Gaussian-process surrogate with an RBF kernel, fitted in pure `numpy`; the bound at a query is $\text{mean} \pm \text{confidence} \cdot \sigma$. The right tool when the response surface has feature interactions or local nonlinearity, where the certified linear evaluator would degrade to the no-information bound. The bounds are informative but *not* certified. Signature:

```
1 GaussianProcessEvaluator(features, r_components,
2     length_scale=0.3, sigma_f=1.0, noise=1e-3,
3     confidence=2.0, min_obs=3)

1 from codesign import (GaussianProcessEvaluator, Antichain,
2     Ports, Reals)
3 R = Ports({"cost": Reals()})
4 ev = GaussianProcessEvaluator(["x"], ["cost"],
5     length_scale=0.5)
6 for x in (0.0, 0.5, 1.0):
7     ev.observe(0, {"x": x},
8         Antichain.singleton(R, {"cost": x}))
9 lo, hi = ev.bound({"x": 0.25})
10 print(round(lo["cost"], 3), round(hi["cost"], 3))
11 # 0.066 0.289
```

Parameters:

- `length_scale=0.3` — RBF length scale (0.3 suits features in roughly $[0, 1]$).
- `sigma_f=1.0` — kernel amplitude, rescaled per output by the empirical standard deviation.
- `noise=1e-3` — observation-noise jitter stabilising the Cholesky factor.
- `confidence=2.0` — multiplier on the predictive standard deviation ($\approx 95\%$ band under Gaussian residuals).
- `min_obs=3` — below this the fallback $(0, +\infty)$ bound is returned.

`observe` additionally invalidates the fit cache (a lazy refit runs on the next `bound`). Errors: raises `KeyError` for a missing feature; `ImportError` if `numpy` is absent.

OnlineResult Dataclass returned by `solve_online`, carrying the surviving antichain plus a full audit of the elimination cascade.

```
1 from codesign import OnlineResult, Antichain, Ports, Reals
2 R = Ports({"cost": Reals()})
3 r = OnlineResult(
4     antichain=Antichain.singleton(R, {"cost": 5.0}),
5     n_evaluated=3, n_eliminated=7, n_candidates=10)
6 print(r.n_evaluated, r.n_eliminated, r.n_candidates)
7 # 3 7 10
```

Fields:

Field	Type	Meaning
<code>antichain</code>	<code>Antichain</code>	Min over evaluated survivors
<code>n_evaluated</code>	<code>int</code>	inner solves run
<code>n_eliminated</code>	<code>int</code>	candidates pruned by bounds
<code>n_candidates</code>	<code>int</code>	candidates at start
<code>history</code>	<code>list[dict]</code>	per-iteration log; each entry has keys <code>pick</code> , <code>antichain</code> , <code>remaining</code> , <code>evaluated</code>
<code>evaluated_ids</code>	<code>list[int]</code>	indices actually evaluated
<code>eliminated_ids</code>	<code>list[int]</code>	indices pruned
<code>incumbent_ids</code>	<code>list[int]</code>	indices in the final antichain

Errors: none raised directly (a plain dataclass).

`solve_online` The budgeted online driver: `solve`'s elimination-based cousin. For each candidate, `candidate_fn` builds a fresh DP; the evaluator's bounds prune provably suboptimal candidates before their inner solve is run. Signature:

```

1 solve_online(candidate_fn, functionality, *,
2               candidates, evaluator, budget=None, max_iter=200,
3               verbose=0, warm_start=None, picker="lcb")

1 from codesign import (AlgebraicDP, Ports, Reals,
2                       LipschitzEvaluator, solve_online)
3 F, R = Ports({"load": Reals()}), Ports({"cost": Reals()})
4 def candidate_fn(c):
5     return AlgebraicDP(F, R, {"cost":
6         lambda f, u=c["price"]: f["load"] * u})
7 cands = [{"price": p} for p in (2.0, 3.0, 5.0, 8.0)]
8 ev = LipschitzEvaluator(["price"], ["cost"], L=10.0)
9 res = solve_online(candidate_fn, {"load": 1.0},
10                   candidates=cands, evaluator=ev,
11                   warm_start=2, picker="ucb")
12 print(res.n_evaluated, res.n_eliminated, res.antichain)
13 # 4 0 Antichain[(cost=2)]

```

Parameters:

- `candidate_fn` — `candidate_fn(candidate) -> DP`.
- `functionality` — the outer `F` vector, passed to every inner solve unchanged.
- `candidates` — list of feature dicts; each must contain every name in `evaluator.features`.
- `evaluator` — the `OptimisticEvaluator` instance (reset and repopulated during the solve).
- `budget=None` — maximum inner solves; `None` is unlimited (elimination still prunes).
- `max_iter=200, verbose=0` — forwarded to each inner solve; `verbose` 0/1/2 for silent/summary/trace.
- `warm_start=None` — seed evaluations before the picker takes over: an `int n` triggers greedy farthest-point sampling of `n` candidates; a list of indices seeds exactly those.
- `picker="lcb"` — selection strategy: `"lcb"` (exploit), `"ucb"` (add exploration bonus, tune via `("ucb", {"kappa": 1.0})`), `"random"`, or a callable (`lo, hi, r_components, **kwargs`) `-> score`.

Errors: raises `ValueError` for an unknown picker name and `TypeError` for a picker of the wrong type; `KeyError` if a candidate is missing a declared feature.

13.5 Temporal layers

The open-loop temporal planners: architecture switching (`temporal`), scalar dynamic programming (`dynamic`), and antichain-valued sequential co-design (`sequential`). See Section 15.

13.5.1 temporal

The `temporal` module implements Case 1 of the temporal co-design layers (section 15): architecture *switching* across a changing environment. A sequence of epochs each poses its own functionality and admits a set of candidate architectures; `solve_schedule` chooses one architecture per epoch to minimise the total of the per-epoch co-design costs plus the switching costs, by an exact Viterbi (shortest-path) pass over the epoch-by-architecture lattice. The per-epoch cost is an ordinary `solve()`, so this layer sits on top of static co-design without modifying it. The `Architecture` defined here is the shared decision object reused by every temporal layer.

Architecture A named candidate configuration that can be active in an epoch or stage; the common decision object of the whole temporal family. It is a dataclass wrapping a design problem.

```
1 Architecture(name: str, dp: Any, tags: Mapping = {})  
  
1 from codesign import AlgebraicDP, Architecture, Ports, Reals, System  
2 s = System("eco")  
3 d = s.provides("demand")  
4 s.requires("cost")  
5 s.add("m", AlgebraicDP(F=Ports({"demand": Reals()}),  
6     R=Ports({"cost": Reals()}),  
7     equations={"cost": lambda f: 10.0}).demand ≥ d  
8 s.constrain("cost", lambda x: x["m.cost"])  
9 eco = Architecture("eco", s.build(), tags={"mode": "eco"})  
10 print(eco.name, eco.tags)           # → eco {'mode': 'eco'}
```

- `name` — `str` — identifier used in results, plots and switching bookkeeping.
- `dp` — `Any` — a design problem accepted by `solve()` (typically a built `System`); may be shared across epochs or epoch-specific.
- `tags` — `Mapping` — free-form metadata carried through to results; not interpreted by the solver.

Errors: none raised directly.

Epoch One environment regime over which a single architecture is active. The changing environment enters through the per-epoch functionality.

```
1 Epoch(name: str, functionality: Mapping,  
2     candidates: Optional[Sequence[Architecture]] = None,  
3     duration: float = 1.0)  
  
1 from codesign import Architecture, Epoch  
2 arch = Architecture("eco", None, tags={"mode": "eco"})  
3 ep = Epoch("glucose", {"mu": 0.8}, candidates=[arch], duration=2.0)  
4 print(ep.name, ep.functionality, ep.duration)  
5 # → glucose {'mu': 0.8} 2.0
```

- `name` — `str` — epoch identifier.
- `functionality` — `Mapping` — the outer functionality demanded in this epoch, passed straight to `solve()`.
- `candidates` — `Sequence[Architecture]` or `None` — architectures admissible here; when `None` the schedule-level default set is used, so some architectures can be made unavailable in some regimes.
- `duration` — `float` — weight on the epoch's running cost, expressing unequal epoch lengths without rescaling the cost function.

Errors: none raised directly (a missing default set is reported by `solve_schedule`; see below).

EpochResult The per-epoch outcome recorded inside a solved schedule. A dataclass; its `total_cost` property sums the running and switching contributions.

```
1 from codesign import EpochResult
2 er = EpochResult(epoch="glucose", architecture="eco",
3                 running_cost=8.0, switch_cost=0.5, feasible=True,
4                 point={"cost": 8.0})
5 print(er.total_cost)           # → 8.5
```

- `epoch` — `str` — the epoch's name.
- `architecture` — `str` — name of the chosen architecture.
- `running_cost` — `float` — duration-weighted scalar cost of the chosen design point (inf if infeasible).
- `switch_cost` — `float` — switching cost charged at this epoch's boundary (zero when the architecture is unchanged).
- `feasible` — `bool` — whether this epoch was satisfiable by its chosen architecture.
- `point` — `Mapping` or `None` — the chosen resource point.
- `tags` — `Mapping` — the chosen architecture's tags.
- `total_cost` (property) — `running_cost` + `switch_cost`.

Errors: none raised directly.

ScheduleResult The outcome of `solve_schedule`: the chosen architecture per epoch with decomposed costs. A dataclass with a `schedule` property and a compact `__repr__`.

```
1 from codesign import EpochResult, ScheduleResult
2 e0 = EpochResult("glucose", "eco", 8.0, 0.0, True, {"cost": 8.0})
3 e1 = EpochResult("acetate", "bal", 6.0, 0.5, True, {"cost": 6.0})
4 sr = ScheduleResult([e0, e1], total_cost=14.5, feasible=True,
5                   n_switches=1)
6 print(sr.schedule)
7 print(sr)
8 # → ['eco', 'bal']
9 # → ScheduleResult(feasible, cost=14.5, switches=1, [eco → bal])
```

- `epochs` — `List[EpochResult]` — one entry per epoch, in order.
- `total_cost` — `float` — running plus switching cost summed over epochs.
- `feasible` — `bool` — `True` iff every epoch was satisfiable by its chosen architecture.
- `n_switches` — `int` — number of epoch boundaries at which the active architecture changed.
- `schedule` (property) — `List[str]` — the chosen architecture name per epoch, in order.

Errors: none raised directly.

solve_schedule Chooses an architecture per epoch minimising running plus switching cost, by an exact Viterbi pass over the epoch/architecture lattice. With `switch_cost == 0` and `hysteresis == 0` the result reduces to the epoch-local greedy choice.

```
1 solve_schedule(epochs, architectures=None, *, cost_fn,
2               switch_cost=0.0, hysteresis=0.0, max_iter=200,
3               solve_kwargs=None) → ScheduleResult
```

```
1 from codesign import (AlgebraicDP, Architecture, Epoch, Ports,
2                       Reals, System, solve_schedule)
3 def mk(name, c):
4     s = System(name); d = s.provides("demand"); s.requires("cost")
5     s.add("m", AlgebraicDP(F=Ports({"demand": Reals()}),
6                             R=Ports({"cost": Reals()}),
7                             equations={"cost": lambda f, c=c: c})).demand ≥ d
8     s.constrain("cost", lambda x: x["m.cost"]); return s.build()
```



```

9 lo = Architecture("lo", mk("lo", 5.0))
10 hi = Architecture("hi", mk("hi", 3.0))
11 eps = [Epoch("e1", {"demand": 1.0}, candidates=[lo]),
12        Epoch("e2", {"demand": 1.0}, candidates=[hi])]
13 r = solve_schedule(eps, cost_fn=lambda p: p["cost"], switch_cost=1.0)
14 print(r.schedule, r.total_cost, r.n_switches) # → ['lo', 'hi'] 9.0 1

```

- **epochs** — `Sequence[Epoch]` — the ordered environment regimes.
- **architectures** — `Sequence[Architecture]` or `None` — default candidate set for any epoch that does not supply its own; required if any epoch leaves **candidates** as `None`.
- **cost_fn** — callable — maps a resource point to a scalar; lower is better.
- **switch_cost** — float or callable — cost charged when the active architecture changes; a float applies uniformly, a callable `f(prev, next) -> float` makes it transition-dependent.
- **hysteresis** — float — extra margin a challenger must beat the incumbent by before a switch is preferred; suppresses chattering between near-equal options.
- **max_iter** — int — forwarded to `solve()` for each epoch solve.
- **solve_kwargs** — Mapping or `None` — extra keyword arguments forwarded to `solve()` (for example `uncertainty`).

Errors: raises `ValueError` when an epoch supplies no **candidates** and no default **architectures** set was passed; the message reads

```

epoch <name> has no candidates and no default architecture set
was supplied. Either set candidates=[...] on the epoch, or pass
architectures=[...] to solve_schedule().

```

An infeasible epoch is not an error: the schedule is returned with `feasible=False` and the offending running cost recorded as `inf` for inspection.

13.5.2 dynamic

The `dynamic` module implements Case 2 with a *scalar* carried state (section 15): a finite-horizon dynamic program whose per-stage decision is which architecture to instantiate, whose per-stage cost is itself a co-design `solve()`, and whose carried scalar state (fuel, charge, cumulative wear) is advanced by a user **transition**. The value function is scalar — at each (stage, state) the single best cost-to-go — found by a backward Bellman recursion. State is discretised onto an explicit `StateGrid`; a transition that would leave the grid envelope is rejected *before* snapping, so an over-spent resource is never silently rescued by the nearest in-range node. It reuses the `Architecture` of the `temporal` module (13.5.1).

Stage One step of the finite-horizon dynamic program: everything needed to score and advance every candidate architecture at this point in time given the carried state. A dataclass.

```

1 Stage(name: str,
2        functionality: Callable[[float], Mapping],
3        transition: Callable[[float, Mapping], float],
4        candidates: Optional[Sequence[Architecture]] = None,
5        admissible: Optional[Callable[[float], bool]] = None)

1 from codesign import Architecture, Stage
2 mode = Architecture("burn", None)
3 st = Stage("leg1", functionality=lambda s: {"demand": 1.0},
4           transition=lambda s, p: s - p["fuel"],
5           candidates=[mode], admissible=lambda s: s ≥ 0.0)
6 print(st.name, st.transition(10.0, {"fuel": 3.0})) # → leg1 7.0

```

- **name** — `str` — stage identifier.

- **functionality** — callable — `functionality(state) -> Mapping`, the outer functionality demanded when the carried resource has value `state`. A state-independent stage ignores its argument.
- **transition** — callable — `transition(state, point) -> new_state`, mapping the incoming state and the chosen resource point to the outgoing state; the returned value is snapped to the nearest grid node.
- **candidates** — `Sequence[Architecture]` or `None` — architectures admissible here; falls back to the schedule-level default when `None`.
- **admissible** — callable or `None` — `admissible(state) -> bool`; states returning `False` are forbidden (infinite value).

Errors: none raised directly.

StageResult A per-stage record along a rolled-out optimal policy. A dataclass with no public methods; constructed by `rollout`.

```
1 from codesign import StageResult
2 sr = StageResult(stage="leg1", architecture="burn", state_in=10.0,
3                 state_out=7.0, stage_cost=4.0, feasible=True,
4                 point={"fuel": 3.0})
5 print(sr.stage, sr.state_in, sr.state_out, sr.stage_cost)
6 # → leg1 10.0 7.0 4.0
```

- **stage** — `str` — the stage’s name.
- **architecture** — `str` — chosen architecture name.
- **state_in** — `float` — snapped carried state entering the stage.
- **state_out** — `float` — snapped carried state leaving the stage.
- **stage_cost** — `float` — co-design cost incurred at this stage.
- **feasible** — `bool` — whether a finite-cost action existed here.
- **point** — `Mapping` or `None` — the chosen resource point.
- **tags** — `Mapping` — the chosen architecture’s tags.

Errors: none raised directly.

StateGrid A one-dimensional discretisation of the carried scalar resource. Tabular dynamic programming needs a finite state set, so a continuous resource is bucketed onto explicit, sorted nodes; transitions landing between nodes are snapped to the nearest. Keeping the grid an explicit object makes the discretisation visible and lets the caller trade accuracy against cost.

```
1 StateGrid(nodes: Sequence[float])

1 from codesign import StateGrid
2 g = StateGrid.linspace(0.0, 10.0, 6) # nodes 0,2,4,6,8,10
3 print(list(g), len(g), g.snap(5.4))
4 # → [0.0, 2.0, 4.0, 6.0, 8.0, 10.0] 6 6.0
```

- **nodes** — `Sequence[float]` — the grid node values (stored sorted). Also iterable, with `len()` giving the node count.

Method	Signature	Behaviour
<code>linspace</code>	<code>linspace(lo, hi, n)</code> (classmethod)	evenly spaced grid of <code>n</code> nodes on <code>[lo, hi]</code> .
<code>snap</code>	<code>snap(value) -> float</code>	the grid node nearest value.

Errors: the constructor raises `ValueError` on an empty node sequence (“`StateGrid` needs at least one node, got an empty sequence. ...”); `linspace` raises `ValueError` when `n < 1` (“`StateGrid.linspace` needs at least one node, got `n=<n>`. Pass `n >= 1` ...”). With `n == 1` a single node at `lo` is returned.

DynamicPolicy A solved, state-indexed policy: the best architecture per (stage, state), produced by `solve_dynamic`. Beyond the single rolled-out path the full table supports closed-loop control, re-queried at the actual realised state when it diverges from the nominal roll-out.

```

1 from codesign import (AlgebraicDP, Architecture, Ports, Reals, Stage,
2                       StateGrid, System, solve_dynamic)
3 s = System("m"); d = s.provides("demand"); s.requires("fuel")
4 s.add("m", AlgebraicDP(F=Ports({"demand": Reals()}),
5                       R=Ports({"fuel": Reals()}),
6                       equations={"fuel": lambda f: 2.0})).demand ≥ d
7 s.constrain("fuel", lambda x: x["m.fuel"])
8 burn = Architecture("burn", s.build())
9 st = Stage("leg", lambda s: {"demand": 1.0},
10          lambda s, p: s - p["fuel"], candidates=[burn])
11 policy = solve_dynamic([st], StateGrid.linspace(0.0, 10.0, 11),
12                      cost_fn=lambda p: p["fuel"])
13 print(policy.cost_to_go(0, 8.0), policy.action_at(0, 8.0))
14 # → 2.0 ('burn', {'fuel': 2.0}, 2.0, 6.0)

```

Method	Behaviour
<code>cost_to_go(t, state)</code>	optimal remaining cost from stage <code>t</code> at the grid node nearest state (<code>inf</code> if none).
<code>action_at(t, state)</code>	best (arch, point, stage_cost, state_out) at (t, state); None if no finite-cost action exists.

Errors: none raised directly (an out-of-table state returns `inf` / `None` rather than raising).

DynamicResult The outcome of a policy rolled out from an initial state, returned by `rollout` and `solve_and_rollout`. A dataclass with a `schedule` property and a compact `__repr__`; it also carries the full policy for closed-loop re-querying.

```

1 from codesign import (AlgebraicDP, Architecture, Ports, Reals, Stage,
2                       StateGrid, System, solve_and_rollout)
3 s = System("m"); d = s.provides("demand"); s.requires("fuel")
4 s.add("m", AlgebraicDP(F=Ports({"demand": Reals()}),
5                       R=Ports({"fuel": Reals()}),
6                       equations={"fuel": lambda f: 2.0})).demand ≥ d
7 s.constrain("fuel", lambda x: x["m.fuel"])
8 burn = Architecture("burn", s.build())
9 st = Stage("leg", lambda s: {"demand": 1.0},
10          lambda s, p: s - p["fuel"], candidates=[burn])
11 grid = StateGrid.linspace(0.0, 10.0, 11)
12 res = solve_and_rollout([st, st], grid, 10.0, cost_fn=lambda p: p["fuel"])
13 print(res.schedule, res.total_cost, res.feasible)
14 # → ['burn', 'burn'] 4.0 True

```

- `stages` — `List[StageResult]` — per-stage record of the optimal policy from the initial state.
- `total_cost` — `float` — optimal cost-to-go from the initial state (running plus terminal).
- `feasible` — `bool` — `True` iff a finite-cost policy exists from the initial state.
- `policy` — `DynamicPolicy` — the full state-indexed policy table.
- `schedule` (property) — `List[str]` — architecture chosen at each stage along the rolled-out path.

Errors: none raised directly.

solve_dynamic Solves the finite-horizon architecture DP by a backward Bellman pass over the (stage, state) lattice, returning a full state-indexed `DynamicPolicy` (13.5.2). At each (stage,

state) it solves the co-design problem for each admissible architecture at the stage's state-dependent functionality, reads the cheapest feasible point, advances the state, and adds the discretised cost-to-go of the successor.

```
1 solve_dynamic(stages, grid, *, cost_fn, architectures=None,
2               terminal_cost=None, max_iter=200, solve_kwargs=None,
3               cache=True) → DynamicPolicy
```

```
1 from codesign import (AlgebraicDP, Architecture, Ports, Reals, Stage,
2                       StateGrid, System, solve_dynamic)
3 s = System("m"); d = s.provides("demand"); s.requires("fuel")
4 s.add("m", AlgebraicDP(F=Ports({"demand": Reals()}),
5                       R=Ports({"fuel": Reals()}),
6                       equations={"fuel": lambda f: 2.0})).demand ≥ d
7 s.constrain("fuel", lambda x: x["m.fuel"])
8 burn = Architecture("burn", s.build())
9 st = Stage("leg", lambda s: {"demand": 1.0},
10          lambda s, p: s - p["fuel"], candidates=[burn])
11 grid = StateGrid.linspace(0.0, 10.0, 11)
12 policy = solve_dynamic([st, st], grid, cost_fn=lambda p: p["fuel"])
13 print(policy.stage_names, policy.cost_to_go(0, 10.0))
14 # → ['leg', 'leg'] 4.0
```

- `stages` — `Sequence[Stage]` — the horizon in forward order (solved backward internally).
- `grid` — `StateGrid` — discretisation of the carried scalar resource.
- `cost_fn` — callable — scalar cost on a resource point; lower is better.
- `architectures` — `Sequence[Architecture]` or `None` — default candidates for stages without their own.
- `terminal_cost` — callable or `None` — `terminal_cost(state) -> float` scoring the state after the final stage; defaults to zero everywhere.
- `max_iter` — `int` — forwarded to `solve()`.
- `solve_kwargs` — `Mapping` or `None` — extra keyword arguments forwarded to `solve()`.
- `cache` — `bool` — cache co-design solves keyed by (architecture name, functionality); safe when `functionality(state)` is deterministic.

Errors: raises `ValueError` when a stage supplies no candidates and no default architectures set was passed; the message reads

```
stage <name> has no candidates and no default architecture set
was supplied. Either set candidates=[...] on the stage, or pass
architectures=[...] to solve_dynamic().
```

rollout Rolls a solved policy forward from a concrete initial state, following the policy's chosen architecture at each stage, advancing the snapped carried state, and assembling a `DynamicResult` (13.5.2).

```
1 rollout(policy, stages, initial_state, *, arch_lookup=None)
2       → DynamicResult
```

```
1 from codesign import (AlgebraicDP, Architecture, Ports, Reals, Stage,
2                       StateGrid, System, rollout, solve_dynamic)
3 s = System("m"); d = s.provides("demand"); s.requires("fuel")
4 s.add("m", AlgebraicDP(F=Ports({"demand": Reals()}),
5                       R=Ports({"fuel": Reals()}),
6                       equations={"fuel": lambda f: 2.0})).demand ≥ d
7 s.constrain("fuel", lambda x: x["m.fuel"])
8 burn = Architecture("burn", s.build())
9 st = Stage("leg", lambda s: {"demand": 1.0},
10          lambda s, p: s - p["fuel"], candidates=[burn])
```

```

11 grid = StateGrid.linspace(0.0, 10.0, 11)
12 policy = solve_dynamic([st, st], grid, cost_fn=lambda p: p["fuel"])
13 res = rollout(policy, [st, st], 10.0)
14 print(res.schedule, res.total_cost)    # → ['burn', 'burn'] 4.0

```

- `policy` — `DynamicPolicy` — a policy from `solve_dynamic`.
- `stages` — `Sequence[Stage]` — the same stage list passed to `solve_dynamic`.
- `initial_state` — `float` — the starting carried value.
- `arch_lookup` — `Mapping` or `None` — maps architecture name to `Architecture` so tags attach to each record; built automatically from the stage candidate lists when omitted.

Errors: none raised directly. When the policy has no feasible action from the reached state the roll-out stops early, the offending record is marked `feasible=False`, and `DynamicResult.feasible` becomes `False`.

solve_and_rollout A convenience wrapper equivalent to `solve_dynamic` followed by `rollout` from `initial_state`. Prefer the two-step form when the policy will be queried at several initial states or in closed loop.

```

1 solve_and_rollout(stages, grid, initial_state, *, cost_fn,
2                   architectures=None, terminal_cost=None, max_iter=200,
3                   solve_kwargs=None, cache=True) → DynamicResult

1 from codesign import (AlgebraicDP, Architecture, Ports, Reals, Stage,
2                       StateGrid, System, solve_and_rollout)
3 s = System("m"); d = s.provides("demand"); s.requires("fuel")
4 s.add("m", AlgebraicDP(F=Ports({"demand": Reals()}),
5                             R=Ports({"fuel": Reals()}),
6                             equations={"fuel": lambda f: 2.0})).demand ≥ d
7 s.constrain("fuel", lambda x: x["m.fuel"])
8 burn = Architecture("burn", s.build())
9 st = Stage("leg", lambda s: {"demand": 1.0},
10           lambda s, p: s - p["fuel"], candidates=[burn])
11 grid = StateGrid.linspace(0.0, 10.0, 11)
12 res = solve_and_rollout([st, st], grid, 10.0, cost_fn=lambda p: p["fuel"])
13 print(res.schedule, res.total_cost, res.feasible)
14 # → ['burn', 'burn'] 4.0 True

```

Parameters: as `solve_dynamic` (13.5.2), plus the positional `initial_state` (`float`) rolled out from. Errors: same `ValueError` as `solve_dynamic` when candidates are missing.

13.5.3 sequential

The **sequential** module is the *antichain-valued* generalisation of the scalar **dynamic** layer (section 15.3): a finite-horizon decision process whose stages are co-design problems coupled by a carried state, solved by a backward Bellman recursion whose value at each (stage, state) is a Pareto antichain rather than a scalar. The accumulated resource (the named cost axes on the antichain) is kept deliberately distinct from the carried state: the transition reads any carried quantity off the full solved point, while only the cost axes accumulate on the front. The scalar **dynamic** layer is the width-one special case. This module also makes three theory results operational — monotone value (Q1, `check_monotonicity`), front = reachable frontier (Q2), and exact factorisation at a reset (Q3, `detect_resets` / `factorise_at_resets`) — and exposes the precompute-then-DP structure (section 15.5). It reuses the **Architecture** (13.5.1) and **StateGrid** (13.5.2).

SeqStage One stage of a sequential co-design problem. Structurally the antichain-valued analogue of **Stage** (13.5.2): returning the grid's bottom node from `transition` makes the stage a reset. A dataclass.

```

1 SeqStage(name: str,
2           functionality: Callable[[float], Mapping],
3           transition: Callable[[float, Mapping], float],
4           candidates: Optional[Sequence[Architecture]] = None,
5           admissible: Optional[Callable[[float], bool]] = None)

1 from codesign import Architecture, SeqStage
2 mode = Architecture("m", None)
3 st = SeqStage("leg", functionality=lambda x: {"demand": 1.0},
4             transition=lambda x, p: x - p["fuel"],
5             candidates=[mode], admissible=lambda x: x ≥ 0.0)
6 print(st.name, st.transition(6.0, {"fuel": 2.0})) # → leg 4.0

```

- **name** — **str** — stage identifier.
- **functionality** — **callable** — **functionality(state)** -> **Mapping**, the outer functionality demanded when the carried state is **state**.
- **transition** — **callable** — **transition(state, point)** -> **new_state**, from the incoming state and a chosen resource point (over the resource ports) to the outgoing state.
- **candidates** — **Sequence[Architecture]** or **None** — architectures admissible here; each one's co-design solve supplies part of the stage antichain $h_k(x)$ (the union of their fronts). Falls back to the problem-level default when **None**.
- **admissible** — **callable** or **None** — **admissible(state)** -> **bool**; forbidden states carry the empty (infeasible) value.

Errors: none raised directly.

sum_combine The consumable/accumulating monoid combination $\oplus$: component-wise sum over the cost axes. Resources accumulate across stages (fuel burned, money spent, wear incurred); the reachable frontier can grow polynomially with the horizon. The default **combine** for **solve_sequential**.

```

1 from codesign import sum_combine
2 a = {"cost": 5.0, "co2": 1.0}
3 b = {"cost": 2.0, "co2": 8.0}
4 print(sum_combine(a, b)) # → {'cost': 7.0, 'co2': 9.0}

```

- **a, b** — **Mapping** — two resource points over the same axes; returns a **dict** of their component-wise sum.

Errors: none raised directly (a **KeyError** results if **b** lacks an axis present in **a**).

join_combine The renewable monoid combination $\oplus$: component-wise maximum (join). The requirement of two stages is the peak, not the sum (a worker, an oven, a bus reused across stages); the reachable frontier stays bounded uniformly in the horizon, and if every stage resets the problem collapses to a static co-design.

```

1 from codesign import join_combine
2 a = {"cost": 5.0, "co2": 1.0}
3 b = {"cost": 2.0, "co2": 8.0}
4 print(join_combine(a, b)) # → {'cost': 5.0, 'co2': 8.0}

```

- **a, b** — **Mapping** — two resource points over the same axes; returns a **dict** of their component-wise maximum.

Errors: none raised directly (a **KeyError** results if **b** lacks an axis present in **a**).

solve_sequential Solves the antichain-valued sequential co-design problem by the backward Bellman recursion in the upper-set value space, carrying a full Pareto antichain of cumulative resources at each (stage, state). Returns a **SeqResult** (13.5.3) whose value is $V_0(\text{initial_state})$.

The snippet below uses a single architecture (a width-one front); with several incomparable candidates the value has width greater than one — see `dp_over_catalog` (13.5.3) for a multi-point front.

```

1 solve_sequential(stages, grid, *, cost_axes, initial_state,
2                   combine=sum_combine, architectures=None,
3                   max_iter=200, solve_kwargs=None) → SeqResult

1 from codesign import (AlgebraicDP, Architecture, Ports, Reals, SeqStage,
2                       StateGrid, System, solve_sequential)
3 s = System("m"); d = s.provides("demand"); s.requires("fuel")
4 s.add("m", AlgebraicDP(F=Ports({"demand": Reals()}),
5                          R=Ports({"fuel": Reals()}),
6                          equations={"fuel": lambda f: 2.0}).demand ≥ d
7 s.constrain("fuel", lambda x: x["m.fuel"])
8 a = Architecture("a", s.build())
9 st = SeqStage("leg", lambda x: {"demand": 1.0},
10              lambda x, p: x - p["fuel"], candidates=[a],
11              admissible=lambda x: x ≥ -1e-9)
12 res = solve_sequential([st, st], StateGrid.linspace(0., 6., 13),
13                        cost_axes=["fuel"], initial_state=6.)
14 print(res.width, sorted(p["fuel"] for p in res.value), res.feasible)
15 # → 1 [4.0] True

```

- `stages` — `Sequence[SeqStage]` — the horizon in forward order (solved backward internally).
- `grid` — `StateGrid` — discretisation of the carried scalar state.
- `cost_axes` — `Sequence[str]` — names of the resource ports to accumulate on the antichain (for example `["cost", "co2"]`); these define the product poset R .
- `initial_state` — `float` — starting carried state; the returned value is V_0 of it.
- `combine` — callable — the monoid combination $\oplus$: `sum_combine` (default) for a consumable resource, `join_combine` for a renewable one.
- `architectures` — `Sequence[Architecture]` or `None` — default candidates for stages without their own.
- `max_iter` — `int` — forwarded to `solve()`.
- `solve_kwargs` — `Mapping` or `None` — extra keyword arguments forwarded to `solve()`.

Errors: raises `ValueError` when a stage supplies no candidates and no default architectures set was passed; the message reads

```

stage <name> has no candidates and no default architecture set
was supplied. Either set candidates=[...] on the stage, or pass
architectures=[...] to the sequential solver (solve_sequential /
check_monotonicity).

```

SeqResult The outcome of `solve_sequential` (and `dp_over_catalog`): the whole-horizon Pareto front from the initial state. A dataclass with a compact `__repr__`. The snippet obtains one from `dp_over_catalog` to show a genuine multi-point front.

```

1 from codesign import StateGrid, dp_over_catalog
2 cat = [{"clean", {"cost": 10.0, "co2": 1.0}},
3        {"cheap", {"cost": 2.0, "co2": 8.0}}]
4 res = dp_over_catalog([cat, cat], StateGrid.linspace(0., 4., 5),
5                       cost_axes=["cost", "co2"], initial_state=4.0,
6                       transition=lambda s, p: s)
7 print(res.width, res.feasible,
8       sorted((p["cost"], p["co2"]) for p in res.value))
9 # → 3 True [(4.0, 16.0), (12.0, 9.0), (20.0, 2.0)]

```


- **value** — **Antichain** — the value antichain $\text{Min } V_0(\text{initial_state})$; each point is a Mapping over the cost axes.
- **width** — **int** — $\text{len}(\text{value})$, the number of incomparable Pareto-optimal totals (the reachable-frontier width).
- **feasible** — **bool** — **True** iff the value antichain is non-empty and free of tops.
- **policy** — **SeqPolicy** — the full state-indexed antichain-valued policy table.

Errors: none raised directly.

SeqPolicy The state-indexed antichain-valued policy from the backward pass, held on **SeqResult**. **policy**. For each (stage, state node) it stores the value antichain $V_k(x)$ and, per Pareto point, the realising architecture and chosen stage point, so an optimal choice sequence can be traced forward.

```

1 from codesign import StateGrid, dp_over_catalog
2 cat = [{"clean", {"cost": 10.0, "co2": 1.0}},
3        {"cheap", {"cost": 2.0, "co2": 8.0}}]
4 res = dp_over_catalog([cat, cat], StateGrid.linspace(0., 4., 5),
5                       cost_axes=["cost", "co2"], initial_state=4.0,
6                       transition=lambda s, p: s)
7 pol = res.policy
8 print(pol.width_at(0, 4.0), pol.stage_names)
9 # → 3 ['stage_0', 'stage_1']

```

Method	Behaviour
<code>value_at(k, state)</code>	the value antichain $V_k(x)$ at the grid node nearest state (empty antichain if absent).
<code>width_at(k, state)</code>	the reachable-frontier width $\alpha_k(x)$, i.e. $\text{len}(\text{value_at}(k, \text{state}))$.

Errors: none raised directly.

MonotonicityReport The result of `check_monotonicity`: whether the (H1)/(H2) hypotheses held on the grid, with sampled witnesses when they did not. A dataclass whose `monotone_value_guaranteed` property is **True** exactly when both hold (the Q1 theorem then guarantees a monotone value).

```

1 from codesign import MonotonicityReport
2 rep = MonotonicityReport(h1_ok=True, h2_ok=True)
3 print(rep.monotone_value_guaranteed, repr(rep))
4 # → True MonotonicityReport(H1=ok, H2=ok, value_monotone=guaranteed)

```

- **h1_ok** — **bool** — (H1) held at every tested grid pair.
- **h2_ok** — **bool** — (H2) held at every tested grid pair.
- **h1_violations, h2_violations** — **List[Tuple[str, float, float]]** — sampled witnesses (**stage_name**, **x**, **x'**) where the condition failed (empty when it held).
- **monotone_value_guaranteed** (property) — **h1_ok** and **h2_ok**.

Errors: none raised directly.

check_monotonicity Numerically verifies (H1) and (H2) on the state grid and returns a **MonotonicityReport** (13.5.3). It is orientation-aware: it accepts the benign “consumable but monotone” orientation (a budget carried as remaining slack) and flags only genuinely non-monotone stages (perishable / fatigue-as-state, with an interior optimum), which fall outside the Q1 guarantee. Spurious violations most often signal a too-coarse grid, since snapping is not order-preserving at bucket boundaries.


```

1 check_monotonicity(stages, grid, *, cost_axes, architectures=None,
2                     max_iter=200, solve_kwargs=None,
3                     max_violations=8) → MonotonicityReport

```

```

1 from codesign import (AlgebraicDP, Architecture, Ports, Reals, SeqStage,
2                       StateGrid, System, check_monotonicity)
3 s = System("m"); d = s.provides("demand"); s.requires("fuel")
4 s.add("m", AlgebraicDP(F=Ports({"demand": Reals()}),
5                          R=Ports({"fuel": Reals()}),
6                          equations={"fuel": lambda f: 2.0})).demand ≥ d
7 s.constrain("fuel", lambda x: x["m.fuel"])
8 a = Architecture("a", s.build())
9 st = SeqStage("leg", lambda x: {"demand": 1.0},
10              lambda x, p: x - p["fuel"], candidates=[a],
11              admissible=lambda x: x ≥ -1e-9)
12 rep = check_monotonicity([st, st], StateGrid.linspace(0., 6., 13),
13                          cost_axes=["fuel"])
14 print(rep.h1_ok, rep.h2_ok, rep.monotone_value_guaranteed)
15 # → True True True

```

- `stages, grid, cost_axes, architectures, max_iter, solve_kwargs` — as `solve_sequential` (13.5.3).
- `max_violations` — `int` — cap on the number of sampled witnesses recorded per hypothesis.

Errors: raises the same `ValueError` as `solve_sequential` when a stage has no candidates and no default set (the message names both “`solve_sequential` / `check_monotonicity`”).

detect_resets Returns the indices of stages that are resets: a stage is a reset iff its transition lands on the grid’s bottom node for every incoming state and every chosen resource point. At a reset the horizon factorises exactly (Q3), in every regime, since the proof uses only distributivity.

```

1 detect_resets(stages, grid, *, cost_axes, architectures=None,
2               max_iter=200, solve_kwargs=None) → List[int]

```

```

1 from codesign import (AlgebraicDP, Architecture, Ports, Reals, SeqStage,
2                       StateGrid, System, detect_resets)
3 s = System("m"); d = s.provides("demand"); s.requires("fuel")
4 s.add("m", AlgebraicDP(F=Ports({"demand": Reals()}),
5                          R=Ports({"fuel": Reals()}),
6                          equations={"fuel": lambda f: 2.0})).demand ≥ d
7 s.constrain("fuel", lambda x: x["m.fuel"])
8 a = Architecture("a", s.build())
9 run = SeqStage("run", lambda x: {"demand": 1.0},
10              lambda x, p: x - p["fuel"], candidates=[a])
11 reset = SeqStage("reset", lambda x: {"demand": 1.0},
12                 lambda x, p: 0.0, candidates=[a])
13 print(detect_resets([run, reset, run],
14                     StateGrid.linspace(0., 6., 13), cost_axes=["fuel"]))
15 # → [1]

```

- `stages, grid, cost_axes, architectures, max_iter, solve_kwargs` — as `solve_sequential` (13.5.3).

Errors: raises `ValueError` (naming the stage) when a stage has no candidates and no default architectures was supplied — the same guard as `solve_sequential` and `check_monotonicity`, so an under-specified stage fails loudly instead of being silently (and vacuously) reported as a reset.

factorise_at_resets Partitions the horizon into maximal quiescence-free runs given the reset indices, returning inclusive (`start`, `end`) ranges. By Q3 the value over the full horizon is the $\oplus$ -product of the values of these independent sub-problems — the order-theoretic, deterministic analogue of regeneration-point decomposition. Pure bookkeeping: it solves nothing.

```
1 factorise_at_resets(stages, resets) → List[Tuple[int, int]]
```

```
1 from codesign import SeqStage, factorise_at_resets
2 f = lambda x: {"demand": 1.0}
3 t = lambda x, p: x
4 stages = [SeqStage(f"s{i}", f, t) for i in range(4)]
5 print(factorise_at_resets(stages, resets=[1]))
6 # → [(0, 1), (2, 3)]
```

- `stages` — `Sequence[SeqStage]` — the horizon (only its length is used).
- `resets` — `Sequence[int]` — reset stage indices, typically from `detect_resets` (13.5.3). A reset closes the run containing it.

Errors: none raised directly.

precompute_catalog Solves every architecture once at a fixed functionality and returns a flat, tagged Pareto catalog — the co-design precomputation step of the Formula 1 seasonal framework (section 15.5). Points that look dominated at this single-stage level are retained if non-dominated in the combined antichain, since they may become optimal once aggregated in an outer DP.

```
1 precompute_catalog(architectures, functionality, cost_axes, *,
2                     max_iter=200, solve_kwargs=None)
3     → List[Tuple[str, Mapping]]
```

```
1 from codesign import (AlgebraicDP, Architecture, Ports, Reals,
2                       System, precompute_catalog)
3 s = System("m"); d = s.provides("demand"); s.requires("fuel")
4 s.add("m", AlgebraicDP(F=Ports({"demand": Reals()}),
5                             R=Ports({"fuel": Reals()}),
6                             equations={"fuel": lambda f: 2.0})).demand ≥ d
7 s.constrain("fuel", lambda x: x["m.fuel"])
8 a = Architecture("a", s.build())
9 cat = precompute_catalog([a], {"demand": 1.0}, cost_axes=["fuel"])
10 print(cat) # → [('a', {'fuel': 2.0})]
```

- `architectures` — `Sequence[Architecture]` — the candidates to solve once.
- `functionality` — `Mapping` — the fixed outer functionality at which each architecture is solved.
- `cost_axes` — `Sequence[str]` — resource ports accumulated on the catalog's antichain.
- `max_iter`, `solve_kwargs` — forwarded to `solve()`.

Returns a list of (`arch_name`, `point`) pairs (each point the full solved resource point, so a carried-state axis remains readable); feed it to `dp_over_catalog` (13.5.3). Errors: none raised directly.

dp_over_catalog Runs the antichain-valued DP that *selects from precomputed catalogs*: each stage's action set is a frozen catalog rather than a live co-design solve, so no `solve()` happens inside the Bellman sweep. Use it when the per-stage co-design is independent of the carried state (the Formula 1 regime); otherwise use `solve_sequential`. Returns a `SeqResult` (13.5.3).

```
1 dp_over_catalog(catalogs, grid, *, cost_axes, initial_state,
2                 transition, combine=sum_combine,
3                 admissible=None) → SeqResult
```

```

1 from codesign import StateGrid, dp_over_catalog, sum_combine
2 cat = [{"clean", {"cost": 10.0, "co2": 1.0}},
3        {"cheap", {"cost": 2.0, "co2": 8.0}}]
4 res = dp_over_catalog([cat, cat], StateGrid.linspace(0., 4., 5),
5                       cost_axes=["cost", "co2"], initial_state=4.0,
6                       transition=lambda s, p: s, combine=sum_combine)
7 print(res.width, sorted((p["cost"], p["co2"]) for p in res.value))
8 # → 3 [(4.0, 16.0), (12.0, 9.0), (20.0, 2.0)]

```

- **catalogs** — `Sequence[Sequence[Tuple[str, Mapping]]]` — one catalog per stage, each a sequence of (arch_name, point) pairs from `precompute_catalog`.
- **grid** — `StateGrid` — discretisation of the carried scalar state.
- **cost_axes** — `Sequence[str]` — resource ports accumulated on the antichain.
- **initial_state** — `float` — starting carried state.
- **transition** — callable — `transition(state, point) -> new_state`.
- **combine** — callable — monoid combination $\oplus$ (default `sum_combine`).
- **admissible** — callable or `None` — state admissibility predicate.

Errors: none raised directly.

13.6 State and closed loop

The vector-state generalisation (`state`, `vector_dp`) and the closed-loop `online_codesign` controller. See Sections 15.4 and 15.6.

13.6.1 state

The `state` module supplies the general carried state of the temporal layer (section 15.4): a hashable state-vector value type and a product grid of named axes carrying the component-wise order the monotonicity results are stated over. A one-axis grid reproduces the scalar `StateGrid` exactly, so the vector layer strictly generalises the scalar carried state rather than paralleling it.

StateVec The hashable value type of a carried state vector: a tuple of (`axis_name`, `value`) pairs sorted by name, so two states with the same contents compare and hash equal. It is a type alias, not a class — `StateVec = Tuple[Tuple[str, Any], ...]` — and is used directly as the key of the dynamic-program table. Build one with `make_state` and read it with `state_get` / `state_as_dict` rather than by indexing the raw tuple.

```

1 from codesign import make_state
2 sv = make_state(fuel=12.0, flag=0) # build a StateVec
3 print(sv)                        # (('flag', 0), ('fuel', 12.0)): sorted, hashable
4 table = {sv: 0.0} # usable directly as a DP-table key
5 print(table[make_state(flag=0, fuel=12.0)]) # 0.0: order-independent

```

- **Contents** — one (`str`, `Any`) pair per axis; canonicalised (sorted by name) so keyword order at construction is irrelevant.

Errors: none raised directly (it is a plain tuple).

make_state Build a canonical `StateVec` from keyword axis values.

```

1 make_state(**axis_values) → StateVec

```

```

1 from codesign import make_state
2 x = make_state(fuel=12.0, flag=0)
3 print(x)                        # (('flag', 0), ('fuel', 12.0))
4 print(isinstance(x, tuple))    # True: hashable, usable as a DP-table key

```

- ****axis_values** — each keyword names an axis and gives its value; the result is normalised so equal contents hash and compare equal.

Errors: none raised directly.

state_get Read one axis value from a state vector. Signature: `state_get(state: StateVec, axis: str) -> Any`.

```
1 from codesign import make_state, state_get
2 x = make_state(fuel=12.0, flag=0)
3 print(state_get(x, "fuel")) # 12.0
```

- **state** — the state vector to read.
- **axis** — the axis name whose value is wanted.

Errors: raises `KeyError` when the axis is absent, with the message "state vector has no axis 'charge'; it carries axes ['flag', 'fuel']" — the message names the missing axis and lists the axes the vector actually carries.

state_as_dict Return a plain dict view of a state vector, convenient inside a transition or functionality callback. Signature: `state_as_dict(state: StateVec) -> Dict[str, Any]`.

```
1 from codesign import make_state, state_as_dict
2 x = make_state(fuel=12.0, flag=0)
3 print(state_as_dict(x)) # {'flag': 0, 'fuel': 12.0}
```

- **state** — the state vector to expand.

Errors: none raised directly.

Axis Abstract base class defining the single-axis contract that every concrete axis satisfies and that `VectorStateGrid` composes into the product order. An axis carries a **name** and the four operations below; the two concrete axes (`ContinuousAxis`, `DiscreteAxis`) are documented as deltas from this contract. The snippet exercises the contract through a concrete axis, since `Axis` itself is not instantiable.

```
1 from codesign import ContinuousAxis, Axis
2 ax = ContinuousAxis("charge", 0.0, 1.0, 5) # a concrete Axis
3 print(isinstance(ax, Axis)) # True
4 print(list(ax.nodes())) # [0.0, 0.25, 0.5, 0.75, 1.0]
5 print(ax.snap(0.3)) # 0.25 (nearest node)
6 print(ax.in_bounds(1.2)) # False
7 print(ax.leq(0.25, 0.5)) # True
```

Method	Contract
<code>nodes()</code>	the axis's discretisation, as a sequence
<code>snap(value)</code>	nearest admissible node to value
<code>in_bounds(value)</code>	whether value lies within the axis
<code>leq(a, b)</code>	the axis order: a at or below b

Errors: the four base methods raise `NotImplementedError`; use a concrete axis.

ContinuousAxis A bucketed real interval, the vector analogue of the scalar grid and a concrete `Axis` (13.6.1). It places **n** evenly spaced nodes on `[lo, hi]`; `snap` returns the nearest node, `in_bounds` tests `[lo, hi]` within a tolerance, and `leq` applies the interval order, reversed when `increasing_is_larger` is `False`. Signature: `ContinuousAxis(name, lo, hi, n, increasing_is_larger=True)`.

```

1 from codesign import ContinuousAxis
2 ax = ContinuousAxis("slack", 0.0, 10.0, 3, increasing_is_larger=False)
3 print(list(ax.nodes())) # [0.0, 5.0, 10.0]
4 print(ax.leq(10.0, 0.0)) # True: reversed, so less slack ranks higher

```

Parameter	Type	Meaning
name	str	axis name (e.g. "charge", "wear1")
lo, hi	float	interval bounds
n	int	number of evenly spaced nodes on [lo, hi]
increasing_is_larger	bool	True: larger value is higher in the order; False reverses it

Errors: raises `ValueError` when `n < 1` (a message stating the axis needs at least one node and that `n=1` yields a single node at `lo`).

DiscreteAxis A finite labelled axis with a caller-supplied order, a concrete `Axis` (13.6.1) for quantities like a regulatory flag or an operating regime. Its `nodes` are the admissible labels; `snap` is the identity on admissible labels (labels are used verbatim); and `leq` follows `order` (least first), treating labels omitted from `order`, or all labels when `order` is `None`, as mutually incomparable — which the monotonicity guard reads as “no order to exploit on this axis”. Signature: `DiscreteAxis(name, values, order=None)`.

```

1 from codesign import DiscreteAxis
2 flag = DiscreteAxis("penalty", values=[0, 1], order=[0, 1])
3 print(flag.nodes()) # [0, 1]
4 print(flag.leq(0, 1)) # True: 0 ranks below 1
5 print(flag.snap(1)) # 1 (snap is identity on admissible labels)
6 free = DiscreteAxis("regime", values=["a", "b"]) # no order given
7 print(free.leq("a", "b")) # False: distinct labels are incomparable

```

Parameter	Type	Meaning
name	str	axis name
values	sequence	admissible labels (any hashable values)
order	sequence, optional	a chain giving the order, least first; omitted labels are incomparable

Errors: raises `ValueError` when `values` is empty (a message asking for the admissible labels).

VectorStateGrid A product grid over several named axes, carrying a full state vector. It enumerates the Cartesian product of its axes’ nodes as the DP state set, snaps and bounds-checks a proposed successor axis by axis, and exposes the component-wise product order used by the monotonicity results: $a \leq b$ iff a is at or below b on every axis. A single-axis grid reproduces the scalar `StateGrid`. Signature: `VectorStateGrid(axes: Sequence[Axis])`.

```

1 from codesign import VectorStateGrid, ContinuousAxis, DiscreteAxis
2 grid = VectorStateGrid([
3     ContinuousAxis("charge", 0.0, 1.0, 3),
4     DiscreteAxis("flag", values=[0, 1], order=[0, 1]),
5 ])
6 print(len(grid)) # 6 state nodes (3 x 2)
7 print(grid.bottom()) # (('charge', 0.0), ('flag', 0))
8 s = grid.snap({"charge": 0.4, "flag": 1})
9 print(s) # (('charge', 0.5), ('flag', 1))

```

```

10 print(grid.in_bounds({"charge": 2.0, "flag": 0})) # False
11 print(grid.leq(grid.bottom(), s)) # True

```

- **axes** — one or more `Axis` objects with distinct names; their order fixes the product.

Method	Behaviour
<code>scalar(name, lo, hi, n, increasing_is_larger=True)</code>	classmethod: a one-axis continuous grid (the scalar case)
<code>nodes()</code>	iterate over every state vector in the product
<code>snap(vector)</code>	snap a <code>{axis: value}</code> mapping to the nearest node
<code>in_bounds(vector)</code>	whether every axis value is within its axis
<code>leq(a, b)</code>	the component-wise product order
<code>bottom()</code>	the least state vector (each axis at its order minimum)
<code>len(grid)</code>	number of nodes (product of axis sizes)

Errors: raises `ValueError` on an empty axis sequence, and on duplicate axis names with the message "VectorStateGrid axis names must be unique, but ['wear'] appear more than once in ['wear', 'wear']". Give each axis a distinct name."

13.6.2 vector_dp

The `vector_dp` module is the general carried-state sequential solver (section 15.4): the antichain-valued Bellman recursion of `solve_sequential` (13.5.3) carrying a full state vector on a `VectorStateGrid` rather than a single scalar. Its card structure mirrors `sequential` throughout, with the product order of the vector grid substituted for the scalar order.

VecStage One stage of a vector-state sequential co-design problem: a dataclass bundling the stage's state-dependent functionality, its carried-state transition, and (optionally) its candidate architectures and admissibility predicate. The vector-state analogue of `sequential`'s `SeqStage`.

```

1 from codesign import VecStage, state_get
2 stage = VecStage(
3     name="leg",
4     functionality=lambda s: {"demand": state_get(s, "wear")},
5     transition=lambda s, p: {"wear": dict(s)["wear"] + p["d_wear"]},
6 )
7 print(stage.name, stage.candidates, stage.admissible) # leg None None

```

Field	Type	Meaning
<code>name</code>	<code>str</code>	stage identifier
<code>functionality</code>	<code>callable</code>	<code>f(state_vec) -> Mapping</code> : the outer functionality demanded, as a function of the incoming state
<code>transition</code>	<code>callable</code>	<code>f(state_vec, point) -> {axis: value}</code> : the outgoing state, snapped onto the grid
<code>candidates</code>	seq. of <code>Architecture</code> , optional	stage architectures; falls back to the solver default when <code>None</code>
<code>admissible</code>	<code>callable</code> , optional	<code>f(state_vec) -> bool</code> forbidding states returning <code>False</code>

Errors: none raised directly (a stage with no candidates and no solver-level default is reported by the solver; see below).

VecResult The outcome of `solve_vector_sequential`: a dataclass holding the value antichain at the initial state, its width, feasibility, and the full policy for roll-out.

```

1 from codesign import (AlgebraicDP, Architecture, Ports, Reals, VecStage,
2   VectorStateGrid, ContinuousAxis, solve_vector_sequential)
3 dp = AlgebraicDP(F=Ports({"demand": Reals()}),
4   R=Ports({"cost": Reals(), "d_wear": Reals()}),
5   equations={"cost": 1.0, "d_wear": 1.0})
6 stage = VecStage("leg", lambda s: {"demand": 1.0},
7   lambda s, p: {"wear": dict(s)["wear"] + p["d_wear"]},
8   candidates=[Architecture("run", dp)])
9 grid = VectorStateGrid([ContinuousAxis("wear", 0.0, 4.0, 5)])
10 res = solve_vector_sequential([stage], grid,
11   cost_axes=["cost"], initial_state={"wear": 0.0})
12 print(res.feasible, res.width) # True 1
13 print(res.value) # Antichain[(cost=1)]

```

Field	Type	Meaning
value	Antichain	the front of cumulative cost-axis totals at the initial state, $V_0(\text{initial})$
width	int	number of points on that front
feasible	bool	True iff value is non-empty and carries no $\top$
policy	VecPolicy	the full solved policy for roll-out

Errors: none raised directly.

VecPolicy A state-vector-indexed, antichain-valued policy. It is produced by `solve_vector_sequential` (read from `VecResult.policy`) rather than constructed directly, and it answers value and best-action queries at any `(stage, state)`, snapping the queried state onto the grid first.

```

1 from codesign import (AlgebraicDP, Architecture, Ports, Reals, VecStage,
2   VectorStateGrid, ContinuousAxis, solve_vector_sequential, make_state)
3 dp = AlgebraicDP(F=Ports({"demand": Reals()}),
4   R=Ports({"cost": Reals(), "d_wear": Reals()}),
5   equations={"cost": 1.0, "d_wear": 1.0})
6 stage = VecStage("leg", lambda s: {"demand": 1.0},
7   lambda s, p: {"wear": dict(s)["wear"] + p["d_wear"]},
8   candidates=[Architecture("run", dp)])
9 grid = VectorStateGrid([ContinuousAxis("wear", 0.0, 4.0, 5)])
10 pol = solve_vector_sequential([stage], grid,
11   cost_axes=["cost"], initial_state={"wear": 0.0}).policy
12 s0 = make_state(wear=0.0)
13 print(pol.width_at(0, s0)) # 1
14 print([p["cost"] for p in pol.value_at(0, s0)]) # [1.0]
15 print(pol.best_action_at(0, s0)) # ('run', {...}, (('wear', 1.0),))

```

`value_at(k, state)` the value antichain at stage `k` and (snapped) `state`; an empty antichain if the node is unreached.

`width_at(k, state)` the width (front size) of `value_at(k, state)`.

`best_action_at(k, state)` one realising `(arch, full_point, succ_state)` at `(k, state)`, picking the choice behind the cost axis's minimal-first point (a myopic roll-out); `None` when no finite choice exists.

Errors: none raised directly.

solve_vector_sequential Solve the vector-state antichain-valued sequential co-design problem. Identical in structure to `solve_sequential` (13.5.3) but carrying a full state vector on a `VectorStateGrid`; the transition returns a `{axis: value}` mapping snapped onto the product grid, and out-of-bounds successors are rejected before snapping on every axis.

```
1 solve_vector_sequential(stages, grid, *, cost_axes, initial_state,
2     combine=sum_combine, architectures=None, max_iter=200,
3     solve_kwargs=None)

1 from codesign import (AlgebraicDP, Architecture, Ports, Reals, VecStage,
2     VectorStateGrid, ContinuousAxis, solve_vector_sequential)
3 dp = AlgebraicDP(F=Ports({"demand": Reals()}),
4     R=Ports({"cost": Reals(), "d_wear": Reals()}),
5     equations={"cost": 1.0, "d_wear": 1.0})
6 stage = VecStage("leg", lambda s: {"demand": 1.0},
7     lambda s, p: {"wear": dict(s)["wear"] + p["d_wear"]},
8     candidates=[Architecture("run", dp)])
9 grid = VectorStateGrid([ContinuousAxis("wear", 0.0, 4.0, 5)])
10 res = solve_vector_sequential([stage, stage], grid,
11     cost_axes=["cost"], initial_state={"wear": 0.0})
12 print(res) # VecResult(feasible, width=1, ...)
13 print([p["cost"] for p in res.value]) # [2.0]: two legs, unit cost each
```

Parameter	Type	Meaning
<code>stages</code>	seq. of <code>VecStage</code>	horizon in forward order (solved backward)
<code>grid</code>	<code>VectorStateGrid</code>	product discretisation of the carried state
<code>cost_axes</code>	seq. of str	resource ports accumulated on the antichain (the poset R)
<code>initial_state</code>	mapping	<code>{axis: value}</code> start; the value returned is $V_0(\text{initial_state})$
<code>combine</code>	callable	the monoid (+): <code>sum_combine</code> (accumulating) or <code>join_combine</code> (renewable)
<code>architectures</code>	seq., optional	default candidates for stages without their own
<code>max_iter, solve_kwargs</code>		forwarded to <code>solve</code>

Errors: raises `ValueError` when a stage has neither `candidates` nor a solver-level `architectures` default (the message names the offending stage and both remedies).

VectorMonotonicityReport The result of `check_vector_monotonicity`: a dataclass recording whether (H1) and (H2) hold over the product order, with sample violations.

```
1 from codesign import VectorMonotonicityReport
2 rep = VectorMonotonicityReport(h1_ok=True, h2_ok=True)
3 print(rep.monotone_value_guaranteed) # True: H1 and H2 both hold
4 print(rep) # VectorMonotonicityReport(H1=ok, H2=ok, value_monotone=...)
```

Field	Type	Meaning
h1_ok	bool	(H1) holds: the stage map is monotone in the carried state
h2_ok	bool	(H2) holds: the transition is monotone in the carried state
h1_violations	list	up to <code>max_violations</code> sampled (<code>stage</code> , <code>x</code> , <code>x'</code>) witnesses
h2_violations	list	sampled (H2) witnesses

The property `monotone_value_guaranteed` is `h1_ok` and `h2_ok`. Errors: none raised directly.

check_vector_monotonicity Verify (H1) and (H2) over the vector grid's product order, the vector-state analogue of the scalar guard. For every ordered comparable pair $x \leq x'$ it checks the stage antichain is consistently oriented (a benign consumable-but-monotone axis passes; a genuinely non-monotone perishable one is flagged) and that the transition is monotone. The product grid can be large, so at most `max_pairs` comparable pairs per stage are sampled; the report is exact when the grid is small enough that the cap is not reached.

```

1 check_vector_monotonicity(stages, grid, *, cost_axes,
2     architectures=None, max_iter=200, solve_kwargs=None,
3     max_violations=8, max_pairs=4000)

1 from codesign import (AlgebraicDP, Architecture, Ports, Reals, VecStage,
2     VectorStateGrid, ContinuousAxis, check_vector_monotonicity)
3 dp = AlgebraicDP(F=Ports({"demand": Reals()}),
4     R=Ports({"cost": Reals(), "d_wear": Reals()}),
5     equations={"cost": 1.0, "d_wear": 1.0})
6 stage = VecStage("leg", lambda s: {"demand": 1.0},
7     lambda s, p: {"wear": dict(s)["wear"] + p["d_wear"]},
8     candidates=[Architecture("run", dp)])
9 grid = VectorStateGrid([ContinuousAxis("wear", 0.0, 4.0, 5)])
10 rep = check_vector_monotonicity([stage], grid, cost_axes=["cost"])
11 print(rep)                                # H1=ok, H2=ok, value_monotone=...
12 print(rep.monotone_value_guaranteed)      # True

```

- `stages`, `grid`, `cost_axes`, `architectures`, `max_iter`, `solve_kwargs` — as for the solver above.
- `max_violations` — cap on recorded witnesses per hypothesis.
- `max_pairs` — cap on comparable pairs sampled per stage; below it the check is exact.

Errors: raises the same `ValueError` as the solver when a stage has no candidates and no default architecture set.

13.6.3 online_codesign

The `online_codesign` module is the closed-loop, measurement-driven counterpart of the open-loop temporal planners (section 15.6, worked in example 15.11). At each control step it senses the measured state, reads the live requirement and environment, re-solves the co-design over the admissible architectures, applies the cheapest feasible choice by stepping the true plant, and logs the outcome, so divergence between plan and execution is absorbed by the next sensing step.

ControlStep The audit record of one closed-loop control step: a dataclass capturing the step's measured state, conditions, chosen architecture and point, cost, and feasibility. `run_online_codesign` emits one per step.

```

1 from codesign import ControlStep
2 step = ControlStep(step=0, measured_state=10.0, requirement={"rate": 2.0},
3     environment={}, architecture="node", point={"energy": 2.0, "ops": 3.0},
4     cost=3.0, feasible=True)
5 print(step) # ControlStep(t=0, arch='node', cost=3, ok)

```

Field	Type	Meaning
step	int	step index
measured_state	Any	the state sensed at this step
requirement	Mapping	the live functionality demanded
environment	Mapping	exogenous parameters, recorded for the log
architecture	str	the chosen architecture (empty when infeasible)
point	Mapping, optional	the chosen resource point (None when infeasible)
cost	float	the selected point's cost (∞ when infeasible)
feasible	bool	whether any architecture was feasible

Errors: none raised directly.

OnlineCoDesignResult The trajectory produced by `run_online_codesign`: a dataclass holding the per-step log, the total cost of feasible steps, and whether every step was feasible. The `schedule` property extracts the sequence of chosen architectures.

```

1 from codesign import OnlineCoDesignResult, ControlStep
2 s = ControlStep(0, 10.0, {"rate": 2.0}, {}, "node",
3     {"energy": 2.0, "ops": 3.0}, 3.0, True)
4 res = OnlineCoDesignResult(steps=[s], total_cost=3.0, feasible=True)
5 print(res.schedule) # ['node']
6 print(res) # OnlineCoDesignResult(feasible, steps=1, ...)

```

Field	Type	Meaning
steps	list of ControlStep	the closed-loop audit trail
total_cost	float	summed cost over feasible steps
feasible	bool	True iff every step was feasible

The property `schedule` returns `[s.architecture for s in steps]`. Errors: none raised directly.

resolve_at Solve every architecture at a fixed functionality and pick the cheapest feasible point by `cost_fn`. This is the single myopic co-design solve performed once per control step; it is exposed separately so a caller can drive the re-solve directly. Signature: `resolve_at(architectures, functionality, cost_fn, *, max_iter=200, solve_kwargs=None)`.

```

1 from codesign import AlgebraicDP, Architecture, Ports, Reals, resolve_at
2 dp = AlgebraicDP(F=Ports({"rate": Reals()}),
3     R=Ports({"energy": Reals(), "ops": Reals()}),
4     equations={"energy": lambda f: f["rate"], "ops": 3.0})
5 name, point, cost = resolve_at([Architecture("node", dp)],
6     {"rate": 2.0}, cost_fn=lambda p: p["ops"])
7 print(name, cost) # node 3.0
8 print(point["energy"]) # 2.0

```

- `architectures` — the admissible configurations to solve.
- `functionality` — the live demand to solve each at.
- `cost_fn` — scalar cost on a resource point; the cheapest feasible point wins.
- `max_iter`, `solve_kwargs` — forwarded to `solve`.

Returns `(arch_name, point, cost)`, or `("", None, inf)` when no architecture is feasible. Errors: none raised directly.

run_online_codesign Run the closed-loop, measurement-driven co-design loop for `n_steps` steps. Each step senses, reads the live requirement and environment, re-solves via `resolve_at`, applies the cheapest feasible choice by stepping the `plant` (the true process, not a nominal model), and logs a `ControlStep`; an infeasible step holds the state and is recorded without aborting the run.

```
1 run_online_codesign(architectures, *, n_steps, sensor, requirement,
2     plant, cost_fn, environment=None, initial_state=None,
3     max_iter=200, solve_kwargs=None)

1 from codesign import (AlgebraicDP, Architecture, Ports, Reals,
2     run_online_codesign)
3 dp = AlgebraicDP(F=Ports({"rate": Reals()}),
4     R=Ports({"energy": Reals(), "ops": Reals()}),
5     equations={"energy": lambda f: f["rate"], "ops": 3.0})
6 res = run_online_codesign([Architecture("node", dp)], n_steps=3,
7     sensor=lambda t, s: s,
8     requirement=lambda t, s: {"rate": 2.0},
9     plant=lambda t, s, a, p: s - p["energy"],
10    cost_fn=lambda p: p["ops"], initial_state=10.0)
11 print(res)           # OnlineCoDesignResult(feasible, steps=3, ...)
12 print(res.schedule)  # ['node', 'node', 'node']
13 print(res.total_cost) # 9.0
```

Parameter	Type	Meaning
<code>architectures</code>	seq. of Architecture	re-evaluated every step
<code>n_steps</code>	int	number of closed-loop steps
<code>sensor</code>	callable	<code>sensor(step, prev_state) -> measured_state</code>
<code>requirement</code>	callable	<code>requirement(step, measured) -> functionality</code>
<code>plant</code>	callable	<code>plant(step, measured, arch, point) -> next_state</code> ; the true process
<code>cost_fn</code>	callable	scalar cost selecting the point each step
<code>environment</code>	callable, optional	<code>environment(step, measured) -> params</code> ; logged, empty by default
<code>initial_state</code>	Any	handed to the first <code>sensor</code> call
<code>max_iter</code> , <code>solve_kwargs</code>		forwarded to <code>solve</code>

Errors: none raised directly. Fold `environment` into `requirement` when it should influence the solve; it is otherwise only recorded.

14 Worked Examples

The package ships 25 runnable examples under `examples/` and the same 25 as executed notebooks under `notebooks/`. Each is referenced below by its filename. This section walks through examples 1–17; the temporal, vector-state, and online examples 18–25 are covered in Section 15.

14.1 Example 1: the drone (Fig. 48 of the paper)

01_drone.py. The canonical MCDP from Censi’s paper: the “hello world” of monotone co-design, and the smallest complete use of the operator API (FunctionDP closed by loop; see 13.2.1 and 13.2.3).

Model

A battery powers an actuator that must lift the battery itself plus a fixed payload, against a fixed avionics power draw. The functionalities are the mission endurance (s), an `extra_payload` (kg) and an `extra_power` (W); the resource of interest is the battery mass (kg). The physics is a lumped model,

$$m \geq \frac{(c(g(m + m_{\text{pay}}))^2 + P_{\text{ex}}) T}{\alpha},$$

with $\alpha = 1.8 \times 10^6$ J/kg the cell specific energy. The battery mass m appears on *both* sides: it is the loop axis. The inner FunctionDP emits the required mass under two names – `battery_mass` (fed back by loop, axis `battery_mass`) and `report_mass` (mirrored onto the outer R so the converged value survives on the interface rather than collapsing to a unit poset). All ports are on $\mathbb{R}_{\geq 0}$ (see 13.1.1).

Code

```
1 def build_drone():
2     F = Ports({
3         "endurance": Reals(unit="s"),
4         "extra_payload": Reals(unit="kg"),
5         "extra_power": Reals(unit="W"),
6         "battery_mass": Reals(unit="kg"),
7     })
8     R = Ports({
9         "battery_mass": Reals(unit="kg"), # loop axis
10        "report_mass": Reals(unit="kg"), # mirror, stays visible
11    })
12    inner = FunctionDP(F, R, drone_inner_h, name="drone-inner")
13    return loop(inner, axis="battery_mass", name="drone")
```

Output

```
Short, light: endurance=60s, payload=0.10kg, extra_P=1.0W
            iters=9, feasible=True, Antichain[(report_mass=0.0003564 kg)]

Medium, modest: endurance=300s, payload=0.50kg, extra_P=5.0W
            iters=22, feasible=True, Antichain[(report_mass=0.04921 kg)]

Longer mission: endurance=600s, payload=0.50kg, extra_P=5.0W
            iters=41, feasible=True, Antichain[(report_mass=0.1283 kg)]

Marginal: endurance=600s, payload=1.00kg, extra_P=10.0W
            iters=16, feasible=False, Antichain[(report_mass=⊥)]

Infeasible: endurance=1800s, payload=1.00kg, extra_P=10.0W
            iters=8, feasible=False, Antichain[(report_mass=⊥)]
```

Reading

The example sweeps five mission profiles, three feasible and two infeasible. The converged battery mass grows steeply with endurance – doubling the mission from 300 s to 600 s roughly triples the mass (0.049 kg to 0.128 kg), because heavier batteries demand more lift power, which demands still more battery, the feedback the loop resolves. Beyond the feasibility boundary the fixed point is $\top$ (printed as the top element): no finite mass closes the loop. Iteration counts (8–41) track distance from that boundary: cases that diverge to $\top$ or settle near the origin resolve fastest, while the near-feasible 600 s/0.5 kg case takes longest to climb. Examples 14.6 and 14.7 rebuild this exact model through the MCDP and `System` builders; the solver itself is `solve` (13.2.4).

14.2 Example 2: integer optimisation (Sec. VI-D)

`02_integer_optimization.py`. The classic Sec. VI-D problem from the paper, and the cleanest illustration of *multi-point* antichains ascending under the Kleene iteration. It exercises `Naturals` (13.1.1), a multi-valued `FunctionDP` (13.2.1) and the antichain algebra (13.1.2).

Model

Find the minimal pair $(x, y) \in \mathbb{N} \times \mathbb{N}$ with

$$x + y \geq \lceil \sqrt{x} \rceil + \lceil \sqrt{y} \rceil + c.$$

Both loop variables are bundled into a single composite axis `xy` (a nested `Ports` of two `Naturals`), so one `Loop` closes the pair. For each candidate the inner relation enumerates every split of the deficit c across the two coordinates – x_{out} ranging over $[\lceil \sqrt{x} \rceil, \text{target} - \lceil \sqrt{y} \rceil]$ – so the antichain gains several incomparable points at once. Infinities are mapped to the top pair, giving the iteration a well-defined $\top$ to rest at when no finite pair works.

Code

```
1 target = sx + sy + c          # sx,sy = ceil(sqrt(x_in)),ceil(sqrt(y_in))
2 pts = []
3 for x_out in range(sx, target - sy + 1):
4     y_out = target - x_out
5     if y_out < sy:
6         break
7     pts.append({"xy": {"x": x_out, "y": y_out},
8               "xy_report": {"x": x_out, "y": y_out}})
9 return Antichain.from_set(R, pts)
10 # ... inner wrapped and closed:
11 inner = FunctionDP(F=F, R=R, h_fn=h, name=f"sqrt_sum(c={c_value})")
12 return Loop(inner, axis="xy")
```

Output

```
c = 1: iters = 6, feasible = True
S_0: { (0, 0) }
S_1: { (0, 1), (1, 0) }
S_2: { (0, 2), (1, 1), (2, 0) }
S_3: { (0, 3), (1, 2), (2, 1), (3, 0) }
S_4: { (0, 3), (2, 2), (3, 0) }
S_5: { (0, 3), (3, 0) }
S_6: { (0, 3), (3, 0) }
M(c=1) = { (0, 3), (3, 0) }
```

```

c = 4: iters = 6, feasible = True
...
M(c=4) = { (0, 7), (3, 6), (4, 4), (6, 3), (7, 0) }

```

Reading

The trace makes the *ascent* concrete: the seed $S_0 = \{(0,0)\}$ grows, peaks (four points at S_3 for $c = 1$), then *contracts* as the `Min` operation drops points that become dominated, settling at the antichain $M(c)$ of minimal solutions. For $c = 4$ it converges in six steps to a five-point front. The example deliberately flags a typo in the paper: the paper's $M(1) = \{(1,0), (0,1)\}$ is wrong; the solver's $\{(0,3), (3,0)\}$ is correct, since $(1,0)$ gives $1 \not\geq \lceil \sqrt{1} \rceil + 0 + 1 = 2$. That the number of points is non-monotone across iterations is the whole point of working in antichains rather than scalars. Example 14.5 plots these very iterates; the same `make_looped` builder is imported there.

14.3 Example 3: AUV seabed surveying (Sec. VIII)

`03_auv_seabed.py`. Example 10 of the paper: a genuinely cyclic co-design where velocity trades against sensor width, both feeding back through mission time. It shows a `Loop` (13.2.3) over a `FunctionDP` that emits a small Pareto spread, then collapses it with `minimize_cost` (13.2.4).

Model

An autonomous underwater vehicle sweeps an area A (m^2) at velocity v (m/s) with sensor field of view r (m). The couplings are intrinsically cyclic: a larger v shortens the time $T = kA/(vr)$ but raises actuation power $P_{\text{act}} = \psi v^3$; a wider r also shortens T but raises sensing power $P_{\text{sens}} = \chi r$ and cost. Energy is $E = (P_{\text{act}} + P_{\text{sens}})T$. The design pair (v, r) is bundled into the loop axis `design`; the outer resources T , E , `cost` stay visible. At each step the inner relation enumerates a few (v, r) candidates above the current loop value (up to the physical caps $v \leq 3$, $r \leq 5$), producing the front; hitting a cap lifts the axis to $\top$ to signal infeasibility.

Code

```

1 pts = []
2 for v_try in (v, min(v*1.3, V_MAX), min(v*1.7, V_MAX)):
3     for r_try in (r, min(r*1.3, R_MAX), min(r*1.7, R_MAX)):
4         if v_try < v_in or r_try < r_in:
5             continue
6         T_try = K_GEOM * A / (v_try * r_try)
7         E_try = (PSI_A*v_try**3 + CHI_A*r_try) * T_try
8         pts.append({"design": {"v": v_try, "r": r_try},
9                     "T": T_try, "E": E_try,
10                     "cost": SENSOR_COST_A * r_try})
11 return Antichain.from_set(R, pts)

```

Output

```

Area = 100 m^2
  iters=2, feasible=True
  T=1176s, E=29.6kJ, $=100
  T=905s, E=29.5kJ, $=130
  T=692s, E=29.5kJ, $=170
  best by composite cost: T=692s, E=29.5kJ, $=170

Area = 1000 m^2

```



```

iters=2, feasible=True
T=11765s, E=295.9kJ, $=100
...
best by composite cost: T=6920s, E=295.1kJ, $=170

```

Reading

Each scenario yields a three-point Pareto front trading survey time against sensor cost: the fastest sweep ($T = 692$ s) also costs the most (\$170, the widest sensor), while the cheapest (\$100) is slowest. Energy barely moves across the front – the drag and sensing terms nearly cancel over these (v, r) choices – so it is time-versus-cost that drives the decision. Time scales linearly with A ($100 \text{ m}^2 \rightarrow 10\,000 \text{ m}^2$ scales T by exactly $100\times$), confirming the coverage relation. The composite scalar cost ($1/\text{s} + 0.05/\text{kJ} + 1/\text{\$}$) is dominated by mission time, so `minimize_cost` always selects the fastest design. Contrast with the multi-objective handling in Example 14.8, where the front is genuinely two-dimensional.

14.4 Example 4: UncertainDP and ODE_DP

`04_uncertain_and_ode.py`. Two short demos of the advanced primitives: `UncertainDP` (13.2.1) for set-bracketed uncertainty, and `ODE_DP` (13.2.1) for deriving a monotone relation from a differential equation. Both are built on `AlgebraicDP` (13.2.1) internally.

Model

Uncertainty. A battery’s specific energy is known only to lie in $[1.6, 2.0]$ MJ/kg. Two `AlgebraicDP`s bracket it: $m = \text{cap}/1.6\text{e6}$ (pessimistic, more mass) and $m = \text{cap}/2.0\text{e6}$ (optimistic). `UncertainDP` pairs them as `lower/upper`; `with_mode` selects which bound to solve. *ODE.* A heater holds a temperature rise ΔT above ambient; Newton cooling gives $\dot{T} = (P_{\text{in}} - h\Delta T)/C$, so at steady state $P_{\text{in}} = h\Delta T$. `ODE_DP` integrates the right-hand side $\dot{x} = h\Delta T - x$ to its steady state and extracts the root as the required power – a monotone R relation recovered from dynamics rather than written in closed form.

Code

```

1 uncertain = UncertainDP(F=F, R=R, lower=optimistic,
2                         upper=pessimistic, mode="upper",
3                         name="battery_uncertain")
4 for mode in ("lower", "upper"):
5     result = solve(uncertain.with_mode(mode), {"capacity": 3.6e6})
6
7 heater = ODE_DP(
8     F=F, R=R,
9     rhs=lambda x, t, f: H_LOSS * f["delta_T"] - x,
10    extract=lambda x: {"power": float(x)},
11    mode="steady_state", x0_fn=lambda f: 0.0, name="heater_ode")

```

Output

UncertainDP demo: battery sizing under specific-energy uncertainty

```

optimistic   (lower): mass = 1.800 kg
pessimistic  (upper): mass = 2.250 kg

```

ODE_DP demo: power required to hold a steady temperature rise

<code>delta_T =</code>	<code>5 K</code>	<code>--></code>	<code>P_in =</code>	<code>4.0 W</code>	<code>(= h_loss * delta_T = 4.0 W)</code>
<code>delta_T =</code>	<code>20 K</code>	<code>--></code>	<code>P_in =</code>	<code>16.0 W</code>	<code>(= h_loss * delta_T = 16.0 W)</code>
<code>delta_T =</code>	<code>50 K</code>	<code>--></code>	<code>P_in =</code>	<code>40.0 W</code>	<code>(= h_loss * delta_T = 40.0 W)</code>

Reading

For 1 kWh (3.6 MJ) the optimistic cell needs 1.80 kg, the pessimistic 2.25 kg; the true mass lies between, and any design that survives the 2.25 kg (upper) case is robust to the chemistry actually delivered. This is the set-based philosophy of Sec. VII made operational: solve both brackets, commit to the pessimistic one. The heater demo is a sanity check – the numerically integrated steady state reproduces the closed form $P_{\text{in}} = h \Delta T$ exactly ($h = 0.8 \text{ W/K}$), confirming that ODE_DP recovers the algebraic relation a modeller would otherwise hand-derive. See 8 for the theory these two primitives implement; Examples 14.11 and 14.12 push the uncertainty machinery further.

14.5 Example 5: visualising the Kleene ascent

05_visualize_kleene.py. A visual companion to Example 14.2: it renders each iterate $S_0, S_1, \dots$ of the Sec. VI-D problem as a scatter panel, reproducing the structure of Fig. 36 of the paper. It shows how the trace recorded by solve (13.2.4, via record_trace=True) feeds a plotting routine.

Model

The MCDP is unchanged from Example 14.2 – the script imports make_looped from 02_integer_optimization rather than redefining it (the module name starts with a digit, so importlib is used). The novelty is entirely in consuming result.trace: a list of TraceEntry records (13.2.4), one per Kleene iterate, each carrying the antichain at that step. Each panel plots the finite points of one S_k in $\mathbb{N} \times \mathbb{N}$; $\top$ points are skipped.

Code

```

1 result = solve(looped, {"c": c_value}, max_iter=50, record_trace=True)
2 trace = result.trace
3 for k, entry in enumerate(trace):
4     A = entry.antichain
5     xs = [p["xy"]["x"] for p in A.points if p["xy"]["x"] != inf]
6     ys = [p["xy"]["y"] for p in A.points if p["xy"]["y"] != inf]
7     axes[k].scatter(xs, ys, s=60, c="C3", zorder=3)
8     axes[k].set_title(f"$S_{{k}}$ ({{len(A.points)}} pts)")
9 fig.savefig(out_path, dpi=140, bbox_inches="tight")

```

Output

The script writes one PNG per value of $c \in \{1, 4, 8\}$ (matplotlib in the Agg backend, no display required). Console output is only the progress log:

```

Plotting Kleene iteration traces into ../outputs ...
wrote ../outputs/kleene_trace_c1.png
wrote ../outputs/kleene_trace_c4.png
wrote ../outputs/kleene_trace_c8.png

```

Committed copies of the figures live under docs/images/ (Figure 1).

Kleene ascent on $x + y \geq \text{ceil}(\text{sqrt}(x)) + \text{ceil}(\text{sqrt}(y)) + 4$
converged in 6 iterations, $|M(c=4)| = 5$ minimal points

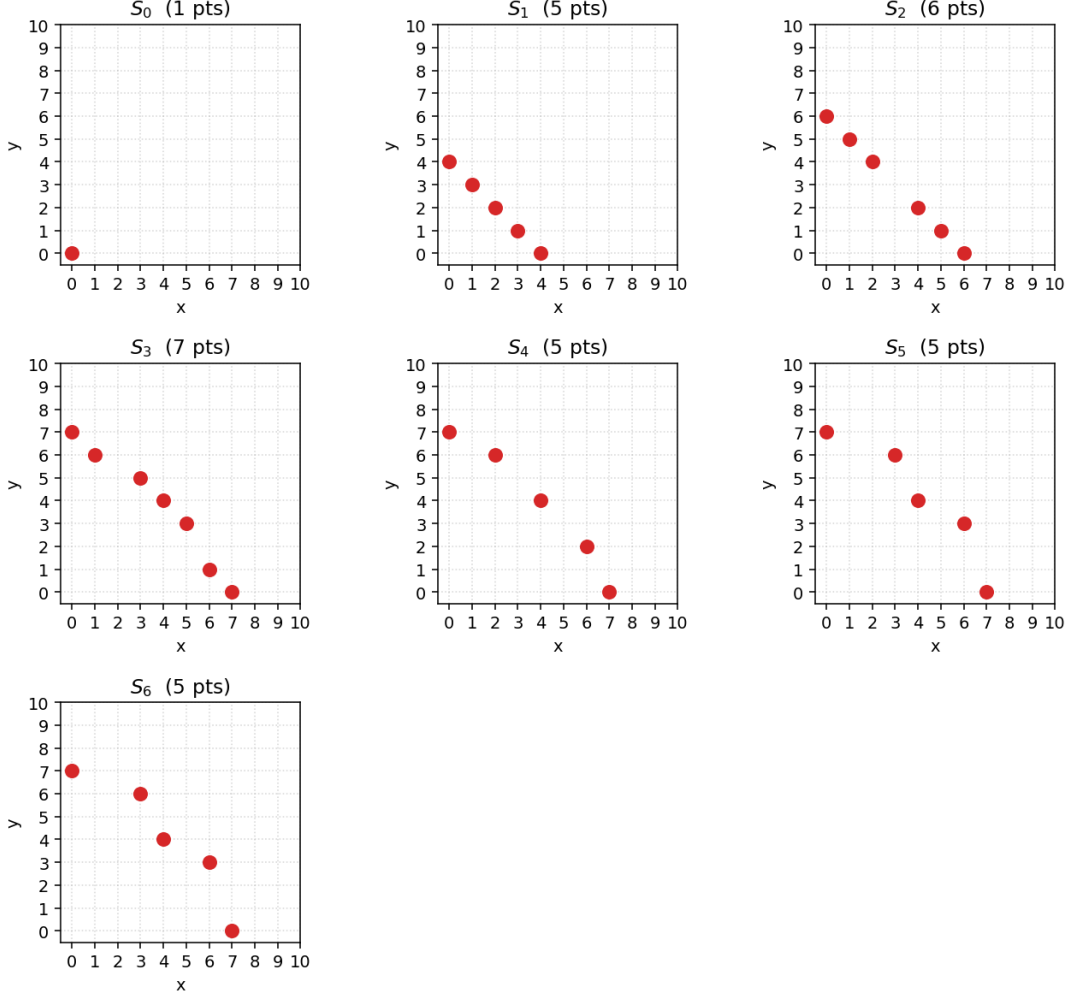


Figure 1: Kleene ascent for $c = 4$: each panel is one iterate S_k of the antichain in $\mathbb{N} \times \mathbb{N}$, from the seed $\{(0,0)\}$ to the five-point fixed point $M(4)$.

Reading

The panels make the antichain algebra tangible: points appear along a frontier $x + y = \text{const}$, the frontier advances outward, and then dominated points fall away under Min until only the incomparable minimal pairs remain. The figure is the exact geometric content of the Sec. VI-D trace printed textually in Example 14.2; reading the two together is the fastest way to build intuition for what a Kleene fixed point over antichains actually looks like. This is the only example in the first eight that touches matplotlib and therefore has a non-trivial import cost (≈ 1.2 s wall-clock, essentially all of it library import).

14.6 Example 6: the drone in MCDPL syntax

`06_drone_mcdpl_syntax.py`. The Example 14.1 drone rebuilt through the declarative MCDP builder (13.3.1), whose notation mirrors the paper's `mcdp{...}` blocks. It demonstrates the highest-level authoring layer: `provides/requires/ constraint/loop_on` inside a context manager.

Model

The MCDP is identical to Example 14.1: same physics, same feedback on `battery_mass`. Only the *spelling* changes. Inside a `with MCDP("drone") as m:` block, functionalities are declared with `m.provides`, resources with `m.requires`, each resource is given its defining expression by `m.constraint`, and the recursion is closed by `m.loop_on("battery_mass")`. As before, `report_mass` mirrors the loop value onto the visible interface. `m.build()` compiles the block down to the same `FunctionDP-in-a-Loop` the operator API produces by hand.

Code

```
1 with MCDP("drone") as m:
2     m.provides("endurance", unit="s")
3     m.provides("extra_payload", unit="kg")
4     m.provides("extra_power", unit="W")
5     m.provides("battery_mass", unit="kg")      # loop variable
6     m.requires("battery_mass", unit="kg")      # loop axis
7     m.requires("report_mass", unit="kg")      # mirror
8     m.constraint("battery_mass", battery_mass_eq)
9     m.constraint("report_mass", battery_mass_eq)
10    m.loop_on("battery_mass")
11 return m.build()
```

Output

```
DP(loop(drone, axis=battery_mass): endurance:R+[s]×extra_payload:R+[kg]
    ×extra_power:R+[W] -> A[report_mass:R+[kg]])

Short, light: endurance=60.0, extra_payload=0.1, extra_power=1.0
    iters=9, feasible=True, Antichain[(report_mass=0.0003564 kg)]

Medium, modest: endurance=300.0, extra_payload=0.5, extra_power=5.0
    iters=22, feasible=True, Antichain[(report_mass=0.04921 kg)]

Longer mission: endurance=600.0, extra_payload=0.5, extra_power=5.0
    iters=41, feasible=True, Antichain[(report_mass=0.1283 kg)]

Infeasible: endurance=1800.0, extra_payload=1.0, extra_power=10.0
    iters=8, feasible=False, Antichain[(report_mass=⊥)]
```

Reading

The numbers are identical to Example 14.1, to the last digit (0.0003564 kg, 0.04921 kg, 0.1283 kg) and identical iteration counts (9, 22, 41, 8). That is the point: the MCDP builder is a front-end that lowers to exactly the same internal DP as the operator API, so the choice between them is purely one of authoring ergonomics. The printed signature confirms the compiled object is `loop(drone, axis=battery_mass)` – the same `Loop` as before. Prefer this layer when a model is naturally a flat list of constraint equations; drop to `FunctionDP/loop` (13.2.1, 13.2.3) when a relation needs arbitrary Python. Example 14.7 shows the third route, the `System/Module` builder.

14.7 Example 7: the drone, modular

`07_drone_modular.py`. The Example 14.1 drone a third way: as a `System` (13.3.2) of `Module` subclasses wired by operator-overloaded `>=` constraints. This is the layer for genuinely modular models – each subsystem carries its own F/R and relation.

Model

Two `Module` subclasses declare class-level F/R and an `h`: `Battery` maps `capacity`→`mass` via specific energy, `Actuator` maps `lift_force`→`power` as $c \text{ lift}^2$. The `System` exposes the outer ports, adds the two modules, and states three inequalities that read like the textbook constraints: battery capacity $\geq \text{power} \times \text{endurance}$, actuator lift $\geq g(m_{\text{batt}} + m_{\text{pay}})$, and total mass $\geq m_{\text{batt}} + m_{\text{pay}}$. Each `module.port` is a handle; `>=` records a constraint rather than evaluating a boolean.

Code

```
1 sys = System("drone")
2 endurance = sys.provides("endurance", unit="s")
3 extra_payload = sys.provides("extra_payload", unit="kg")
4 extra_power = sys.provides("extra_power", unit="W")
5 total_mass = sys.requires("total_mass", unit="kg")
6 battery = sys.add("battery", Battery())
7 actuator = sys.add("actuator", Actuator())
8 battery.capacity >= (actuator.power + extra_power) * endurance
9 actuator.lift_force >= G * (battery.mass + extra_payload)
10 total_mass >= battery.mass + extra_payload
11 return sys      # sys.build() compiles to a looped DP
```

Output

```
System('drone'):
  provides: endurance, extra_payload, extra_power
  requires: total_mass
  subsystems:
    battery: (capacity) -> (mass)
    actuator: (lift_force) -> (power)
  constraints:
    battery.capacity >= ((actuator.power + extra_power) * endurance)
    actuator.lift_force >= (9.81 * (battery.mass + extra_payload))
    total_mass >= (battery.mass + extra_payload)

Short, light: endurance=60.0, ...
  iters=17, feasible=True, Antichain[(total_mass=0.1004 kg)]
Medium, modest: ...
  iters=43, feasible=True, Antichain[(total_mass=0.5492 kg)]
Longer mission: ...
  iters=81, feasible=True, Antichain[(total_mass=0.6283 kg)]
Infeasible: ...
  iters=16, feasible=False, Antichain[(total_mass=⊥)]
```

Reading

The `System` reports `total_mass` (battery + payload), so the numbers differ from Examples 14.1 and 14.6 – which reported only the battery – but the underlying battery masses agree exactly: e.g. the medium case here is $0.5492 = 0.04921 + 0.5$ kg. Iteration counts are roughly $2\times$ those of the single-axis model (43 vs 22, 81 vs 41) because the compiled loop now closes two coupled internal axes (battery mass and actuator power) rather than one, so the fixed point takes more sweeps to propagate. The declarative `>=` form and the lambda form `sys.constrain(target, fn)` compile to the same constraint list. This is the recommended layer for multi-subsystem designs; Example 14.8 scales it to three modules including a catalog.

14.8 Example 8: a modular vehicle with Pareto tradeoffs

`08_vehicle_modular.py`. A `System` (13.3.2) of three subsystems where one is a discrete `CatalogDP` (13.2.1). Because the catalog holds Pareto-incomparable motors, the system result is a genuine multi-point front over total mass and total cost, collapsed by `minimize_cost` (13.2.4).

Model

Three modules are wired: a `CatalogDP` motor with seven entries (each a `CatalogEntry` giving torque at a (mass, cost); some share torque but trade mass against cost, so they are incomparable), a linear `Chassis` (mass and cost $\propto$ load) and a `Battery` (sized by energy). Five $\geq$ constraints close the loop: chassis load $\geq$ payload + motor + battery mass, motor torque $\geq$ a load-dependent demand, battery energy $\geq$ mission energy, and two aggregations for `total_mass` and `total_cost`. The mutual dependence of chassis mass and motor choice makes this cyclic.

Code

```
1 motor = sys.add("motor", make_motor_catalog()) # 7-entry CatalogDP
2 chassis = sys.add("chassis", Chassis())
3 battery = sys.add("battery", Battery())
4 chassis.load ≥ payload + motor.mass + battery.mass
5 motor.torque ≥ TORQUE_PER_KG * G * (payload + chassis.mass
6               + battery.mass)
7 battery.energy ≥ mission_energy
8 total_mass ≥ payload + motor.mass + chassis.mass + battery.mass
9 total_cost ≥ motor.cost + chassis.cost + battery.cost
```

Output

```
Small parcel: payload=2.0, mission_energy=200000.0
  iters=4, feasible=True
  Pareto front (2 points):
    total_mass= 4.82 kg, total_cost=$ 413.00
    total_mass= 5.78 kg, total_cost=$ 165.00
  cheapest: total_mass=5.78 kg, total_cost=$165.00

Medium load: payload=10.0, mission_energy=1000000.0
  iters=3, feasible=True
  Pareto front (2 points):
    total_mass= 22.49 kg, total_cost=$ 475.00
    total_mass= 20.41 kg, total_cost=$ 969.00
  cheapest: total_mass=22.49 kg, total_cost=$475.00

Heavy + long: payload=20.0, mission_energy=5000000.0
  iters=2, feasible=False
```

Reading

Two of the three missions produce genuine two-point fronts: a lighter but pricier motor versus a heavier but cheaper one. For the small parcel the choice is stark – the light design is only 0.96 kg lighter yet \$248 dearer – so `minimize_cost` on cost picks the 5.78 kg/\$165 option. The heavy-plus-long mission is *infeasible* and the loop detects it in just two iterations: no motor in the catalog can supply the torque once the chassis has grown to carry a 5 MJ battery, so the fixed point is $\top$. This is the first-eight showcase of a discrete catalog driving a system-level antichain; the microgrid flagship (Example 14.13) and the full-vehicle study (Example 14.17) build on the same `CatalogDP` pattern at much larger scale.

14.9 Example 9: a robotic arm with non-trivial topology

09_robotic_arm.py. Five subsystems wired with eight constraints, showing where the operator-overloaded builder (12) earns its keep: the connection graph is neither series nor parallel nor a single clean loop, and the cyclic structure emerges from the constraint lines rather than from any explicit loop machinery.

Model

The arm carries two revolute `Joint` modules (shoulder, elbow), a `Sensor`, a `Controller`, and a `Battery`. Each joint maps functionality (torque, speed) to resources (mass, electric_power) with $\text{mass} = \rho \tau$ and $\text{electric_power} = \tau \omega / \eta$. The topology is genuinely coupled in three places. The controller’s power draw grows with both the sensor sample rate and the joint command rate; the shoulder must lift the payload *and* the elbow joint’s own mass at the end of its longer arm, so `shoulder.torque` reads `elbow.mass` and closes a mechanical cycle through the elbow; and the battery is sized by the mission-integrated power of every electrical subsystem. The single system-level resource is `total_mass` over $\mathbb{R}_{\geq 0}$ (kg).

Code

The constraint block is the whole point: each line is one physical relationship, and `elbow.mass` on the right of the shoulder torque constraint is what makes the Kleene iteration iterate.

```
1 elbow.torque ≥ G * payload_mass * elbow_arm_length
2 # shoulder lifts payload + the elbow joint's own mass: a cycle
3 shoulder.torque ≥ G * (payload_mass + elbow.mass) * shoulder_arm_length
4 sensor.sample_rate ≥ 2.0 * control_rate # Nyquist
5 controller.input_rate ≥ 2.0 * control_rate
6 controller.command_rate ≥ control_rate
7 battery.energy ≥ operating_time * (
8     elbow.electric_power + shoulder.electric_power
9     + controller.power + sensor.power)
10 total_mass ≥ (payload_mass + elbow.mass + shoulder.mass
11     + sensor.mass + controller.mass + battery.mass)
```

Output

Printing the assembled `System` echoes the nine constraints back in normalised form, then three mission profiles solve to a single minimal mass each. Every profile converges in four Kleene steps.

```
Pick-and-place light:
  iters=4, feasible=True
  total_mass = 1.97 kg
Heavier payload:
  iters=4, feasible=True
  total_mass = 7.30 kg
Long-reach precise:
  iters=4, feasible=True
  total_mass = 7.24 kg
```

No figures are produced.

Reading

The four-step convergence for every profile shows the elbow-mass cycle is shallow: one pass to size the elbow, one to feed its mass into the shoulder, and the fixed point settles. The heavier-payload and long-reach cases land at almost the same total mass by different routes — one is

payload-dominated, the other actuator- and battery-dominated through the higher control rate and longer operating time. This is the cleanest small demonstration of the constraint-graph style; contrast it with the modular vehicle of example 14.8 and the flagship microgrid cycle of example 14.13, both of which use the same `System` builder (12) at larger scale.

14.10 Example 10: watching the solver work

`10_solver_trace.py`. The two-module drone of example 14.7, instrumented to exhibit every observability feature of `solve` in one script (see the solver observability section, 7.4, and the reference card 13.2.4): silent mode, the end-of-solve summary, the per-iteration feed, the structured trace, a custom callback, and the `status` field.

Model

A battery (mass = capacity/1.8 MJ/kg) and an actuator (power = $10 F^2$) are coupled by the usual drone cycle: capacity depends on actuator power and endurance, lift force depends on battery mass. The model is deliberately trivial so that the focus stays on the six calls the `__main__` block makes, not on the physics.

Code

The instrumentation is entirely in the keyword arguments to `solve`: `verbose` controls printing, `trace` collects a list of `TraceEntry` records (13.2.4), and `on_iteration` receives each entry as it is produced.

```
1 r = solve(drone, f, trace=True, max_iter=100)
2 print(f"collected {len(r.trace)} trace entries")
3 print(f"first 5 deltas: {[e.delta for e in r.trace[:5]]}")
4
5 def my_logger(entry):
6     if entry.iteration % 5 == 0:
7         print(f"iter {entry.iteration}, |A|={entry.n_points}, "
8               f"delta={entry.delta}")
9
10 solve(drone, f, on_iteration=my_logger, max_iter=100)
```

Output

The nominal solve converges in 43 iterations to the canonical 0.5492 kg. Capping `max_iter` short and posing an under-resourced mission drives `status` through its three distinct values — these are the abbreviated forms of the final beat:

```
# 2. Final summary (verbose=1)
[solve] converged: 43 iters, |A|=1, total=0.3ms, feasible=True

# 6. Status field (max_iter vs converged vs feasible)
max_iter=3:   status='max_iter', feasible=True, iters=3
max_iter=200: status='converged', feasible=True, iters=43
infeasible:   status='diverged', feasible=False, iters=16
```

No figures are produced by the script; the companion notebook (`10_solver_trace.ipynb`) adds a delta-versus-iteration plot showing the characteristic oscillating decay of the Kleene residual to machine precision.

Reading

The decisive line is the last block. `status` separates two conditions that the boolean `feasible` flag conflates: hitting the iteration cap on a solvable problem (`'max_iter'`, where raising the cap would help) versus a genuinely infeasible mission (`'diverged'`, where it would not). The trace's deltas confirm the Kleene iteration is monotone in the fixed-point sense but not monotone step-to-step — the residual zig-zags downward, which is why a single small `max_iter` can report a non-converged but already-feasible incumbent.

14.11 Example 11: set-based deterministic uncertainty

`11_uncertain_drone.py`. The drone of example 14.7 with its battery parametrised by two internal quantities — specific energy and efficiency — that are known only up to an uncertainty set. The question posed is: under the worst-case point of that set, how heavy is the drone? This exercises the set-based half of the uncertainty layer (8), the `Box` (13.4.1) and `Ellipsoid` (13.4.1) brackets.

Model

The battery mass is $\text{capacity}/(\text{specific_energy} \cdot \text{efficiency})$. The nominal parameters (2.0 MJ/kg, 0.90) are chosen so their product, 1.8 MJ/kg of *delivered* energy density, reduces to the canonical drone of examples 14.1, 14.6, and 14.7; the nominal solve converges to the same 0.5492 kg, and both uncertainty sets perturb around that shared reference. Each parameter is declared “more is better,” so worst-case badness is the low-parameter direction. The `Box` admits every combination in the two independent ranges; the `Ellipsoid` is a tilted, correlated set that excludes the joint-extreme corner.

Code

The uncertainty is attached to the module, not the solve; the same system is rebuilt with a different `uncertain_set` per query, and `solve` is asked for the `worst_case` summary.

```
1 bat = Battery()
2 bat.uncertain_set = Box(
3     specific_energy=(1.7e6, 2.3e6, "more_is_better"),
4     efficiency=(0.83, 0.97, "more_is_better"),
5 )
6 drone = make_drone(bat)
7 r_box = solve(drone, f, uncertainty=["worst_case"])
8 wc = list(r_box.worst_case.anticchain.points)[0]["total_mass"]
```

Output

```
Nominal parameters:  total_mass = 0.5492 kg
Box uncertainty:
    worst-case total_mass = 0.5668 kg
    uncertainty penalty   = +0.0176 kg
Ellipsoid uncertainty (smaller, correlated set):
    worst-case total_mass = 0.5527 kg
    uncertainty penalty   = +0.0035 kg
```

No figures are produced.

Reading

The two penalties, +0.0176 kg for the box against +0.0035 kg for the ellipsoid, are the whole lesson. The box worst case sits at the single corner where both parameters are simultaneously at their lowest; the ellipsoid, being correlated and smaller, carves that implausible corner out and returns a far milder worst case. Choosing the uncertainty model is therefore a modelling decision about which joint outcomes are physically credible, not a mere tolerance bound. Example 14.12 takes the same drone stochastically and shows where the worst case sits relative to the mean, p95, and CVaR of a Monte Carlo distribution.

14.12 Example 12: stochastic uncertainty with a Gaussian copula

`12_stochastic_drone.py`. The same drone battery as example 14.11, but now the two internal parameters carry marginal distributions tied by a positive correlation — more energy-dense cells tend also to be more efficient. This is the stochastic half of the uncertainty layer (8): a Stochastic distribution (13.4.1) whose marginals are glued by a GaussianCopula (13.4.1).

Model

Both parameters take a uniform marginal over the same ranges as the box in example 14.11, coupled by a Gaussian copula at correlation 0.4. The nominal product is again 1.8 MJ/kg delivered, so the nominal solve is the canonical 0.5492 kg. A single `solve` call requests all five summaries at once — `worst_case` (from the box still attached), `mean`, `p95`, `cvar95`, and the raw `samples` — over one thousand Monte Carlo draws at a fixed seed.

Code

```
1 bat.uncertain_dist = Stochastic(  
2     marginals={  
3         "specific_energy": stats.uniform(loc=1.7e6, scale=0.6e6),  
4         "efficiency":      stats.uniform(loc=0.83, scale=0.14),  
5     },  
6     copula=GaussianCopula(correlation=[[1.0, 0.4],  
7                                     [0.4, 1.0]]),  
8 )  
9 res = solve(drone, f,  
10     uncertainty=["worst_case", "mean", "p95", "cvar95", "samples"],  
11     n_samples=1000, rng_seed=42)
```

Output

The five summaries come out in the canonical ordering nominal < mean < p95 < cvar95 < worst_case, with a feasibility rate of 1.000.

```
Nominal mass: 0.5492 kg  
  mean total_mass   = 0.5506 kg  
  p95 total_mass    = 0.5632 kg  
  cvar95 total_mass = 0.5647 kg  
  worst-case mass   = 0.5668 kg   (Box, deterministic)  
Raw samples: 1000 antichains (1000 feasible)
```

The script also prints an ASCII histogram of the sampled masses; the companion notebook renders it as a matplotlib histogram with vertical markers at each summary. No image files are written by the script.

Reading

The ordering is the reason to compute all five together. The mean sits barely above nominal because the marginals are centred near the nominal point; p95 and CVaR95 climb into the tail, and the set-based worst case of example 14.11 bounds them all from above. The design engineer reads this as a spectrum of conservatism: size to the mean and half the fleet misses the target, size to CVaR95 and only the worst 5% of parameter draws exceed the mass budget, size to the box worst case and none do — at a heavier vehicle. The copula correlation matters because it concentrates draws along the “both good / both bad” diagonal, shifting the tail summaries relative to an independent model.

14.13 Example 13: the microgrid flagship

13_microgrid.py. An off-grid cabin must supply a daily energy demand and a peak load without grid power. The example is the flagship case study, exercising catalogue choice, a genuine fixed-point cycle, warm-started sweeps, stochastic uncertainty, and the visualisation suite in one script — the whole `System` builder (12) and solver (13.2.4) stack end to end.

Model

Four subsystems contribute over $\mathbb{R}_{\geq 0}$: a solar PV array (cheap but sun-limited), a lithium battery whose chemistry is a discrete parameter (LFP, NMC, LCO, NaIon, each a (Wh/kg, USD/kWh, cycle life) triple), a diesel generator (reliable, carbon-heavy), and a mounting frame. The frame is the source of the cycle: its cost and mass scale with the total supported mass, *frame included*, so the constraint

$$\text{frame.supported_mass} \geq \text{solar.mass} + \text{battery.mass} + \text{diesel.mass} + \text{frame.mass}$$

reads the frame’s own output on its right-hand side and forces the Kleene iteration to iterate to a fixed point rather than resolve in one pass.

Code

The sweep contrasts a cold restart against warm-starting each solve from the previous load’s fixed point via `start_from`.

```
1 warm_iters = 0
2 prev = None
3 for L in loads:                                # 50 points, 5..30 kWh
4     f = {"daily_load_kwh": float(L),
5          "peak_load_kw": 3.0, "backup_hours": 12.0}
6     r = solve(dp, f, max_iter=400, start_from=prev)
7     warm_iters += r.iterations
8     prev = r                                   # reuse the fixed point next step
```

Output

At the test mission (15 kWh/day, 3 kW peak) all four chemistries solve in 24 Kleene iterations; the warm-started sweep saves roughly 10 % of total steps.

LFP	cost=\$	11689	mass= 388.6kg	CO2= 416.1kg/yr	(iters=24)
NMC	cost=\$	14628	mass= 356.2kg	CO2= 416.1kg/yr	(iters=24)
NaIon	cost=\$	11328	mass= 432.8kg	CO2= 416.1kg/yr	(iters=24)

```
cold-start total iters: 1205
warm-start total iters: 1083
```

```

speedup: 1.11x

nominal cost:  $11689   MC mean cost: $11930
p95 cost:      $15162   CVaR95 cost:  $16127

```

The script writes three files to `outputs/`: `microgrid_convergence.png` (the Kleene residual trace), `microgrid_uncertainty.png` (the Monte Carlo cost distribution, x-axis labelled “total cost (USD)”), and `microgrid.dot` (Graphviz source for the system diagram).

Reading

The chemistry table is a Pareto snapshot: NaIon is the cheapest but heaviest, NMC the lightest but dearer, LFP the balanced default, and CO₂ is fixed because it is set by the diesel backup, not the chemistry. The 24-iteration count confirms the frame cycle is real — a acyclic model would resolve in one or two passes. The 1.11× warm-start speedup is modest but free: consecutive loads have nearby fixed points, so seeding from the previous solution shaves off the early Kleene steps. The stochastic pass shows the design is robust — feasibility stays at 1.000 as sun hours vary — while the tail cost climbs by roughly 40% from nominal to CVaR95, the premium for sizing against bad-sun years. Compare the discrete-catalogue mechanics here with the online elimination of example 14.14, which prunes a catalogue instead of enumerating it.

14.14 Example 14: online elimination over a robot catalogue

`14_online_fleet.py`. A logistics service must hit a target throughput and range by buying robots from a catalogue of 200 candidate types, each described by four features (speed, payload, unit cost, energy per km). Naively this is 200 inner solves; the online solver (15.6, 13.4.2) prunes provably suboptimal candidates using an optimistic evaluator and runs inner solves only for the survivors. The example reproduces the spirit of the multi-robot fleet study of Alharbi, Dahleh & Zardini (arXiv:2604.22624).

Model

Each robot’s inner DP is a smooth (fractional-fleet) `AlgebraicDP` mapping the mission (throughput, range) to `(total_cost, total_energy)`, with `total_cost = (throughput/(sp)) c` and `total_energy = range · e · 24`. Three evaluators are compared: a `LipschitzEvaluator` (13.4.2 and neighbours) with per-output constants; a `MonotonicityEvaluator` on the derived feature `cost_per_capacity = c/(sp)`, under which `total_cost` is exactly monotone; and the certified confidence-polytope `LinearParametricEvaluator`.

Code

The three flavours differ only in the evaluator object handed to `solve_online`; the inner-DP factory and mission are shared.

```

1 res = solve_online(make_dp, mission,
2                     candidates=candidates,
3                     evaluator=MonotonicityEvaluator(
4                         features=["cost_per_capacity", "energy_per_km"],
5                         r_components=["total_cost", "total_energy"]),
6                     verbose=0)
7 print(res.n_evaluated, res.n_eliminated, len(res.antichain))

```

Output

The exhaustive baseline finds a 5-point Pareto front. All three evaluators recover every Pareto point; they differ only in how much of the catalogue they had to evaluate.

```
True Pareto front: 5 non-dominated candidates

Lipschitz (L=300/30):          evaluated 192/200  Pareto recovery: OK
Monotonicity (cost_per_cap): evaluated  13/200  Pareto recovery: OK
LinearParametric (certified):evaluated 122/200  Pareto recovery: OK
```

The script writes `fleet_online.png` (three feature-plane panels shading each candidate by status) and `fleet_online_convergence.png` (incumbent antichain size against inner solves performed) to `outputs/`.

Reading

The evaluated counts are the story. Monotonicity with the right derived feature prunes hardest — 13 of 200 — because a single monotone coordinate orders the whole catalogue. Lipschitz is the most general and most conservative, evaluating 192 of 200: with a safe L it never drops a Pareto point but rarely eliminates one either. The certified `LinearParametric` evaluator sits between, at 122 of 200, and recovers all five: it maintains the polytope of linear parameter vectors consistent with every observation and lower-bounds each query by one LP over that polytope, so the bound is guaranteed and never wrongly eliminates an optimal candidate. It prunes less aggressively here because `total_cost` is not affine in the raw features; on a truly linear map it becomes exact and aggressive. This certified bound replaces an earlier OLS $\pm$ confidence-band heuristic that pruned to about 35 of 200 but wrongly dropped one Pareto point on this seed — the trade is aggressiveness for a guarantee. Example 14.16 pushes the same three evaluators onto a deliberately nonlinear surface and shows the certified evaluator declining to guess.

14.15 Example 15: monoclonal antibody fed-batch co-design

`15_bioprocess.py`. A biopharmaceutical company must deliver a monoclonal antibody at a target titer (g/L) and annual demand (kg/year). Four subsystems are coupled cyclically, and two of them are catalogue lookups (13.2.1) resolved by the solver. This is the first biotech-upstream application in the library, calibrated to the 2024–2026 bioprocessing literature, and it uses the same `System` builder (12) as the engineering examples.

Model

A `CellLine` module maps demanded titer to the required peak viable-cell density and oxygen uptake via an effective integrated specific productivity q_P . A `FeedStrategy` module turns the glucose set-point into a metabolic-burden factor with a U-shaped batch-failure penalty around 8 mM. Two `CatalogDP` lookups pick the smallest sufficient bioreactor (single-use 200 L through stainless 25,000 L) and the cheapest sufficient media (four commercial CD formulations). The cycle the Kleene iteration resolves runs through the peak cell density: higher titer needs higher density, which inflates the metabolic burden through the feed, which inflates the density needed for the same titer. The outer objective is the vector (COGS, footprint, CO₂ per gram).

Code

Cell line and feed are fixed at construction so the outer loop sweeps them; the bioreactor and media are chosen by the solver. The three outer resources are assembled from ports plus closed-over parameters via `sys.constrain`.

```

1 cell = sys.add("cell", CellLine(*cell_line))
2 feed = sys.add("feed", FeedStrategy(glucose_setpoint_mm))
3 bior = sys.add("bior", make_bioreactor_dp()) # CatalogDP
4 media = sys.add("media", make_media_dp()) # CatalogDP
5 # metabolically-inflated density must be supported by both catalogues
6 bior.peak_vcd ≥ cell.peak_vcd * feed.metabolic_factor
7 media.peak_vcd ≥ cell.peak_vcd * feed.metabolic_factor
8 sys.constrain("cogs_per_g", cogs_eq) # ports + annual_demand, yield

```

Output

At each of the three mission scales the global Pareto front is a genuine two-point tradeoff between the high-producer workhorse CHO-K1 and the next-generation short-batch CHO-MK, both at the 8 mM feed optimum.

```

=== titer 5.0 g/L, annual demand 100 kg/year ===
design          COGS          footprint          CO2
CHO-K1/glu=8mM $ 44.04/g    39.9 m^2      214.29 g/g
CHO-MK/glu=8mM $ 63.65/g    30.5 m^2      214.29 g/g
Cheapest design overall: CHO-K1/glu=8mM

```

No figures are produced.

Reading

The two Pareto points are not comparable: CHO-K1 wins on cost of goods (low licence fee, long batch) while CHO-MK wins on footprint (high licence fee, short batch, so fewer parallel lines at the same annual output). CHO-K1 is the cheapest overall at every scale, and per-gram COGS, footprint intensity, and CO₂ all fall as demand rises, the signature of amortising catalogue capex over more grams. The 8 mM feed wins at both cell lines because the U-shaped penalty punishes both the starvation and the waste-accumulation extremes. All parameters sit within published ranges (specific productivity from Reinhart *et al.* 2019, metabolic limits from Khattak *et al.* 2010 and Lao & Toth 1997); the values are illustrative, not production-precision. Example 14.16 fixes CHO-K1 and turns to the operating-condition optimisation this model leaves open.

14.16 Example 16: online DOE for the mAb fed-batch process

16_online_doe.py. The bioprocess of example 14.15 is fixed (CHO-K1, 100 kg/year, 5 g/L) and the question shifts to where to operate within a $5 \times 5 \times 5 \times 3 = 375$ -point grid of operating conditions — temperature, pH, glucose target, feed-start day. Each candidate stands in for a 10–14-day bioreactor run costing \$20,000–\$100,000, so the online elimination of example 14.14 (15.6, 13.4.2) is now a Design-of-Experiments budget question.

Model

A closed-form effect model maps each condition vector to an effective productivity and thence to the outer triple (COGS, footprint, CO₂), calibrated from the process-parameter literature (temperature downshift from Yoon *et al.* 2003 and Sou *et al.* 2015, pH from Trummer *et al.* 2006, glucose and feed from Khattak *et al.* 2010 and Gagnon *et al.* 2011). The surface is deliberately nonlinear — a temperature/pH/glucose U-shape and a power-law batch-failure rate. Two non-online baselines (a 75-run factorial DOE at the pH=7.1 slice, and a 40-run uniform random sample) are compared against the Lipschitz, Monotonicity, and certified LinearParametric evaluators at a budget of 40 inner solves, scored by how many of the four distinct Pareto *classes* each recovers.

Code

Recovery is counted by output value, not candidate key, because many grid points share the same (COGS, footprint) outcome.

```
1 true_classes = {(round(p["cogs_per_g"], 2),
2                  round(p["footprint_m2"], 1)) for p in true_pareto}
3 for name, ev in evaluators:
4     res = solve_online(make_dp, {"target_titer": 5.0},
5                          candidates=candidates, evaluator=ev, budget=40)
6     recovered = {(round(o["cogs_per_g"], 2),
7                   round(o["footprint_m2"], 1))
8                  for i in res.incumbent_ids for o in [simulate_run(...)]}
9     print(name, len(true_classes & recovered), "/", len(true_classes))
```

Output

The exhaustive baseline (375 solves) yields four Pareto classes. At budget 40 the tuned Lipschitz evaluator matches the 75-run factorial DOE at recovering 3 of 4; the certified LinearParametric evaluator recovers zero, and correctly so.

strategy	evals	elim	Pareto recovered
Factorial DOE (pH=7.1 slice)	75	-	3 / 4 classes (75%)
Random sample (seed=42)	40	-	3 / 4 classes (75%)
Lipschitz (normalised features)	40	0	3 / 4 classes (75%)
Monotonicity (cogs only)	40	0	0 / 4 classes (0%)
LinearParametric	40	0	0 / 4 classes (0%)
Monotonicity + 4 corner warm-start runs: recovery 1/4 classes (25%)			

The companion notebook visualises the pick trajectory in the (temperature, glucose) projection.

Reading

The zero from LinearParametric is the safe-degradation property, not a failure. On the linear catalogue of example 14.14 the same evaluator recovered every Pareto point; here the map is markedly nonlinear, no single affine parameter set fits the observations, the confidence polytope cannot support a non-trivial bound, and the evaluator falls back to the no-information value rather than risk eliminating a candidate it cannot certify as suboptimal. Monotonicity alone is likewise silent because its lower bounds tighten only for candidates above every observation, and the Pareto front lives at the low-feature corner; seeding four hand-picked corner runs lifts it from 0 to 1 class. The generalisable lesson is that the structural prior does the work and must match the problem: a certified global prior is exact on an affine map and deliberately silent off it, a Gaussian process buys reach on smooth nonlinear surfaces at the price of an uncertified bound, and pure local bounds are always safe but need denser observations or a warm-start to concentrate.

14.17 Example 17: full-vehicle co-design across ICE, hybrid, and EV

17_car_codesign.py. The largest example in the package (3351 lines) and the first to model three architecturally distinct variants in one design study. A passenger car is decomposed into per-architecture MCDP modules — 24 for ICE, 23 for hybrid, 18 for EV — each with its own F and R ports, assembled with the `System` builder (12) and the catalogue DP (13.2.1) over engines, transmissions, motors, and battery packs. Because the file is so large, only the EV mass/energy loop and the sweep skeleton are quoted here; the three `build*_car` bodies follow the same template.

Model

ICE, hybrid, and EV share a common chassis core (body, front/rear suspension, front/rear brakes, steering, tires, wheels) and auxiliary core (HVAC, interior, safety, lighting/infotainment); they differ in the powertrain modules. Two coupled cycles close inside every architecture and are resolved by the Kleene iteration. The first is the *mass spiral*: every load-bearing module reads the design curb weight as an F input, and the macro aggregation sums every module weight back into `curb_weight`, so heavier vehicle drives heavier components drives heavier vehicle. The second is the *energy-storage loop*: fuel-tank size (ICE/HEV) or battery capacity (EV/HEV) is sized by consumption $\times$ range, and the storage's own mass re-enters the curb weight. For EVs the pack is 20–35 % of curb weight, so this loop dominates and convergence takes 30–50 iterations against 15–25 for ICE. The macro objective is the vector (cost, curb weight, CO₂), with fuel or energy consumption, maintenance, and durability also reported.

Code

Inside `build_ev_car`, one weight expression is fed to every load-bearing module (the mass spiral) and the battery is sized from the mission range and a consumption lambda (the energy loop).

```
1 total_weight_expr = (body_m.weight + fr_m.weight + rr_m.weight
2   + bf_m.weight + br_m.weight + st_m.weight + ti_m.weight
3   + wh_m.weight + mot_m.weight + bat_m.weight + pe_m.weight
4   + obc_m.weight + red_m.weight + btm_m.weight + DRIVER_MASS_KG)
5 for m in (fr_m, rr_m, bf_m, br_m, st_m, ti_m, wh_m, li_m):
6     m.design_mass ≥ total_weight_expr          # mass spiral
7 sys.constrain("hv_battery.target_range_km",
8               lambda x, r=mission["target_range_km"]: r)
9 sys.constrain("hv_battery.energy_kWh_per_100km",
10              ev_energy_kWh_per_100km)        # closes energy loop
```

Each architecture has a sweep function enumerating its catalogue combinations; `main` runs all three across four missions and keeps the cheapest feasible design per pair.

```
1 def sweep_ev(mission, *, verbose=False):
2     results = []
3     for body in _eligible_bodies(mission):
4         for motor in EV_MOTORS:
5             for battery in EV_BATTERIES:
6                 dp = build_ev_car(mission=mission, body=body,
7                                   motor=motor, battery=battery, ...)
8                 res = solve(dp, dict(mission), max_iter=200)
9                 if res.feasible and res.anticchain.points:
10                     results.append((label, dict(list(
11                         res.anticchain.points)[0])))
12     return results
```

Output

The script prints a per-mission table and a 10-year-TCO summary. The trimmed summary below reproduces the headline claims. For the Family Daily mission (5 passengers, 700 km range, 9 s 0–100): the cheapest ICE is \$35,523 at 196 g/km, the cheapest HEV \$40,272 at 91 g/km with the best 10-year TCO at \$56,427, and the cheapest EV \$64,666 at 58 g/km but at 2434 kg curb weight. The 7-passenger 800 km EV (Suburban Utility) has zero feasible designs.

Mission	Arch	Cost	Weight	CO ₂	10y TCO
Family Daily	ICE	\$35,523	1710kg	196g/km	\$64,998
Family Daily	HEV	\$40,272	1719kg	91g/km	\$56,427
Family Daily	EV	\$64,666	2434kg	58g/km	\$76,052
Suburban Utility	HEV	\$42,510	1910kg	113g/km	\$61,066

Suburban Utility	EV	-	-	-	-	(0 feasible)
------------------	----	---	---	---	---	--------------

The full module graphs are shipped as `docs/diagrams/car_{ice,hev,ev}.svg`; the `draw_system` renderer shades the mass-spiral strongly-connected component in amber.

Reading

The three architectures trade cost against CO₂ monotonically — ICE cheapest to build and dirtiest, EV dearest and cleanest, HEV in between — but the 10-year TCO reorders them: the HEV’s low fuel bill gives it the best total cost of ownership on the Family Daily mission, undercutting both the cheaper-to-buy ICE and the far dearer EV. The 2434 kg EV curb weight is the mass spiral and energy loop compounding: a 100 kWh 800V pack is needed for 700 km, and the pack’s own mass inflates every load-bearing component. The Suburban Utility EV infeasibility — 7 passengers over 800 km — is the model correctly reproducing the real-world absence of such a product against 2024 pack-level energy density (160–180 Wh/kg): the required pack mass never reaches a fixed point below the chassis limit. Calibration cites Genta & Morello, the Bosch handbook, Heywood, Hofmann, and the IEA EV Outlook among others; the numbers are “within published ranges for 2020–2024 production vehicles,” and the contribution is the compositional analysis, not the point values. This example is the capstone of the static-solve examples; the temporal, sequential, and online-co-design regimes begin with the following section (15).

15 Temporal, Vector-State, and Online Co-Design

The examples up to 17 solve a co-design problem *once*: a fixed functionality is posed and the Kleene iteration returns the minimal resource antichain. Many engineering systems are not static. The best architecture changes as the environment changes; a resource is spent or worn across a sequence of stages; the design must be re-decided online as measurements arrive. This section documents the six layers the package adds on top of the ordinary `solve()` to handle these regimes, and the theory that makes them exact. All six reuse a common decision object, the **Architecture** (a named wrapper around a **System**), so they compose with the whole rest of the framework.

15.1 Case 1: architecture switching (temporal)

The simplest temporal problem is *switching*: a system whose best architecture changes because the environment changes, with a cost to switch. The environment is a sequence of epochs, each posing its own functionality and admitting a subset of candidate architectures. `solve_schedule` chooses one architecture per epoch to minimise the total of the per-epoch co-design costs plus the switching costs, by an exact Viterbi (shortest-path) pass over the epoch-by-architecture lattice. The switching cost may be a uniform scalar or a per-transition callable, and a hysteresis parameter lets a brief unfavourable epoch be ridden out on the incumbent rather than switched into and back out of. The per-epoch cost is a full `solve()`, so switching sits on top of static co-design rather than replacing it.

15.2 Case 2, scalar value: dynamic programming (dynamic)

When a resource is carried *between* stages, the problem becomes a dynamic program. `solve_dynamic` runs a finite-horizon backward Bellman recursion whose per-stage decision is which architecture to instantiate, whose per-stage cost is itself a co-design `solve()`, and whose carried scalar state (fuel, charge, cumulative wear) is advanced by a user **transition**. The value function is scalar: at each (stage, state) it is the single best cost-to-go. The state is discretised onto an explicit

StateGrid; a transition that would leave the grid envelope is rejected *before* snapping, so an over-spent resource (a negative fuel level) is never silently rescued by the nearest in-range node. The solver returns a state-indexed **DynamicPolicy**, queryable in closed loop at off-nominal states, and **rollout** threads it forward from a concrete initial state.

15.3 Case 2, antichain value: sequential co-design (sequential)

The multi-objective generalisation replaces the scalar value with a Pareto antichain. Following the sequential co-design theory, fix an ordered commutative resource monoid $(R, \leq, \oplus, 0)$ and work in the value space of upper sets of R ordered by reverse inclusion (a larger feasible set is a lower resource threshold, hence “easier”). A sequential co-design problem over stages $0, \dots, N$ and a state poset $(X, \leq_X, \perp_X)$ assigns to each stage k a state-parametrised minimal-resource map $h_k : X \rightarrow \mathcal{U}R$ and a transition $\phi_k : X \times R \rightarrow X$. The value is the backward recursion $V_{N+1} \equiv R$ and

$$V_k(x) = \text{Min} \bigcup_{r \in h_k(x)} (\uparrow r \oplus V_{k+1}(\phi_k(x, r))),$$

the antichain-valued Bellman operator, implemented by **Antichain.union_min**. In the package the accumulated resource (the named cost axes on the antichain) is kept deliberately *distinct* from the carried state x : the transition reads any carried quantity off the full solved point, while only the cost axes accumulate on the front. The scalar **dynamic** layer is the width-one special case (a single cost axis gives a singleton antichain).

Three results are made operational. **(Q1) Monotone value.** If, for every stage, (H1) h_k is monotone in the carried state (more carried state makes the stage no easier) and (H2) ϕ_k is monotone in the state, then every V_k is monotone. (H1) is load-bearing and necessary. **check_monotonicity** verifies both on the state grid; it is orientation-aware, accepting the benign “consumable but monotone” orientation (a budget carried as remaining slack) and flagging only genuinely non-monotone (perishable or fatigue-as-state) stages, which have an interior optimum and fall outside the guarantee. **(Q2) Front = reachable frontier.** $\text{Min } V_k(x)$ equals the antichain of minimal reachable cumulative resources over feasible choice sequences; there is no tail-pruning gap, and the front size is the width of the reachable set, which grows polynomially in the horizon for a summed objective on a fixed number of axes (**sum_combine**) and stays bounded for a renewable one (**join_combine**). **(Q3) Exact factorisation.** At a reset stage (a quiescent transition landing on $\perp_X$ regardless of input), **detect_resets** and **factorise_at_resets** split the horizon into a $\oplus$ -product of independent sub-problems, the order-theoretic deterministic analogue of regeneration-point decomposition; the proof uses only distributivity, so it holds in every regime.

15.4 The general carried state: vectors (state, vector_dp)

Realistic problems carry structured state, not a lone scalar. The Formula 1 seasonal co-design of Neumann, Zardini, and colleagues [23] carries a state vector of two battery-wear levels plus a discrete regulatory-penalty flag; a self-reconfiguring robot carries the accumulated wear of each drive module plus a shared energy budget. The **state** module provides a **VectorStateGrid**: a product grid over named axes, each either a **ContinuousAxis** (a bucketed real interval, snapped and bounds-checked like the scalar grid) or a **DiscreteAxis** (a finite labelled set with a caller-supplied order, or mutually incomparable when no order is given). **solve_vector_sequential** runs the same antichain-valued Bellman recursion carrying the full state vector, with the transition returning an $\{\text{axis} : \text{value}\}$ mapping snapped onto the product grid and out-of-bounds successors rejected on every axis. The monotonicity results carry over verbatim with the component-wise product order substituted for the scalar order, and **check_vector_monotonicity** verifies (H1)/(H2) over that order. A single-axis grid reproduces the scalar **sequential** result exactly, so the vector layer strictly generalises rather than parallels it.

A subtle correctness point governs the backward pass. The stage antichain must not be Pareto-reduced on the cost axes *before* the transition is applied: a point dominated on cost may be the only feasible choice from a constrained carried state (a higher-cost morphology that spares a worn module), and its consequence for the carried state is invisible in the cost projection. The implementation therefore enumerates all solved points and lets `union_min` prune only after the transition. This is what makes the reconfigurable-robot example (21) feasible.

15.5 Precompute-then-DP: the Formula 1 structure

There are two distinct ways to combine co-design with dynamic programming, and they should not be conflated. The `sequential` solver above re-solves the co-design at every (stage, state), which is necessary when the per-stage functionality depends on the carried state. The Formula 1 seasonal framework [23] uses the cheaper opposite order: the co-design layer is run *once* to produce track-dependent Pareto-optimal performance-versus-wear mappings, which are then frozen into a catalog of implementations that an outer finite-horizon dynamic program selects among. No co-design solve happens inside the Bellman sweep; the DP only indexes the precomputed catalog. The package exposes this explicitly: `precompute_catalog` solves the architectures once at a fixed functionality and returns the tagged Pareto catalog (retaining points that look dominated at the single-stage level, since they may become optimal once aggregated in the DP, exactly the F1 paper’s observation), and `dp_over_catalog` runs the antichain-valued DP selecting catalog indices. Precompute-then-DP agrees exactly with `solve_sequential` when the co-design is state-independent, and is preferred there for cost; when the co-design genuinely depends on the carried state, only the re-solving `sequential` form is correct.

15.6 Online feedback co-design (`online_codesign`)

The layers above are open-loop planners: the whole horizon is solved in advance. The closed-loop counterpart re-decides the design online as measurements arrive. `run_online_codesign` implements a receding-horizon, measurement-in-the-loop controller: at each step it (i) senses the measured state of the running process, (ii) reads the live requirement and environmental conditions, (iii) re-solves the co-design at those conditions over the admissible architectures (`resolve_at` picks the cheapest feasible point), (iv) applies the chosen configuration by stepping the *true* plant, and (v) logs a `ControlStep`. Because the plant is stepped with the true process rather than a nominal transition, any divergence between plan and execution is absorbed by the next sensing step, which is what distinguishes feedback from open-loop replay. This is the co-design instance of control co-design (CCD, [25]) in its nested form, here the myopic variant (re-solve a single static co-design each step); the model is assumed known, so measurements update the carried state and the conditions but not the co-design model itself. Two natural extensions are a receding-horizon lookahead that plans several steps ahead with the vector DP and commits only the first, and online learning of the model from measurements (coupling in the online-learning layer of section 9); both are left for future increments.

15.7 Example 18: metabolic architecture switching

`18_metabolic_switching.py`. The first temporal example (Case 1 of section 15): the best *architecture* changes over time because the environment changes, and the object solved is a schedule of which architecture to run in each epoch, charged for switching. It exercises `solve_schedule` (13.5.1) over the `Epoch` (13.5.1) and `Architecture` (13.5.1) objects of section 15.

Model

An organism alternates between a glycolytic architecture (cheap per unit flux, capped growth rate, no overhead; models fast growth on glucose) and a gluconeogenic one (a larger per-flux bur-

den and a fixed glyoxylate-shunt overhead, but a higher ceiling; models slower, costlier growth on acetate). Each architecture is a small co-design **System**: the outer functionality is the demanded growth rate $\mu \in \mathbb{R}_{\geq 0}$ (1/h), and the outer resource is a dimensionless **burden** summed from an enzyme module (burden linear in μ , or ∞ above $\mu_{\max}$) and a maintenance module (the fixed overhead). The environment is five epochs. Only the substrate-matching architecture is admissible per epoch, except a contested **mixed** epoch, flanked by acetate on both sides, where both are candidates, so the scheduler has a genuine choice. Serving **mixed** on the cheaper glycolytic pathway forces two extra switches (acetate $\rightarrow$ glucose-type and back); riding it out on the incumbent costs none. The trade is resolved by the per-switch re-acclimation cost, the diauxic-shift lag.

Code

```
1 epochs = [
2     Epoch("glucose_1", {"mu": 0.8}, candidates=[GLYCOLYTIC]),
3     Epoch("acetate_1", {"mu": 0.5}, candidates=[GLUCONEOGENIC]),
4     Epoch("mixed", {"mu": 0.6},
5         candidates=[GLYCOLYTIC, GLUCONEOGENIC]),
6     Epoch("acetate_2", {"mu": 0.5}, candidates=[GLUCONEOGENIC]),
7     Epoch("glucose_2", {"mu": 0.8}, candidates=[GLYCOLYTIC]),
8 ]
9 # Same environment, two switching economies:
10 sched_lo = solve_schedule(epochs, cost_fn=burden_cost, switch_cost=0.05)
11 sched_hi = solve_schedule(epochs, cost_fn=burden_cost, switch_cost=0.8)
```

Output

```
Low re-acclimation cost (0.05 per switch):
  mixed      → glycolytic      burden= 0.600 (+switch 0.05)
  total burden = 5.800, switches = 4, feasible = True

High re-acclimation cost (0.8 per switch):
  mixed      → gluconeogenic   burden= 1.920
  total burden = 8.520, switches = 2, feasible = True

Mixed-epoch pathway: glycolytic (cheap switching)
                     → gluconeogenic (costly switching)
```

Reading

Identical environment, different switching economics, qualitatively different metabolic program. Under a low switch cost the scheduler takes four switches and serves **mixed** on the locally cheaper glycolytic pathway (total burden 5.800); under a high switch cost the two round-trip switches no longer pay, so the organism holds the incumbent gluconeogenic pathway through **mixed** (two switches, total burden 8.520). The mixed epoch’s pathway is the visible pivot, and it is exactly the temporal coupling that a static co-design cannot see.

15.8 Example 19: rover module activation

`19_rover_modules.py`. Case 2 of section 15, with a genuinely carried scalar resource. A planetary rover activates and deactivates operating modes across mission phases on a battery that depletes and recharges from solar. It exercises `solve_dynamic` (13.5.2) over a `StateGrid` (13.5.2), returning a `DynamicPolicy` (13.5.2) rolled out from two starting charges.

Model

Each mode (`drive`, `science`, `comms`, `survival`) is a small co-design `System` that, given a phase capability demand, sizes a shared power bus and returns two outer resources: `energy_Wh` (subtracted from the battery) and a `cost` proxy that adds an opportunity penalty `BEST - value` for the mission value *not* delivered, kept non-negative as the resource poset requires. The outer object is a finite-horizon dynamic program over six phases: the per-phase decision is the mode; the per-phase cost is read from that mode's co-design solve; and the battery state of charge $s \in \mathbb{R}_{\geq 0}$ (Wh) is carried by the transition $s' = \min(\text{cap}, s - \text{spent} + \text{solar})$, admissible while $s \geq 0$. A heavy-drawing mode buys capability now at the cost of charge later, so the optimal policy threads mode activations through the evolving state of charge, standard spacecraft power-mode practice.

Code

```
1 policy = solve_dynamic(  
2     stages, grid,  
3     cost_fn=mision_cost,          # point → energy + opportunity penalty  
4     terminal_cost=terminal_reward, # small credit for leftover charge  
5 )  
6 # One policy, valid for any initial charge on the grid; roll it out twice.  
7 full = rollout(policy, stages, BATTERY_CAPACITY_WH) # 300 Wh  
8 low  = rollout(policy, stages, 90.0)                # depleted start
```

Output

```
Start at full charge (300 Wh):  
  phase_1 → science   soc_in= 300.0 Wh ... soc_out= 245.0 Wh  
  ...  
  phase_5 → science   soc_in=  80.0 Wh ... soc_out=  25.0 Wh  
  phase_6 → comms     soc_in=  25.0 Wh ... soc_out=  10.0 Wh  
  total cost = 727.50, feasible = True  
  
Start at low charge (90 Wh):  
  phase_1 → comms     soc_in=  90.0 Wh ... soc_out=  75.0 Wh  
  ... (comms held every phase) ...  
  total cost = 918.00, feasible = True
```

Reading

The same policy, read at different starting charges, produces different module-activation schedules. From a full battery the rover runs high-value science until reserve falls, then steps down to comms for the last phase; starting depleted it can never afford science's draw and holds comms throughout, so it sheds load exactly as a power-constrained mission would. The higher total cost from the depleted start (918.00 against 727.50) is the accumulated opportunity penalty of the deferred objectives, the price the DP pays to keep the battery non-negative.

15.9 Example 20: antichain-valued sequential co-design

`20_sequential_codesign.py`. The sequential layer of section 15.3: the value at each stage and state is a whole Pareto front of cumulative resource totals, not one number. Its antichain-valued Bellman recursion is `solve_sequential` (13.5.3), and the $(H1)/(H2)$ guard is `check_monotonicity` (13.5.3).

Model

A multi-leg survey mission of four legs. At each leg the operator picks one of three incomparable modes, spanning a cost/CO₂ trade (**eco**: low CO₂, high cost; **balanced**; **rapid**: low cost, high CO₂), each drawing down a shared energy budget carried between legs. Because cost and CO₂ are incommensurable, the value of a whole-mission plan is not a single number but a Pareto front of cumulative (cost, CO₂) totals, accumulated on two fixed axes by `sum_combine`, while the carried energy is a separate state axis read off the chosen point (admissible while non-negative). The framework guarantees the value front equals the reachable frontier: every returned point is achievable by some feasible mode sequence, and no non-dominated achievable total is missed.

Code

```
1 res = solve_sequential(  
2     stages, grid,  
3     cost_axes=["cost", "co2"],      # the two accumulated resource axes  
4     initial_state=ENERGY_CAPACITY,  # carried scalar budget (30 units)  
5     combine=sum_combine,            # totals add across legs  
6 )  
7 rep = check_monotonicity(stages, grid, cost_axes=["cost", "co2"])  
8 # rep.monotone_value_guaranteed is True: (H1) and (H2) hold.
```

Output

```
Whole-mission Pareto front of (cost, CO2) totals (9 points):  
cost=   8.0  co2=  36.0      <- all rapid: cheapest, dirtiest  
cost=  12.0  co2=  31.0  
...  
cost=  20.0  co2=  21.0      <- interior mixes of modes  
...  
cost=  40.0  co2=   4.0      <- all eco: cleanest, costliest  
  
Monotonicity guard: MonotonicityReport(H1=ok, H2=ok,  
                                     value_monotone=guaranteed)
```

Reading

The solver returns the exact reachable frontier of whole-mission plans: nine incomparable points running from all-rapid (8.0, 36.0) through all-eco (40.0, 4.0), with interior mixes between. Each point is a different per-leg mode schedule. The front's size is the width of the reachable set; with a summed objective on two fixed axes it grows polynomially, here linearly, in the horizon, not exponentially, which is why it stays small and legible. The passing $(H1)/(H2)$ guard certifies the value is monotone in the carried budget: more energy never shrinks the achievable front.

15.10 Example 21: reconfigurable robot with a state vector

`21_reconfigurable_robot.py`. The vector-state layer of section 15.4: the carried state is not a scalar but a vector. It exercises `solve_vector_sequential` (13.6.2) over a `VectorStateGrid` (13.6.1), guarded by `check_vector_monotonicity` over the product order.

Model

A field robot reconfigures between three morphologies across five legs: **tracked** (low energy, wears the track module fast), **wheeled** (wears the wheel module fast, spares the track), and **hybrid** (splits the load, wears both a little). Each morphology is a co-design `System` emitting

(energy,ops) plus the per-module wear increments it applies this leg. The carried state is a three-axis vector: per-module wear for the two independent drive modules plus a shared energy budget, advanced by $\text{wear}' = \text{wear} + \Delta\text{wear}$ and $\text{energy}' = \min(\text{cap}, \text{energy} - \text{draw} + \text{solar})$, admissible while each wear stays under its limit and energy stays non-negative. Cost (energy and an operations penalty) accumulates on two axes; the value is therefore an antichain over the cost axes parametrised by the state vector, which a scalar DP could not represent.

Code

```

1  grid = VectorStateGrid([
2      ContinuousAxis("track_wear", 0.0, WEAR_LIMIT, 11),
3      ContinuousAxis("wheel_wear", 0.0, WEAR_LIMIT, 11),
4      ContinuousAxis("energy", 0.0, ENERGY_CAPACITY, 13),
5  ])
6  res = solve_vector_sequential(
7      stages, grid, cost_axes=["energy", "ops"],
8      initial_state={"track_wear": 0.0, "wheel_wear": 0.0,
9                    "energy": ENERGY_CAPACITY},
10     combine=sum_combine,
11 )

```

Output

```

grid size: 1573 state nodes

Mission Pareto front of (energy, ops) totals (5 points):
  energy= 13.0  ops= 17.0
  energy= 14.0  ops= 16.0
  energy= 15.0  ops= 15.0
  energy= 16.0  ops= 14.0
  energy= 17.0  ops= 13.0

Vector monotonicity guard: VectorMonotonicityReport(
  H1=ok, H2=ok, value_monotone=guaranteed)

```

Reading

The product grid of $11 \times 11 \times 13 = 1573$ state nodes carries the full three-axis state. The mission front is a five-point (energy,ops) staircase, each point a different morphology schedule: the low-energy end leans on the wheeled morphology, and the low-ops end mixes morphologies to spread wear so neither drive module hits its limit and forces an expensive fallback. This is the structured multi-component state of the Formula 1 seasonal problem (15.12) in a robotics setting. The passing vector guard certifies the value is monotone over the product order: more spare wear budget and more energy never shrink the achievable front.

15.11 Example 22: online feedback co-design

22_online_feedback_codesign.py. The closed-loop layer of section 15.6: no offline plan is followed; each step is solved against what was actually measured. It exercises run_online_codesign (13.6.3), whose audit trail is a list of ControlStep (13.6.3) records.

Model

A solar-powered environmental sensor node runs in the field over twelve control steps. At each step it senses its true battery charge, reads the live data-rate demand and environment, re-solves

its co-design at those measured conditions, applies the cheapest feasible configuration, and steps the true plant. The node has three configurations, each a co-design `System` sizing radio and compute against a demanded rate and returning (energy,ops): `low_power` (always feasible), `nominal`, and `high_rate` (feasible only at a high enough rate and when charge allows). The measured charge enters the solve as functionality, so an energy-hungry configuration whose draw would exceed the available charge is simply infeasible, which is how the feedback path gates the co-design. This is control co-design (CCD, [25]) in its nested, myopic form: re-solve one static co-design each step. A storm at steps 4–7 raises the demand and the day/night cycle changes solar recharge; the loop adapts to both.

Code

```
1 result = run_online_codesign(
2     CONFIGS, n_steps=N_STEPS,
3     sensor=sensor,           # reads true charge back (closes the loop)
4     requirement=requirement, # {"rate": demand, "charge": measured_soc}
5     environment=environment, # solar input this step
6     plant=plant,            # steps the true battery, clips to [0, cap]
7     cost_fn=cost_fn,        # minimise ops cost
8     initial_state=BATTERY_CAPACITY,
9 )
```

Output

```
Closed-loop audit trail:
  t  soc_in  rate  solar  config    energy  ops
  3   100.0    3   14  nominal    11.0   3.0
  4   100.0    8   12  high_rate   34.0   1.0
  5    78.0    9   10  high_rate   37.0   1.0
  6    51.0    9    8  high_rate   37.0   1.0
  7    22.0    7    8  nominal    19.0   3.0
  8    11.0    3   10  nominal    11.0   3.0
  9    10.0    3   12  low_power    5.0   5.0
 10    17.0    3   14  nominal    11.0   3.0
total ops cost = 32.0, feasible = True
```

Reading

The node escalates to `high_rate` through the storm (steps 4–7) when the demanded rate justifies it. The closed loop then reads the depleting night-time charge and the re-solve gates the hungry configuration out: at step 9 charge has fallen to 10.0 and only `low_power` is feasible, before solar recovers and the node climbs back to `nominal`. No offline plan is consulted, and because the plant is stepped with the true process rather than a nominal transition, any divergence between plan and execution is absorbed by the next sensing step, which is what distinguishes feedback from open-loop replay. Two natural extensions, receding-horizon lookahead with the vector DP and online learning of the model (section 9), are left for later increments.

15.12 Example 23: hierarchical co-design for a Formula 1 season

`23_formula1_season.py`. A faithful reproduction, in the framework’s vocabulary, of the seasonal co-design of Neumann, Habermacher, Fieni, Cerofolini, Zardini, and Onder [23], and the canonical instance of the precompute-then-DP structure of section 15.5. Layer one freezes a `CatalogDP` (13.2.1) with `precompute_catalog` (13.5.3); layer two is a scalar-maximisation season DP built on `solve_dynamic` (13.5.2). A heavy runner.

Model

The two layers are explicit. The *race-level co-design* (layer one) is a genuine MCDP solve: for each track, battery size, and discretised incoming battery age, a **CatalogDP** over energy-deployment strategies emits a Pareto front of (race_time, wear), since deploying more electrical energy lowers race time but ages the battery faster, and an aged pack cannot reach the low race times a fresh one can. **precompute_catalog** freezes those fronts once. The *season-level dynamic program* (layer two) is a scalar-maximisation MDP carrying the state vector $(w_{b,1}, w_{b,2}, x_{ex})$, the fractional wear of the two regulation-permitted battery units plus a flag recording whether a replacement penalty has been incurred. At each race the controls are which unit to run, which frozen deployment strategy to select, and whether to install a fresh unit (a grid penalty of ten places for the first replacement, five for each later one, per the modelled FIA rules). Race time maps to a finishing position through an empirical time-gap model and a probabilistic grid-start correction scaled by the track's overtaking difficulty, and the finishing-position distribution is integrated against the FIA points table to give expected championship points; the DP maximises the season total by backward induction. No co-design solve happens inside the season sweep, which is what distinguishes this from the re-solving **sequential DP**.

Code

```
1 # Layer 1: freeze one (race_time, wear) front per track x battery x age.
2 catalog = precompute_catalog(
3     tracks, battery_sizes, age_buckets, deploy_strategies)
4 # 112 fronts = 8 tracks x 2 batteries x 7 age buckets.
5
6 # Layer 2: scalar-max season DP over the frozen catalog, carrying
7 # (w_b1, w_b2, x_ex); no co-design solve inside the season sweep.
8 season = solve_season(catalog, races, points_table)
```

Output

```
112 race fronts precomputed (8 tracks x 2 batteries x 7 ages).
Optimal season expected points: 134.3
Optimal per-race policy:
  race      unit batt  deploy    time repl grid E[pts]
  Monza          1  4MJ deploy_1.00  5024.0  yes  13  15.7
  ...
  Zandvoort     1  4MJ deploy_1.00  5191.2   no   3  15.9
Finding 1 (local penalty, global gain): 2 replacement(s).
Finding 2: forward total = 134.3, reversed total = 134.8
  forward replaces at ['Monza', 'Spa']; reversed at ['Spa', 'Monza'].
Rendered paper-analogue figures (outputs/f1_paper_fig1..3.png)
```

Reading

Two of the paper's findings are reproduced. First, the optimal policy accepts a *local penalty for a global gain*: it takes a battery replacement at one race, sacrificing points that day (a worse grid slot), to keep a fresh, low-wear pack available for aggressive deployment across the remaining races, raising the season total to 134.3. Second, race order shifts the optimal policy: because the grid-penalty cost is track-dependent, reversing the calendar moves the optimal replacement onto a cheaper track, so the attainable total is near-invariant (134.8 against 134.3) while the replacement set reorders, the temporal coupling the paper emphasises. The season DP is validated against exhaustive brute-force enumeration in **tests/test_formula1.py**; the DP optimum matches exactly. The example and notebook also regenerate the paper's three key

figures *in the same format* for visual comparison (the race-time-versus-wear fronts, the grid-start position penalty, and the finishing-position density); these now write to the `outputs/` directory as `f1_paper_fig1`, `_fig2`, and `_fig3`. They reproduce the paper’s figure *structure and qualitative behaviour* from the framework’s own co-design solves, not the paper’s specific numbers, since the paper’s fronts come from an optimal-control lap simulation and a fitted position model that are not reproduced here. The sense in which the framework “does the same thing” is that it produces the same structured artefacts feeding the same season-level dynamic program.

15.13 Example 24: catalog-driven car co-design from one table

`24_car_catalog_codesign.py`. The co-design counterpart of the hand-wired car study (14.17): here a single architecture table plus one `build_architecture()` function replaces the three separate `build_ice/build_hybrid/build_ev` builders, and every discrete powertrain choice becomes a `CatalogDP` (13.2.1) entry rather than a bespoke module. It is the whole-vehicle instance of catalog-driven assembly (13.5.3) over a `System` (13.3.2). It is a heavy runner, though it completes in about one second.

Model

A passenger vehicle is decomposed into 22 subsystems (powertrain, chassis, interior and auxiliary). A single 12-row `ARCHITECTURE_CATALOG` drives the whole study: each row pre-selects the discrete powertrain choices (engine, transmission, e-motor topology, boost, battery strategy, charger, drivetrain) for one point on the modern technology spectrum, from pure ICE through mild, full, and plug-in hybrid, range-extender EV, to battery-electric, while the parametric modules size themselves from mission demand. Body, suspension, and tyres stay full `CatalogDPs`, swept one row at a time by `solve_architecture` so that each assembled `System` is single-valued and its constraint network carries only monotone quantities. The outer mission is a compact C-segment target (5 seats, 370 L, 170 km/h, 500 km range, 0–100 in 11.5 s); the outer resources are production cost, curb weight, unified primary energy per 100 km, fuel per 100 km, tailpipe CO₂, and durability. Two coupled feedback cycles are closed by Kleene iteration on the monotone network: the *mass spiral* (every load-bearing module reads the total curb mass, which their own masses in turn determine) and the *energy/battery spiral* (consumption depends on mass, and for a BEV the battery is sized to range × consumption, feeding back into mass). The unified energy metric is primary energy input, which is why an ICE lands near 48 kWh/100 km while a BEV lands near 15: the combustion path discards most of the fuel’s chemical energy as heat.

The data-provenance policy is explicit. A `SOURCES` block at the top of the file footnotes every value that was added or spot-checked against public data (2024–2025): battery pack price (BloombergNEF 2024), pack gravimetric density (a 2025 *World Electric Vehicle Journal* review), fuel lower-heating values and CO₂ factors (UBC and Waterloo notes), engine dry masses, and the Corolla-bracketed mission spec. Values not individually footnoted are kept from the scaffold and lie within the cited published ranges. They are illustrative, not OEM-specific: the MCDP framework is what is being validated; the numbers serve the framework.

Code

```
1 # One 12-row table drives the whole study (name, indices, params):
2 ARCHITECTURE_CATALOG = [
3     ("ICE_economy",    1, 0, 0, 1.4, 1.00, 0.0, 0.0, F, F, F),
4     ("HEV_full",      2, 3, 2, 1.0, 0.60, 2.0, 0.0, T, F, F),
5     ("BEV_long_range", 0, 4, 4, 1.0, 0.00, 80.0, 150.0, T, F, F),
6     # ... 9 more rows spanning MHEV / PHEV / REEV / diesel ...
7 ]
8
9 def build_architecture(arch, mission, *, body_row, susp_row, tire_row):
```

```

10 sys = System(arch["name"])
11 sys.add("engine", make_engine_dp([eng])) # 1-entry CatalogDP slice
12 # ... 21 more modules ...
13 # Mass spiral: load-bearing modules read the converged curb mass.
14 sys.constrain("susp_front.axle_load_kg", lambda x: 0.55*total_mass(x))
15 sys.constrain("brakes_front.vehicle_mass_kg", total_mass)
16 # Energy/battery spiral: BEV pack sized to range x consumption.
17 sys.constrain("battery.target_battery_kWh", battery_target_kWh)
18 return sys.build() # Kleene iteration closes both cycles

```

Output

```

--- Per-architecture feasibility (cheapest feasible design) ---
Architecture      cls  feas  cost$  mass kg  kWh/100  L/100  CO2
ICE_economy        ICE  yes   17,447   930    47.8    5.3    123
Diesel_long_range  ICE  yes   24,400  1081    40.6    4.1    109
HEV_full           FHEV yes   26,945  1315    42.2    4.1    94
PHEV_small         PHEV yes   36,162  1537    34.9    3.1    71
REEV               REEV  yes   37,182  1613    21.6    1.1    26
BEV_long_range     BEV  yes   44,572  1788    14.8    0.0    0
BEV_AWD_perf       BEV  yes   68,076  2517    17.0    0.0    0

--- Pareto front over (cost, energy/100km, mass) ---
ICE_economy        $ 17,447   47.8 kWh/100   930 kg
Diesel_long_range  $ 24,400   40.6 kWh/100  1081 kg
PHEV_small         $ 36,162   34.9 kWh/100  1537 kg
REEV               $ 37,182   21.6 kWh/100  1613 kg
BEV_long_range     $ 44,572   14.8 kWh/100  1788 kg

```

Reading

All twelve architectures are feasible for the compact mission, and the cheapest-feasible sweep recovers the expected technology ordering: the ICE economy car is lightest (930 kg) and cheapest (\$17,447) but thirstiest (47.8 kWh/100 km, 123 g/km CO₂), while the battery-electric long-range car is the most efficient (14.8 kWh/100 km, zero tailpipe CO₂) at the highest mass (1788 kg) and near the highest cost. Between them the hybrids and the range-extender trade cost against energy and CO₂ monotonically, and the mass climb from ICE to BEV is the visible signature of the energy/battery spiral: a BEV's pack is sized to range $\times$ consumption, and the extra pack mass raises consumption, which the Kleene loop resolves to a fixed point. The five-point Pareto front over (cost, energy, mass) keeps only the non-dominated members, the same structured artefact the temporal examples consume, produced here from a single declarative table rather than three hand-written builders.

15.14 Example 25: online co-design paper benchmarks

`25_online_paper_benchmarks.py`. A statistical replication of the synthetic benchmarks of Alharbi, Dahleh, and Zardini, “Compositional Online Learning for Multi-Objective System Co-Design” (arXiv:2604.22624) [3], the paper behind the online-learning layer of section 9. It drives the library's optimistic evaluators, `MonotonicityEvaluator` (13.4.2) and `LipschitzEvaluator` (13.4.2), whose `bound()` reproduces the paper's eqs. (23)–(25); the outer sampling loop is written out here rather than delegated to `solve_online` (13.4.2) because the library *scans* a candidate list with an upper-confidence picker whereas the paper *samples* a base measure with rejection. A heavy runner (about 6 s with the EA panel).

Model

The paper studies online multi-objective decision-making in monotone co-design and recovers the target-feasible antichain of non-dominated resources using as few expensive evaluations as possible. Its engine is Algorithm 1 (Rejection Sampler with Optimistic Evaluators): draw a candidate from a low-discrepancy Halton base measure, compute history-dependent optimistic bounds, and skip it without evaluation when either (13) its optimistic resource is already dominated by the incumbent antichain, or (14) its optimistic functionality can never meet the target. Only survivors consume the budget. Two synthetic families of Section VII-B are replicated: **E1/Monotone** ($g : [0, 1]^3 \rightarrow [0, 1]^2$, a weighted sum of thresholded indicators; the monotone join bound) and **E2/Lipschitz** ($g : [0, 1]^4 \rightarrow [0, 1]^2$, a triangle wave through an $L=2$ linear map; the Lipschitz cone bound). The score is the cumulative hypervolume difference (HVD), $\sum_t [\text{HV}(A_{\text{ref}}) - \text{HV}(\hat{A}_t)]$, of the incumbent antichain against a reference point $(1, 1)$; lower is better and $\text{HVD} \rightarrow 0$ on convergence. Baselines are the same Halton proposals with no elimination, uniform random draws over the shared discretised pool, and an optional pymoo EA panel (NSGA-III / MOEA/D / RVEA).

Two deviations from the paper are documented in the source and preserved here. First, taken literally the monotone family is degenerate (an unconstrained monotone g is minimised at the box corner, so there is nothing to learn); the example therefore makes the target binding, recovering $\text{FixFunMinRes}(f)$ over the feasible upper set, which exercises *both* elimination conditions and leaves the Lipschitz family untouched. Second, families E3 (intermodal mobility) and E4 (heterogeneous multi-robot) are *not* replicated: their expensive blocks are external models not present in this repository. Because the paper’s seeds, grid, and atom count are unpublished, the replication is statistical, not bit-exact; only the relative claims are validated.

Code

```

1 # Paper Algorithm 1: rejection sampler with optimistic evaluators.
2 while accepted < budget and ptr < len(pool):
3     i = ptr; ptr += 1
4     if front and rng.random() >= delta:                # else force-accept
5         lo, hi = evaluator.bound(cand[i])                # optimistic bounds
6         if constrained and (hi[RC[0]] < f[0] or hi[RC[1]] < f[1]):
7             continue                                    # (14): optimistic functionality misses target
8         if _dominated_by_incumbent(lo, front):
9             continue                                    # (13): optimistic resource already dominated
10    r = (float(res[i, 0]), float(res[i, 1]))                # accept: query block
11    evaluator.observe(i, cand[i], _singleton_antichain(r))
12    accepted += 1
13    if feas[i]:
14        front = pareto_insert(front, r)
15    cum += hv_star - hv2d(front, ref)                    # cumulative HVD

```

Output

TABLE I	-- Monotone problems (E1), FixFunMinRes(f)	[M1..M4 shown]
Ours	9.51e+00 7.94e+00 4.85e+00 1.50e+01	
Halton	1.55e+01 1.46e+01 7.29e+00 2.63e+01	
MOEA/D	1.62e+01 1.50e+01 1.78e+01 2.30e+01	

TABLE II	-- Lipschitz problems (E2), pure minimization	[L1..L4 shown]
Ours	1.07e+01 2.56e+01 1.26e+01 9.25e+00	
Halton	1.15e+01 2.75e+01 1.29e+01 1.05e+01	
MOEA/D	9.00e+00 2.98e+01 8.71e+00 1.30e+01	

VALIDATION


```

(a) Ours < Halton on ALL monotone instances
    ours : Ours beats Halton on 8/8 → PASS
(b) Ours < Halton on MOST Lipschitz instances
    ours : Ours beats Halton on 8/8 (need ≥ 6) → PASS
(c) [EA panel] paper: Ours best-or-2nd; MOEA/D blowups on Lipschitz.
    ours : best-or-2nd monotone 8/8, lipschitz 3/8
    ours : Lipschitz MOEA/D max/med=3.3x    Ours max/med=2.3x
    → NOT REPRODUCED at higher statistical strength (WP-P3).

VERDICT: (a) PASS, (b) PASS [elimination-vs-Halton reproduce]
         (c) NOT REPRODUCED -- EA panel.

```

Reading

The replication is deliberately honest about what carries and what does not. Claims (a) and (b) reproduce robustly: the optimistic elimination beats the identical Halton proposals with no elimination on all eight instances of *both* families (here 8/8 and 8/8; even stronger than the paper’s 7/8 under independent re-runs at RUNS=20, ITERS=500, two master seeds and the full EA panel), and Ours is best-of-all on every monotone instance. These are the fair apples-to-apples comparison, since all three methods draw from the same shared discrete pool and differ only by the elimination step. Claim (c) did *not* reproduce: on the smooth Lipschitz family the continuous EAs (MOEA/D in particular) match or beat Ours, and MOEA/D shows no blowups here (max/median $\approx 3.3\times$, not the paper’s $\approx 20\times$ on L8), so it is neither poor nor high-variance in this harness. The leading hypotheses are that the EA panel optimises the *continuous* box while Ours, Halton and Random are confined to the fixed discrete pool (an asymmetry favouring EAs on smooth maps, with EA HVD clipped at zero against a discrete reference they can beat), that the budget is far below paper scale, and that the EAs are untuned (fixed pop = 20). The example prints NOT REPRODUCED rather than a fragile PASS, and mirrors the reasoning in its own *Validation status* docstring; only the elimination-vs-Halton claims are counted as reproduced. Two log-scale HVD-versus-iteration figures are written to `outputs/` (`25_online_monotone.png` and `25_online_lipschitz.png`).

16 Realistic Problem Domains for Temporal Co-Design

The temporal, vector-state, and online layers of section 15 were built with concrete application domains in mind. This section surveys realistic problems, drawn from the recent literature, that map onto these layers, as candidates for future worked examples and as evidence that the abstractions are not invented in a vacuum. Each paragraph names the co-design structure (which layer applies) alongside the domain.

16.1 Self-reconfiguring and modular robots

Self-reconfiguring modular robots deliberately change their morphology, by rearranging the connectivity of their modules, to adapt to terrain, task, or damage [26]: a worm shape for a pipe, legs for rough ground, a wheel for flat terrain. This is directly the temporal switching and vector-state setting: the morphology is the architecture, the terrain or task sequence is the environment, and per-module wear plus shared energy is the carried state vector (example 21 is a first cut). Recent work makes the coupling sharper. Terrain-aware morphology search in dynamic environments [27] chooses a morphology per scene under an energy budget; multi-task space applications map task requirements to modular-unit morphology through an inverse-mapping design step [28]; and reconfigurable cobots such as the CONCERT platform and freeform strut-node systems (FreeSN, SMORES) expose a catalog of admissible configurations with real reconfiguration costs, the switching-cost term of `solve_schedule`. Floor-cleaning tiling robots (hTrihex,

hTetro) reconfigure to cover obstacle-strewn areas at lower energy than fixed-shape coverage planning [29], an energy-versus- coverage Pareto per configuration.

16.2 Programmable self-assembly

Programmable self-assembly designs building blocks with tunable binding interactions so they assemble into a target structure. The design problem is multi-objective, trading assembly *yield*, assembly *speed*, and *economy* (number of distinct particle species), which is exactly a Pareto-front co-design. Hübl and Goodrich show that simultaneously optimising assembly time and yield through the binding-energy and concentration design space can speed assembly by orders of magnitude without lowering yield, and that economical, semi-addressable designs (reusing building blocks) can beat fully-addressable ones on all three axes [30, 31]. The temporal angle is the assembly *protocol*: a staged schedule of temperature or concentration set-points is a sequence of co-design decisions coupled by the carried state of intermediate-species concentrations, a natural fit for the vector-state sequential DP, with Markov-state-model protocol optimisation [32] as the dynamic backbone.

16.3 Multi-objective protein design

Protein design inherently trades competing objectives, stability, function, solubility, immunogenicity, and manufacturability, and Pareto-based multi-objective sampling produces candidate sets spanning these tradeoffs for informed downstream selection [33]. This is a co-design antichain over design objectives. Recent methods make the front explicit: ST-PARM aligns inference-time generation to a controllable stability-versus-function tradeoff; MosPro and NSGA-2 variants produce Pareto-optimal sequence sets; MAProt negotiates a structure-versus-function consensus across agents. The temporal and online angle is closed-loop directed evolution: frameworks such as EVOLVEpro couple a generative or predictive model with iterative wet-lab feedback, proposing a batch, measuring it, and updating, which is precisely the online feedback co-design loop of section 15.6 with the measured functionality replacing the modelled one (the model- learning extension of that loop).

16.4 Control co-design (CCD)

Control co-design, the simultaneous optimisation of a physical plant and its controller, is an established engineering discipline, described by Garcia-Sanz as a game changer [25], with two standard paradigms: nested (an outer plant loop wrapping an inner control solve) and simultaneous [24]. The nested paradigm is exactly the structure of the online co-design loop. Concrete instances that map onto the temporal and online layers include robust control co-design with a receding-horizon MPC inner loop [34]; plant-and-controller optimisation for energy systems with MPC [35]; control co-design of spacecraft trajectory and rocket-engine design in NASA’s OpenMDAO/GMAT tooling; digital-twin control co-design of active suspensions via reinforcement learning; and adaptive MPC with online recursive-least-squares identification for load-frequency control under renewable disturbances, the archetype of the model-learning extension. These supply both the vocabulary (plant, controller, nested vs simultaneous) and the benchmark problems (active suspension, hybrid-electric vehicle, airborne wind energy) for positioning the online layer.

16.5 Summary of the mapping

Across these domains the same three structural questions recur, and the package answers each: which architecture to run as conditions change (switching, **temporal**); how a resource or wear vector carried across stages shapes the optimal schedule (vector-state DP, **vector_dp**); and how

to re-decide the design online from measurements (feedback, `online_codesign`). The reconfigurable robot (21) and adaptive sensor node (22) are first worked examples in the robotics and control-co-design cells of this table; self-assembly and protein design are identified as high-value future examples whose multi-objective, staged-protocol, and closed-loop-evolution structure the existing layers already support.

17 Modelling Guidelines and Pitfalls

A few patterns that come up repeatedly when authoring MCDPs.

Expose loop variables you want to inspect. When using the operator API directly, `Loop` projects its axis out of the outer R . To inspect the converged loop value, include it in the inner R under a *different name*: for example, both `battery_mass` as the loop axis and `report_mass` mirrored for outer- R visibility. The `System` builder handles this transparently.

Cap physical maxima. When a design variable has a physical ceiling ($v_{\max}$, $r_{\max}$), make h return a $\top$ -valued antichain whenever the iterate exceeds it. The Kleene ascent will then converge to infeasibility rather than oscillating or diverging.

Use `FunctionDP` or `CatalogDP` for breadth. `AlgebraicDP` always returns a singleton antichain. For a genuine Pareto front, use `FunctionDP` (enumerate the tradeoffs explicitly) or `CatalogDP` (provide multiple incomparable entries).

Scalarisation is downstream. The MCDP solver returns the Pareto front. Choosing which point to ship is a separate concern: apply `minimize_cost` with a scalar objective, or inspect the front and pick by hand.

Catalog Cartesian product can blow up. When several subsystems return multi-valued antichains, the inner h of the `System` takes their Cartesian product. For typical models (one or two catalog subsystems among many algebraic ones), this is not a bottleneck; but if many subsystems are catalog-driven, the inner antichain may grow large between `Min` prunings.

Numerical precision in feedback loops. `Antichain.eq` compares points exactly. For contractive floating-point feedback, the iteration may oscillate by an ulp near the fixed point and consume many iterations before converging. The divergence cap and `max_iter` guard against runaway, but choosing `max_iter` generously (200 to 500) is wise for nontrivial models.

18 Limitations and Future Work

Version 0.2.0 covers the algorithmic core: posets, antichains, six primitive DP types, three composition operators, the Kleene solver, and the two builders. The following are intentional omissions, all tractable extensions:

- **MCDPL surface syntax.** The paper’s `mcdp { ... }` text format is not parsed. The MCDP builder gives the same shape in Python.
- **Non-finitely-representable antichains.** The library works on finite, exactly-represented antichains. For relations whose Pareto front is continuous, the only support is the bracket pattern of `UncertainDP`. Approximation kernels (Sec. VII of Censi 2015) are not yet implemented.

- **Visualisation of the design graph.** Example 5 visualises the Kleene ascent over $\mathbb{N} \times \mathbb{N}$, but there is no general renderer for the operator tree or for the subsystem network of a `System`.
- **Symbolic/automatic differentiation of h .** All relations are evaluated numerically.
- **Distributed or parallel solving.** The Kleene iteration is sequential.

Contributions, particularly on these items, are welcome.

19 API Reference Summary

The public API exported by `import codesign` is summarised below.

Posets. `Reals`, `Naturals`, `Ports`, `Discrete`, abstract base `Poset`.

Antichains. `Antichain`.

Design problems. `DesignProblem` (abstract base), `AlgebraicDP`, `FunctionDP`, `CatalogDP`, `CatalogEntry`, `ConstraintDP`, `ODE_DP`, `UncertainDP`.

Composition. `Series`, `Parallel`, `Loop` (classes); `series`, `par`, `loop` (function aliases).

Primitives. `adder`, `multiplier`, `scale`, `constant`, `identity`.

Solver. `solve`, `kleene_loop`, `minimize_cost`, `SolveResult`, `TraceEntry`. Warm-start via the `start_from=` argument on `solve`.

Builders. `MCDP`, `System`.

Class-based modules. `Module` (declarative `DesignProblem` base class).

Constraint DSL. `Port`, `Expr`, `ModuleHandle` (returned by `System.add`), and the helper functions `sqrt`, `exp`, `log` for use inside constraint expressions.

Uncertainty. Set-based: `UncertaintySet` (abstract), `Box`, `Ellipsoid`, `Disk`, `Circle`. Stochastic: `Stochastic`, plus copulas `Independence` and `GaussianCopula`. Result: `UncertaintyResult`. Entry point: `solve_with_uncertainty` (the same machinery is also reachable through `solve(dp, f, uncertainty=[...])`).

Online learning. `OptimisticEvaluator` (abstract); `MonotonicityEvaluator`, `LipschitzEvaluator`, `LinearParametricEvaluator`. Result: `OnlineResult`. Entry point: `solve_online`.

Visualisation. Imported as from `codesign import viz`. Functions: `plot_antichain`, `plot_convergence`, `plot_uncertainty`, `to_dot`.

Module-level. `__version__` (the string "0.2.0").

References

The framework references [1]–[4] underpin the theory and software. Entries [5]–[11] are the parameter-calibration sources for the monoclonal-antibody bioprocess examples (15 and 16); entries [12]–[22] are the calibration sources for the full-vehicle co-design example (17); entries [23]–[35] underpin the temporal, vector-state, and online co-design layers (sections 15–16, examples 18–22) and the realistic problem domains surveyed there. All calibration values are illustrative, drawn from the published ranges in these sources, and are not OEM- or product-specific.

How to cite

If you use `codesign-mcdp` in academic work, please cite it. A machine-readable `CITATION.cff` file at the repository root lets GitHub’s “Cite this repository” button generate a citation automatically. In BibTeX:

```
@software{briat_codesign_mcdp,  
  author = {Briat, Corentin},  
  title  = {codesign-mcdp: A Python Library for  
           Monotone Co-Design Problems},  
  year   = {2026},  
  version = {0.2.0},  
  url    = {https://github.com/cbriat/codesign-mcdp}  
}
```

Please also cite the underlying theory, Censi (2015), listed as reference [1] under *Framework and theory* below.

Framework and theory

- [1] Andrea Censi. *A Mathematical Theory of Co-Design*. arXiv:1512.08055, 2015. <https://arxiv.org/abs/1512.08055>
- [2] B. A. Davey and H. A. Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 2nd ed., 2002.
- [3] Saud Alharbi, Munther A. Dahleh, and Gioele Zardini. *Compositional Online Learning for Co-Design*. arXiv:2604.22624, 2026. <https://arxiv.org/abs/2604.22624>
- [4] The MCDPL software distribution. <https://co-design.science/software/>

Bioprocess examples (15, 16)

- [5] S.K. Yoon, S.H. Kim, and G.M. Lee. Effect of low culture temperature on specific productivity and transcription level of anti-4-1BB antibody in recombinant Chinese hamster ovary (CHO) cells. *Biotechnology Progress* 19:1383–1386, 2003.
- [6] S.N. Sou, C. Sellick, K. Lee, et al. How does mild hypothermia affect monoclonal antibody glycosylation? *Biotechnology and Bioengineering* 112(6):1165–1176, 2015.
- [7] E. Trummer, K. Fauland, S. Seidinger, et al. Process parameter shifting: Part I. Effect of DOT, pH, and temperature on the performance of Epo-Fc expressing CHO cells cultivated in controlled batch bioreactors. *Biotechnology and Bioengineering* 94(6):1033–1044, 2006.
- [8] S.F. Khattak, Z. Xing, B. Kenty, I. Koyrakh, and Z.J. Li. Feed development for a fed-batch CHO production process by semi-steady-state analysis. *Biotechnology Progress* 26(3):797–804, 2010.

- [9] M. Gagnon, G. Hiller, Y.-T. Luan, A. Kittredge, J. DeFelice, and D. Drapeau. High-end pH-controlled delivery of glucose (HIPDOG) effectively suppresses lactate accumulation in CHO fed-batch cultures. *Biotechnology and Bioengineering* 108(6):1328–1337, 2011.
- [10] D. Reinhart, L. Damjanovic, C. Kaisermayer, W. Sommeregger, A. Gili, B. Gasselhuber, et al. Bioprocessing of recombinant CHO-K1, CHO-DG44, and CHO-S: CHO expression hosts favor either mAb production or biomass synthesis. *Biotechnology Journal* 14(3):e1700686, 2019.
- [11] C. Lao and D. Toth. Effects of ammonium and lactate on growth and metabolism of a recombinant CHO cell culture. *Biotechnology Progress* 13:688–691, 1997.

Automotive co-design example (17)

- [12] G. Genta and L. Morello. *The Automotive Chassis, Vol. 1: Components Design*. Springer, 2009. (Suspension stiffness, damper characteristics, brake sizing, tire load ratings.)
- [13] Robert Bosch GmbH. *Bosch Automotive Handbook*, 10th ed., 2018. (Engine specific power, exhaust aftertreatment, electrical loads.)
- [14] W.W. Pulkrabek. *Engineering Fundamentals of the Internal Combustion Engine*, 2nd ed. Pearson, 2003. (Thermal efficiency, heat rejection, fuel flow rates.)
- [15] J.B. Heywood. *Internal Combustion Engine Fundamentals*, 2nd ed. McGraw-Hill, 2018. (Combustion thermodynamics, BSFC ranges, durability.)
- [16] P. Hofmann. *Hybridfahrzeuge*, 2nd ed. Springer, 2014. (Power-split topologies, sizing rules, efficiency models.)
- [17] H. Naunheimer et al. *Automotive Transmissions*, 2nd ed. Springer, 2011. (Gearbox losses, weights, costs.)
- [18] International Energy Agency. *Global EV Outlook*, 2023. (Battery pack cost and pack-level energy-density trends, 2020–2030.)
- [19] F. Nemry et al. *Environmental Improvement of Passenger Cars*. JRC Scientific and Technical Reports, 2008. (CO₂ emission factors, fuel densities, well-to-wheel.)
- [20] U.S. Environmental Protection Agency. *Automotive Trends Report*, 2024. (Fleet-average fuel economy, weight-class data, drag-coefficient data.)
- [21] J. Larminie and J. Lowry. *Electric Vehicle Technology Explained*, 2nd ed. Wiley, 2012. (Motor efficiency maps, inverter losses, charging-system architecture.)
- [22] P. Mock et al. *European Vehicle Market Statistics Pocketbook 2023/24*. International Council on Clean Transportation, 2023. (Mass-versus-mission scatter data for calibration.)

Temporal, vector-state, and online co-design (18–22, sections 15–16)

- [23] M.-P. Neumann, R. Habermacher, G. Fieni, A. Cerofolini, G. Zardini, and C.H. Onder. Hierarchical Co-Design for Multi-Race Strategy Optimization in Formula 1. In *29th IEEE Intelligent Transportation Systems Conference (ITSC)*, 2026 (in press). ETH Research Collection <https://doi.org/10.3929/ethz-c-000784888>. (The precompute-then-DP structure and the multi-component seasonal state vector.)

- [24] D. R. Herber and J. T. Allison. Nested and Simultaneous Solution Strategies for General Combined Plant and Control Design Problems. *Journal of Mechanical Design* 141(1):011402, 2018. (The nested vs simultaneous CCD paradigms.)
- [25] M. Garcia-Sanz. Control Co-Design: An engineering game changer. *Advanced Control for Applications* 1(1):e18, 2019.
- [26] Self-reconfiguring modular robots (overview). See e.g. *Modular reconfigurable robots: Toward on-demand multifunctional applications*, Science Robotics, 2024, and the survey *Modular Self-Configurable Robots—The State of the Art*, *Actuators* 12(9):361, 2023.
- [27] Z. Liu, Q. Lu, J. Luo, and Z. Wang. Terrain-aware morphology searching algorithm for self-reconfigurable modular robot in dynamic environment. *Applied Soft Computing* 186:114182, 2026.
- [28] X. An, Q. Jia, G. Chen, Y. Liu, and H. Liu. Morphological Design and Performance Analysis of a Self-reconfigurable Modular Robot for Multi-task Space Applications. *Journal of Mechanical Engineering* 61(21):152–167, 2025.
- [29] Modular Self-Configurable Robots—The State of the Art. *Actuators* 12(9):361, 2023. (Floor-cleaning tiling robots hTrihex/hTetro and reconfiguration for coverage.)
- [30] M. C. Hübl and C. P. Goodrich. Simultaneous optimization of assembly time and yield in programmable self-assembly. *The Journal of Chemical Physics* 164(8):084904, 2026.
- [31] M. C. Hübl and C. P. Goodrich. Accessing Semi-Addressable Self-Assembly with Efficient Structure Enumeration. *Physical Review Letters* 134(5):058204, 2025.
- [32] Optimization of Non-Equilibrium Self-Assembly Protocols Using Markov State Models. arXiv:2210.05749. (Staged assembly-protocol optimisation.)
- [33] Multi-objective protein design (representative works): *Searching for the Pareto frontier in multi-objective protein design*, *Biophysical Reviews* 9:311–324, 2017; MosPro, *Pareto-optimal sampling for multi-objective protein sequence design*, 2025; ST-PARM and MAProt, bioRxiv, 2026; EVOLVEpro (closed-loop directed evolution).
- [34] J. Sun et al. Robust Control Co-Design with Receding-Horizon MPC. arXiv:2104.02025. (The online-feedback co-design pattern.)
- [35] D. J. Docimo, Z. Kang, K. A. James, and A. G. Alleyne. Plant and Controller Optimization for Power and Energy Systems with Model Predictive Control. *Journal of Dynamic Systems, Measurement, and Control*, 2021.